MATERIAL DE ESTUDOS

Aprendendo programação web

Um curso passo a passo, do zero, construído sobre um projeto real: o Vault Keeper.

REACT · TYPESCRIPT · SUPABASE · GIT · DEPLOY

19 capítulos com exercícios, roteiro de reconstrução e glossário · Outubro de 2026

Sumário

COMECE AQUI

Como estudar (leia primeiro) 3

BASE

00 Visão geral: o que construímos e como as peças se encaixam 6

01 Ferramentas: terminal, Node, npm, Vite e IntelliJ 11

02 JavaScript essencial: a parte que o projeto realmente usa 17

FRONT-END

03 TypeScript: dando forma aos dados 25

04 React: fundamentos 31

05 React: estado, efeitos e hooks 38

06 Rotas e organização do código 46

07 CSS moderno 51

DADOS

08 Banco de dados e Supabase 60

09 Ligando o app ao banco 70

QUALIDADE

10 Lógica pura e testes 80

11 Git e GitHub 87

NO AR

12 Build e deploy: do seu computador ao mundo 94

13 Segurança de um app web 100

14 Acessibilidade e qualidade de código 108

PRÁTICA

15 Reconstruindo do zero: o roteiro passo a passo 115

16 Exercícios e desafios 123

17 Erros e lições do projeto 131

18 Cola rápida e glossário 139

INTRODUÇÃO

Estudos: aprendendo programação web com o Vault Keeper

Este material usa **o seu próprio projeto** (o catálogo de revistas) como livro-texto. Em vez de exemplos abstratos como "Pessoa" e "Carro", cada conceito aparece no código que você já viu funcionando: `MagazineCard`, `applyFilters`, `useSession`, as políticas de segurança do banco, o deploy no Vercel.

Você chegou até aqui construindo. Agora o objetivo é **entender por que cada linha existe** e conseguir criar algo parecido sozinho.

Como estudar (leia isto antes)

1. **Abra o código ao lado.** Cada capítulo cita arquivos (por exemplo `src/lib/filters.ts`). Abra-os no IntelliJ (`⌘⇧O` e digite o nome) e leia o trecho real, não só o que está copiado aqui.
2. **Digite, não copie.** Quando um capítulo pedir um exercício, escreva o código à mão. Copiar e colar não treina o cérebro.
3. **Quebre de propósito.** Troque um valor, apague uma linha, veja o erro. Entender *como* algo falha é metade de entender como funciona.
4. **Faça os exercícios antes de olhar as dicas.** Travar faz parte. Só espie a dica depois de 10 minutos tentando.
5. **Um capítulo por vez, sem pular.** A ordem importa: cada um usa o anterior.
6. **Use o git como rede de segurança.** Antes de experimentar, faça `git status`. Se estragar tudo, `git restore .` volta ao último commit.

> **Dica de ouro:** quando não entender algo, escreva a pergunta com suas palavras ("por que o `useEffect` tem `[]` ?"). Muitas vezes a resposta aparece só de formular a pergunta. Se não aparecer, a pergunta bem feita vira a melhor busca na internet ou o melhor pedido de ajuda.

O índice

#	Capítulo	O que você aprende	Tempo sugerido
00	Visão geral	O que construímos, arquitetura, tecnologias, fluxo dos dados	1 h
01	Ferramentas	Terminal, Node, npm, Vite, variáveis de ambiente, IntelliJ	2 h
02	JavaScript essencial	A parte do JS que o projeto realmente usa	4 h
03	TypeScript	Tipos, interfaces, genéricos, `null`, por que ajudam	3 h
04	React: fundamentos	Componentes, JSX, props, listas, eventos, formulários	4 h

05	React: estado, efeitos e hooks	`useState`, `useEffect`, `useRef`, hooks próprios	5 h
06	Rotas e organização	SPA, React Router, estrutura de pastas	2 h
07	CSS moderno	Variáveis, flex, grid, responsividade, temas	5 h
08	Banco de dados e Supabase	SQL, RLS, views, Storage, autenticação, migrations	6 h
09	Ligando o app ao banco	CRUD, async/await, erros, upload, consistência	4 h
10	Lógica pura e testes	Vitest, TDD, casos de borda	3 h
11	Git e GitHub	Commits, push, histórico, desfazer	3 h
12	Build e deploy	Vite build, Vercel, variáveis, domínios	2 h
13	Segurança	O que protege (e o que não protege) um app web	3 h
14	Acessibilidade e qualidade	HTML semântico, teclado, lint, refatoração	3 h
15	Reconstruindo do zero	Roteiro passo a passo para refazer o app sozinho	20 h
16	Exercícios e desafios	Prática por capítulo e mini-projetos	contínuo
17	Erros e lições do projeto	Os tropeços reais que tivemos e o que ensinam	2 h
18	Cola rápida e glossário	Resumos para consulta e dicionário de termos	consulta

---

Roteiro sugerido: 10 semanas, cerca de 6 a 8 horas por semana

Semana	Capítulos	Entrega da semana
1	00, 01	Você roda o projeto, explica a arquitetura em voz alta e navega pelo terminal sem medo
2	02	Resolve os exercícios de JS sem consultar o capítulo
3	03, 04	Cria um componente novo (`StatCard`) com props tipadas
4	05	Cria um hook próprio (`useLocalStorage`) e usa em dois lugares
5	06, 07	Adiciona uma página nova com rota e um tema de cor novo
6	08	Cria uma tabela nova (`wishlist`) com RLS, num projeto Supabase de teste
7	09	Faz o CRUD completo da `wishlist` no app
8	10, 11	Escreve testes para uma função nova (TDD) e faz 5 commits bem descritos
9	12, 13, 14	Publica uma cópia do app e revisa a segurança com o checklist do capítulo 13
10	15, 17	Reconstrói uma versão enxuta do app do zero, sem olhar o código original

Se estiver cansado de um assunto, **alterne**: um dia de teoria, um dia de prática. O cérebro fixa melhor assim.

Pré-requisitos

- Um computador com Mac, IntelliJ IDEA e o Node.js instalado (você já tem os três).
- Saber o básico de lógica: variáveis, `if`, repetição, funções. O capítulo 02 revisa o que o projeto usa.

- Uma conta no GitHub, no Supabase e no Vercel (você já criou as três).
- **Nenhum** conhecimento prévio de React, TypeScript ou SQL.

O que este material NÃO cobre (de propósito)

- Estado global com bibliotecas (Redux, Zustand). O projeto não precisou.
- Servidores próprios em Node/Express. O Supabase fez esse papel.
- Testes de componentes e de ponta a ponta (citados como próximo passo no capítulo 10).
- Aplicativos de celular nativos.

Esses assuntos ficam melhores depois que a base deste material estiver sólida.

Convenções usadas nos capítulos

- `código` **em linha** é um nome de arquivo, função ou comando.
- Blocos marcados `bash` são comandos para o terminal; um comando por bloco, para você rodar um de cada vez.
- 🗂️ **No projeto** indica onde o conceito aparece no seu código.
- ⚠️ **Cuidado** marca erros comuns, muitos deles cometidos de verdade neste projeto.
- 🧪 **Experimente** é um mini-exercício de dois minutos.
- ✅ **Checklist** fecha o capítulo: se não consegue explicar um item com as próprias palavras, releia a seção.

Bons estudos. Comece pelo capítulo 00.

CAPÍTULO 00 DE 18

Visão geral: o que construímos e como as peças se encaixam

Objetivo

Entender o **mapa completo** antes de entrar em qualquer detalhe: que problema o app resolve, de que partes é feito, que tecnologia faz o quê e como um clique do usuário viaja até o banco de dados e volta.

---

1. O problema que o app resolve

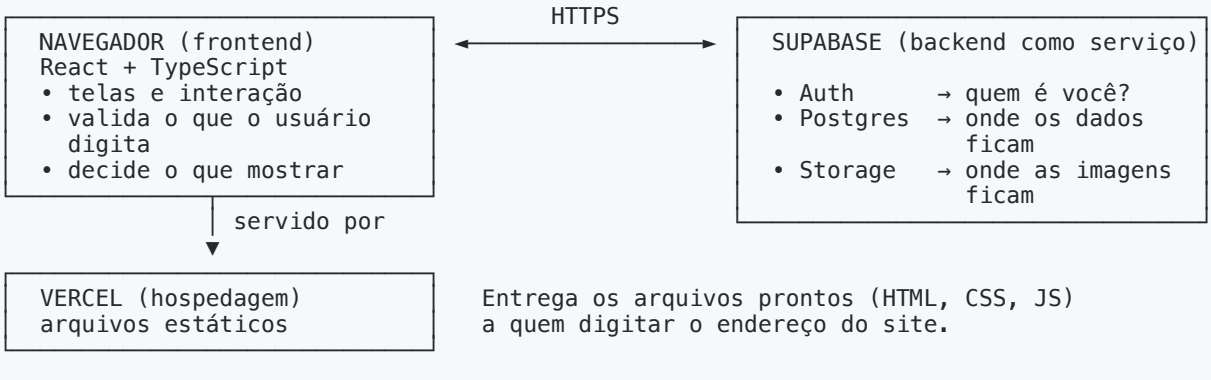
Uma pessoa tem uma coleção de revistas e quer:

1. **Catalogar** cada edição (título, número, mês, ano, condição, capa).
2. Ver tudo com **capas grandes**, em três modos de visualização.
3. **Buscar, filtrar e ordenar**.
4. Mostrar parte do acervo (ou todo) num **catálogo público**, que qualquer visitante vê sem criar conta.
5. Manter o **controle** só para si: só o dono cadastra, edita e apaga.

Quase todo sistema que você vai construir na carreira é variação disto: **CRUD** (Create, Read, Update, Delete) sobre algum tipo de dado, com login, permissões e uma tela bonita por cima.

---

2. As três camadas



Camada	Pergunta que responde	Onde mora no projeto
Frontend	"O que o usuário vê e toca?"	`src/`
Backend (Supabase)	"Onde guardo? Quem pode ver o quê?"	Painel do Supabase + `supabase/migrations/`

Hospedagem (Vercel)	"Como o mundo acessa?"	Painel do Vercel + `vercel.json`

⚠️ **Ideia mais importante do capítulo:** o código do frontend roda **no navegador de quem visita**, e qualquer pessoa pode ler e alterar esse código. Por isso **segurança nunca pode morar só no frontend**. Quem protege os dados é o banco, com o **RLS** (capítulo 08 e 13).

---

3. As tecnologias e por que cada uma está aqui

Tecnologia	O que é	Por que usamos
Node.js	Executa JavaScript fora do navegador	Roda as ferramentas de desenvolvimento (Vite, testes)
npm	Gerenciador de pacotes	Instala as bibliotecas listadas no `package.json`
Vite	Ferramenta de desenvolvimento e empacotamento	Servidor local com recarga instantânea e o `build` final
TypeScript	JavaScript com tipos	Pega erros antes de rodar; documenta o formato dos dados
React	Biblioteca para montar interfaces com componentes	A tela vira peças reutilizáveis que reagem a mudanças de dados
React Router	Troca de páginas sem recarregar	Rotas `/`, `/catalog`, `/public`
CSS puro (com variáveis)	Estilo visual	Você aprende CSS de verdade; temas claro/escuro com poucas linhas
Supabase	"Backend pronto": banco Postgres, login e arquivos	Evita escrever e hospedar um servidor próprio
PostgreSQL / SQL	Banco relacional e sua linguagem	Guarda as revistas com regras de segurança
Vitest	Executor de testes	Garante que a lógica (filtros, leitura de nomes) continua certa
oxlint	Verificador de qualidade de código	Aponta práticas duvidosas
Git + GitHub	Histórico do código e cópia na nuvem	Segurança para experimentar; base da publicação
Vercel	Hospedagem	Publica sozinho a cada `git push`

---

4. Mapa de pastas

```
vault-keeper/
├── index.html                ← a única página HTML; o React "se pendura" aqui
├── package.json              ← lista de bibliotecas e de comandos (scripts)
├── vite.config.ts            ← configuração do Vite
├── tsconfig*.json            ← configuração do TypeScript
├── vercel.json               ← regra de reescrita para as rotas funcionarem na hospedagem
├── .gitignore                ← o que o git NÃO deve guardar (segredos, node_modules)
├── .env.local                ← seus segredos locais (fora do git!)
├── public/                   ← arquivos servidos como estão (ícones da aba)
├── supabase/migrations/      ← mudanças do banco, versionadas em SQL
├── estudos/                  ← este material
└── src/
    ├── main.tsx              ← ponto de entrada: monta o React na página
    ├── App.tsx               ← cabeçalho + rotas + tema
    ├── types.ts              ← o formato de uma Magazine
    ├── index.css             ← variáveis de cor, reset, tema claro/escuro
    ├── App.css               ← estilo de todos os componentes
    ├── pages/                ← uma por rota: Home, Catalog, PublicCatalog, NotFound
    ├── components/           ← peças reutilizáveis: MagazineCard, Modal, MagazineForm...
    ├── hooks/                ← lógica reutilizável com estado: useSession, useTheme, useViewMode
    └── lib/                  ← lógica "pura" e acesso a serviços: filters, parseCoverFilename,
                                magazineService, supabase
```

A regra de organização, em uma frase: cada pasta tem *um tipo de responsabilidade*. Telas inteiras em `pages/`, peças de tela em `components/`, lógica com estado em `hooks/`, lógica sem tela e acesso a serviços em `lib/`. Quando você sabe o que um arquivo faz, sabe onde ele mora.

---

5. A viagem de um clique: "Adicionar revista"

Acompanhe o que acontece quando o dono preenche o formulário e clica em **Adicionar**. Cada passo é um arquivo real do projeto.

```
1. MagazineForm.tsx      handleSubmit() é chamado
   │                     └─ monta o objeto da revista (título, ano, is_public...)
   ▼
2. Catalog.tsx           handleAdd() recebe o objeto e o arquivo da capa
   │
   ├─► 3. magazineService.ts  uploadCover()
   │        └─ envia a imagem ao Storage, na pasta <seu user_id>/
   │        └─ o Supabase CONFERE: a pasta é a sua? (regra do Storage)
   │        └─ devolve o endereço público da imagem
   │
   ├─► 4. magazineService.ts  addMagazine()
   │        └─ INSERT na tabela magazines, com user_id
   │        └─ o Postgres CONFERE: auth.uid() = user_id? (regra RLS)
   │        └─ devolve a linha criada, já com o id
   ▼
5. Catalog.tsx           setMagazines([created, ...magazines])
   │                     └─ o estado muda
   ▼
6. React                 redesenha a tela: o novo card aparece
```

E se algo falhar no passo 4? O `catch` em `handleAdd` apaga a imagem que já subiu no passo 3 (para não deixar lixo no Storage) e mostra a mensagem de erro. **Duas operações que precisam andar juntas exigem cuidado com o meio do caminho.** Você verá isso em detalhe no capítulo 09.

🧪 **Experimente:** abra `src/pages/Catalog.tsx` e `src/lib/magazineService.ts` lado a lado. Localize cada um dos seis passos acima. Se achar todos, você já entende a espinha dorsal do app.

---

6. Os conceitos que atravessam o projeto inteiro

Você vai encontrar estas ideias de novo e de novo, em capítulos diferentes:

Ideia	Em uma frase	Onde você vê
Dados descem, eventos sobem	O pai passa dados e funções para o filho; o filho avisa o pai chamando a função	Props em todos os componentes
Nunca modifique, sempre crie novo	Para mudar uma lista ou objeto, crie uma cópia alterada	`setMagazines([...])`, `filter`, `map`
Estado derivado	O que dá para calcular, não guarde	`visibleMagazines = applyFilters(...)`
Funções puras	Mesma entrada, mesma saída, sem efeitos colaterais	`applyFilters`, `parseCoverFilename`
Camada de serviço	Um lugar só fala com o banco	`magazineService.ts`
Espere o banco antes de mudar a tela	A tela nunca "mente" sobre o que está salvo	`confirmDelete`, `handleTogglePublic`
Frontend é território hostil	Validação no navegador é conforto, não segurança	RLS, validação de upload
Variáveis (tokens) em um lugar	Mudar o visual editando poucas linhas	`:root` em `index.css`

---

7. O que o app faz, funcionalidade por funcionalidade

Funcionalidade	Arquivos principais
Login (só e-mail e senha, cadastro fechado)	`AuthForm.tsx`, `useSession.ts`
Listar a coleção do dono	`Catalog.tsx`, `listMyMagazines`
Criar, editar, apagar revista	`MagazineForm.tsx`, `Modal.tsx`, `ConfirmDialog.tsx`
Capa com prévia e validação	`MagazineForm.tsx`, `uploadCover`
Tornar pública ou privada	`handleTogglePublic`, `setMagazinePublic`
Importar várias capas de uma vez	`BulkImport.tsx`, `parseCoverFilename.ts`
Buscar, filtrar, ordenar	`CatalogFilters.tsx`, `filters.ts`
Três modos de visualização	`ViewModeToggle.tsx`, `MagazineGallery.tsx`, `useViewMode.ts`
Tema claro e escuro	`useTheme.ts`, `index.css`
Catálogo público sem login	`PublicCatalog.tsx`, view `public_magazines`
Aviso de maioridade	`AgeGate.tsx`
Testes	`filters.test.ts`, `parseCoverFilename.test.ts`

✅ Checklist do capítulo 00

- ☐ Explico a diferença entre frontend, backend e hospedagem com minhas palavras.
- ☐ Sei dizer por que a segurança não pode ficar só no frontend.

- ☐ Reconheço o que cada pasta de `src/` guarda.
- ☐ Sei narrar os seis passos do "Adicionar revista".
- ☐ Consigo citar três dos oito "conceitos que atravessam o projeto".

Próximo: 01. Ferramentas.

CAPÍTULO 01 DE 18

Ferramentas: terminal, Node, npm, Vite e IntelliJ

Objetivo

Perder o medo do terminal e entender as ferramentas que ficam *entre* você e o código: o que `npm install` faz de verdade, o que há dentro do `package.json`, o que o Vite entrega e como as variáveis de ambiente funcionam.

---

1. O terminal

O terminal é uma janela onde você **digita comandos em vez de clicar**. Parece antigo, mas é mais rápido, repetível e é como todas as ferramentas de programação funcionam por baixo.

A regra que mais evita erros: o terminal "está em algum lugar"

O terminal sempre está dentro de uma pasta, e o prompt mostra qual:

```
unknown1@UNKNOWN3 vault-keeper %
                  └── pasta atual
```

Comandos do projeto (`npm install`, `npm run dev`, `git status`) só funcionam **dentro da pasta do projeto**. No começo deste projeto, vários erros `ENOENT ... package.json` aconteceram porque o terminal estava em `~` ou em `~/dev`, e o `package.json` está em `~/dev/vault-keeper`. A mensagem de erro dizia exatamente isso: `open '/Users/unknown1/dev/package.json'`. **Ler o caminho no erro resolve metade dos problemas.**

Comandos essenciais

Comando	O que faz	Exemplo
`pwd`	Mostra a pasta atual	`pwd`
`ls`	Lista os arquivos	`ls -la` (inclui ocultos e detalhes)
`cd`	Entra numa pasta	`cd ~/dev/vault-keeper`
`cd ..`	Sobe uma pasta	
`mkdir`	Cria pasta	`mkdir -p estudos`
`cat`	Mostra o conteúdo de um arquivo	`cat package.json`
`cp`	Copia	`cp a.txt b.txt`
`mv`	Move ou renomeia	`mv antigo.txt novo.txt`
`rm`	**Apaga, sem lixeira e sem volta**	`rm arquivo.txt`
`clear`	Limpa a tela	ou `⌘K`

Símbolos úteis: `~` é a sua pasta pessoal; `.` é a pasta atual; `..` é a pasta de cima; `&&` encadeia comandos ("rode o segundo só se o primeiro der certo").

🧪 **Experimente:** no terminal do IntelliJ (`⌥F12`), rode `pwd` e depois `ls`. Depois `cd src` e `ls` de novo. Volte com `cd ..`.

⚠️ **Cuidado com o `rm`:** não existe lixeira. Antes de apagar, rode `ls` para ver o que vai sumir. Dentro de um repositório git, o que está commitado sempre dá para recuperar; o que não está, não.

---

2. Node.js, npm e npx: quem é quem

Nome	O que é	Analogia
Node.js	Programa que executa JavaScript fora do navegador	O "motor"
npm	Gerenciador de pacotes que acompanha o Node	A "loja de peças"
npx	Executa um pacote sem instalá-lo definitivamente	"Experimentar na loja"

Por que precisamos do Node se o app roda no navegador? Porque as **ferramentas de desenvolvimento** (Vite, TypeScript, Vitest) são programas escritos em JavaScript e rodam no Node. O app final roda no navegador, mas *construí-lo* usa o Node.

---

3. O `package.json`: a "receita" do projeto

Abra `package.json`. Ele tem três partes que importam:

```
{
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "oxlint",
    "preview": "vite preview",
    "test": "vitest run",
    "test:watch": "vitest"
  },
  "dependencies": {
    "@supabase/supabase-js": "^2.117.2",
    "react": "^19.2.8",
    "react-dom": "^19.2.8",
    "react-router-dom": "^7.18.4"
  },
  "devDependencies": {
    "typescript": "~6.0.2",
    "vite": "^8.3.0",
    "vitest": "^5.0.3"
  }
}
```

`scripts`: atalhos para comandos longos

Você roda `npm run dev` e o npm executa `vite`. É só um apelido.

Comando	Faz	Quando usar
`npm run dev`	Sobe o servidor local com recarga automática	O dia a dia
`npm run build`	Confere os tipos (`tsc -b`) e gera a versão final em `dist/`	Antes de publicar
`npm test`	Roda todos os testes uma vez	Antes de cada commit
`npm run test:watch`	Fica rodando os testes a cada salvamento	Enquanto escreve código
`npm run lint`	Aponta práticas duvidosas	Antes de commitar

(`npm test` e `npm start` são os únicos que dispensam o `run`.)

`dependencies` × `devDependencies`

- `dependencies`: o app **precisa** no navegador (React, Supabase, Router).
- `devDependencies`: só ajudam **enquanto desenvolve** (TypeScript, Vite, Vitest).

Os símbolos `^` e `~` (versionamento semântico)

Versões têm três números: `MAIOR.MENOR.CORREÇÃO`, por exemplo `19.2.8`.

- `^19.2.8` aceita atualizar para `19.x.x` (menor e correção, nunca o maior).
- `~6.0.2` aceita só `6.0.x` (apenas correções).
- Sem símbolo: exatamente aquela versão.

O número **maior** muda quando algo quebra a compatibilidade; por isso o `^` não pula para ele.

`node_modules` e `package-lock.json`

- `node_modules/`: onde o npm baixa tudo. É enorme e **nunca vai para o git** (está no `.gitignore`). Se apagar, `npm install` recria.
- `package-lock.json`: registra a versão **exata** de cada pacote instalado, para que todos tenham o mesmo ambiente. **Este vai para o git.**

Instalando e removendo

```
npm install                     # instala tudo o que o package.json pede
npm install react-router-dom    # instala um pacote e o registra em dependencies
npm install -D vitest           # -D registra em devDependencies
npm uninstall nome-do-pacote    # remove
```

---

4. Vite: o que ele faz por você

O navegador entende HTML, CSS e JavaScript. Mas escrevemos **TypeScript** e **JSX**, que o navegador não entende. O Vite faz a ponte:

- **No desenvolvimento (`npm run dev`):** traduz os arquivos sob demanda e atualiza a página no instante em que você salva (isso se chama HMR, *Hot Module Replacement*).
- **Na produção (`npm run build`):** junta tudo, traduz, minimiza e gera uma pasta `dist/` com poucos arquivos rápidos de baixar.

📁 **No projeto:** `index.html` tem uma `<div id="root">` vazia e uma linha `<script type="module" src="/src/main.tsx">`. O `main.tsx` pega essa `div` e "monta" o React dentro dela:

```
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Tudo que você vê no site nasce dessa linha. O resto é o `App` e seus filhos.

(`StrictMode` é uma ajuda só do desenvolvimento: ele roda certas coisas duas vezes de propósito para revelar bugs escondidos. Não afeta o site publicado.)

A pasta `public/`

Arquivos aqui são servidos **exatamente como estão**, sem processamento. Por isso os ícones da aba (`favicon.svg`, `icon-512.png`) moram lá.

---

5. Variáveis de ambiente: configuração fora do código

O app precisa saber o endereço e a chave do Supabase. Escrever isso direto no código seria ruim (e para segredos, perigoso). A solução: **variáveis de ambiente**, guardadas em um arquivo à parte.

`.env.local` (fora do git):

```
VITE_SUPABASE_URL=https://....supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_...
```

No código (`src/lib/supabase.ts`):

```
const url = import.meta.env.VITE_SUPABASE_URL
const key = import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY

if (!url || !key) {
  throw new Error('Faltam VITE_SUPABASE_URL ou VITE_SUPABASE_PUBLISHABLE_KEY no .env.local')
}
```

Pontos importantes:

1. **O prefixo `VITE_` é obrigatório.** O Vite só envia ao navegador as variáveis que começam com ele. É uma proteção: o `VERCEL_OIDC_TOKEN` que a CLI colocou no mesmo arquivo **não** chega ao site.
2. **O erro que lançamos de propósito** (`throw new Error`) é uma boa prática: se faltar configuração, o app falha *alto e claro*, em vez de abrir em branco e deixar você adivinhando.

3. **Essas variáveis são "queimadas" no código na hora do build.** O valor vira texto dentro do JavaScript final. Qualquer visitante consegue lê-las no navegador. Por isso só colocamos ali o que **pode** ser público (a chave *publishable*), e jamais uma chave secreta.
4. **O arquivo `.env.local` não vai para o git.** O `.gitignore` tem as linhas `*.local` e `.env*`. Cada ambiente (seu Mac, o Vercel) recebe os seus valores por outro caminho.

⚠️ **Cuidado:** depois de mudar o `.env.local`, **reinicie** o `npm run dev`. O Vite só lê o arquivo ao iniciar.

---

6. Os arquivos `tsconfig`

Configuram o TypeScript. Você não precisa decorar, mas vale conhecer três opções do `tsconfig.app.json`:

Opção	O que faz	Por quê
`"noUnusedLocals": true`	Erro se uma variável é declarada e não usada	Mantém o código limpo
`"verbatimModuleSyntax": true`	Exige `import type` para tipos	Deixa claro o que existe só na compilação
`"erasableSyntaxOnly": true`	Proíbe recursos de TS que geram código extra	Mantém o TS "puro"

Esse rigor é a razão de o TypeScript te avisar de coisas pequenas, como o `use` importado e nunca usado que apareceu no começo do projeto.

---

7. IntelliJ: atalhos que valem ouro (Mac)

Atalho	Faz
`⌘⇧O`	Abre qualquer arquivo pelo nome
`⌘B`	Vai à definição do que está sob o cursor
`⌘E`	Arquivos recentes
`⇧F6`	**Renomeia com segurança**, atualizando todos os usos
`⌥⏎`	Mostra correções e sugestões para o erro sob o cursor
`⌘/`	Comenta/descomenta a linha
`⌥F12`	Abre/fecha o terminal
`⌘K`	Janela de **commit**
`⌘⇧K`	**Push**
`⌘⇧F`	Busca em todos os arquivos do projeto

🧪 **Experimente:** abra `Catalog.tsx`, clique sobre `applyFilters` e aperte `⌘B`. Você pula para `filters.ts`. Para voltar, use `⌘[`.

---

8. A lista de verificação antes de cada commit

Um hábito profissional que custa 30 segundos:

```
npx tsc -b      # os tipos estão certos?
npm test        # os testes passam?
npm run build   # o build final funciona?
```

Se os três passam, o commit é seguro. Foi exatamente isso que fizemos antes de quase todo `git commit` deste projeto.

Erros comuns

Sintoma	Causa provável	Solução
`ENOENT ... package.json`	Terminal na pasta errada	`cd ~/dev/vault-keeper`
`command not found: vite`	Esqueceu o `npm install`	`npm install`
Variável de ambiente `undefined`	Faltou o prefixo `VITE_` ou não reiniciou o servidor	Corrija o nome e reinicie
Site abre em branco	Erro de JavaScript	Abra o console do navegador (`⌥⌘J`)

Exercícios

1. Sem olhar este capítulo, liste o que cada script do `package.json` faz.
2. Rode `npm ls --depth=0`. Quais pacotes aparecem? Bate com `dependencies` + `devDependencies`?
3. Crie um arquivo `.env.local` de teste num projeto novo, com `VITE_OLA=mundo`, e mostre o valor na tela com `import.meta.env.VITE_OLA`. Depois renomeie a variável para `OLA` e veja o que acontece.
4. Apague a pasta `node_modules` e recupere-a com um comando. Quanto tempo levou?

✅ Checklist do capítulo 01

- ☐ Olho o prompt do terminal antes de rodar comandos do projeto.
- ☐ Sei a diferença entre Node, npm e npx.
- ☐ Explico `dependencies` × `devDependencies` e o significado do `^`.
- ☐ Sei o que é `node_modules` e por que não vai para o git.
- ☐ Explico por que o prefixo `VITE_` existe e por que a chave pública pode ficar no app, mas segredos não.
- ☐ Uso `tsc`, `test` e `build` antes de commitar.

Próximo: 02. JavaScript essencial.

CAPÍTULO 02 DE 18

JavaScript essencial: a parte que o projeto realmente usa

Objetivo

Dominar o JavaScript moderno que aparece no Vault Keeper. Este capítulo **não** é um curso completo de JS: é o subconjunto que você precisa para ler e escrever todo o código do projeto. Cada seção aponta onde o recurso aparece.

> **Como usar:** abra o console do navegador (`⌥⌘J` no Safari/Chrome, aba Console) e teste cada exemplo. Código que você executa fica na memória; código que só lê, não.

---

1. Variáveis: `const` e `let`

```
const nome = 'Joana'     // não pode ser reatribuída
let contador = 0         // pode mudar
contador = contador + 1
```

Regra do projeto: use `const` sempre; troque para `let` só se precisar reatribuir. `const` não significa que o valor é imutável (um array `const` pode receber itens), apenas que a *variável* não aponta para outra coisa.

Evite `var`: é a forma antiga, com comportamento confuso.

---

2. Tipos básicos

Tipo	Exemplo	Observação
`string`	`'oi'`, `"oi"`, `` `oi` ``	texto
`number`	`42`, `3.14`	inteiros e decimais são o mesmo tipo
`boolean`	`true`, `false`	
`null`	`null`	"propositalmente vazio"
`undefined`	`undefined`	"nunca recebeu valor"
`object`	`{ a: 1 }`	inclui arrays

Template strings (crase)

```
const titulo = 'Marina Lima'
const edicao = 292
const texto = `Nº ${edicao} · ${titulo}`   // "Nº 292 · Marina Lima"
```

Com crase, `${...}` insere qualquer expressão. 📁 `Catalog.tsx`: `` `Apagar "${pendingMagazine.title}"?` ``

Truthy e falsy: o que conta como "falso" num `if`

São **falsy**: `false`, `0`, `''` (texto vazio), `null`, `undefined`, `NaN`. Todo o resto é **truthy**. Por isso `if (title)` é falso para título vazio.

⚠️ **Cuidado:** `0` é falsy! Se um ano pudesse ser `0`, `{magazine.year && ...}` mostraria um `0` na tela em vez de nada. Para o ano de uma revista isso não acontece, mas é um clássico em outros contextos.

`==` ✕ `===`

Use sempre `===` (igualdade estrita) e `!==`. O `==` converte tipos e dá surpresas (`'1' == 1` é `true`).

---

3. Objetos

```
const revista = {
  title: 'Joana Prado',
  year: 1999,
  month: 12,
  is_public: true,
}

revista.title            // 'Joana Prado'
revista['year']          // 1999
revista.is_public = false
```

Desestruturação (extrair campos)

```
const { title, year } = revista        // cria as variáveis title e year
```

📁 `useSession.ts`: `supabase.auth.getSession().then(({ data }) => ...)` extrai `data` do objeto recebido.
📁 `magazineService.ts`: `const { data, error } = await supabase...`

Spread `...` (copiar e combinar)

```
const copia = { ...revista }                    // cópia rasa
const alterada = { ...revista, year: 2000 }     // copia e troca UM campo
```

📁 `Catalog.tsx`: `{ ...m, is_public: !m.is_public }`, "a mesma revista, mas com `is_public` invertido". O campo escrito **depois** do spread vence. Se escrever `{ year: 2000, ...revista }`, o `year` da `revista` sobrescreve o seu.

Encadeamento opcional `?.` e coalescência nula `??`

```
const modelo = revista.cover_model?.trim()   // se cover_model for null/undefined, devolve undefined (sem erro)
const titulo = revista.title ?? 'Sem título'  // se title for null/undefined, usa o texto padrão
```

- `?.` evita o erro "não é possível ler propriedade de undefined".
- `??` só troca `null` e `undefined`. Já o `||` troca *qualquer* falsy (inclusive `0` e `''`).

```
0 ?? 10     // 0     (0 é um valor válido)
0 || 10     // 10    (0 é falsy, então trocou)
```

📁 `MagazineForm.tsx`: `useState(initial?.title ?? '')` é "use o título inicial, se existir; senão, vazio".
📁 `filters.ts`: `m.cover_model ?? ''` evita juntar o texto `"null"` na busca.

---

4. Arrays e os métodos que você mais usa

```
const numeros = [3, 1, 2]
numeros.length           // 3
numeros[0]               // 3 (a contagem começa em 0)
```

Os métodos abaixo **não alteram** o array original (devolvem um novo). É a base do jeito React de pensar.

Método	Faz	Exemplo	Resultado
`map`	Transforma cada item	`[1,2,3].map(n => n * 2)`	`[2,4,6]`
`filter`	Mantém os que passam no teste	`[1,2,3].filter(n => n > 1)`	`[2,3]`
`find`	Primeiro que passa (ou `undefined`)	`[1,2,3].find(n => n > 1)`	`2`
`some`	Algum passa?	`[1,2,3].some(n => n > 2)`	`true`
`every`	Todos passam?	`[1,2,3].every(n => n > 0)`	`true`
`includes`	Contém?	`[1,2,3].includes(2)`	`true`
`join`	Junta em texto	`['a','b'].join(' · ')`	`'a · b'`
`slice`	Copia um pedaço	`[1,2,3].slice(1)`	`[2,3]`

⚠️ **Cuidado: `sort` ALTERA o array original.** Por isso, em `filters.ts`, ordenamos o resultado do `filter` (que já é uma cópia nova). Para ordenar sem risco, copie antes: `[...lista].sort(...)`.

Como funciona a função de comparação do `sort`

```
filtered.sort((a, b) => (b.year ?? 0) - (a.year ?? 0))
```

A função recebe dois itens. Se devolver **negativo**, `a` vem antes; **positivo**, `b` vem antes; **zero**, ficam como estão. `b - a` ordena do maior para o menor.

O `||` aqui dentro desempata: `(b.year ?? 0) - (a.year ?? 0) || (b.month ?? 0) - (a.month ?? 0)`. Se a diferença de anos for `0` (falsy), usa a diferença de meses. É um dos poucos casos em que o `||` é o certo.

📁 **No projeto:**

- `Catalog.tsx`: `magazines.filter((m) => m.id !== pendingDeleteId)` (remove um item).
- `Catalog.tsx`: `magazines.map((m) => (m.id === updated.id ? updated : m))` (troca um item).
- `Catalog.tsx`: `magazines.find((m) => m.id === editingId)` (acha o item em edição).
- `MagazineCard.tsx`: `[...].filter(Boolean).join(' · ')` junta só as partes que existem.

O truque `.filter(Boolean)`

`Boolean` aplicado a cada item devolve `true` para truthy e `false` para falsy, então remove `null`, `undefined`, `''` e `0`:

```
['Nº 293', null, '12/1999', ''].filter(Boolean)   // ['Nº 293', '12/1999']
```

---

5. Funções

```
function somar(a, b) { return a + b }          // declaração
const somar2 = (a, b) => a + b                 // arrow function (função de seta)
const dobrar = n => n * 2                      // um parâmetro: parênteses opcionais
const criar = () => ({ ok: true })             // devolver objeto: coloque entre parênteses
```

- **Arrow function com `{}`** precisa de `return` explícito. **Sem `{}`**, o valor da expressão é devolvido.
- Funções são **valores**: dá para passar como argumento, guardar em variável, devolver de outra função. É exatamente o que o React faz com `onClick={() => ...}` e `onDelete={setPendingDeleteId}`.

Closures (a função "lembra" o ambiente onde nasceu)

```
function criarContador() {
  let n = 0
  return () => { n = n + 1; return n }
}
const c = criarContador()
c()  // 1
c()  // 2
```

📁 `Catalog.tsx`: o `useEffect` usa uma variável `cancelled` que a função de limpeza (`return () => { cancelled = true }`) consegue alterar mais tarde. Isso só funciona por causa das closures.

---

6. Módulos: `import` e `export`

Cada arquivo é um módulo isolado. Você escolhe o que sai dele.

```
// filters.ts
export interface Filters { ... }                    // export nomeado
export const defaultFilters = { ... }
export function applyFilters(...) { ... }

// Catalog.tsx
import { applyFilters, defaultFilters, type Filters } from '../lib/filters'
```

Tipo	Escrita	Importação
Nomeado	`export function x`	`import { x } from './arquivo'`
Padrão (um por arquivo)	`export default App`	`import App from './App'`

- Caminhos que começam com `./` ou `../` são **relativos** ao arquivo atual. `'../lib/filters'` = "suba uma pasta, entre em `lib`, pegue `filters`".
- Sem `./`, é um pacote do `node_modules` (`'react'`, `'@supabase/supabase-js'`).
- `import type { Magazine } from '../types'` importa **só tipo** (some na compilação).

---

7. Assíncrono: `Promise`, `async` e `await`

Operações demoradas (internet, banco) não podem travar a página. O JavaScript as trata como **promessas**: um valor que **vai** existir.

```
// Modo "promessa" com .then
supabase.auth.getSession().then(({ data }) => { ... })

// Modo moderno com async/await (mais legível)
async function carregar() {
  const { data, error } = await supabase.from('magazines').select('*')
  ...
}
```

Regras:

1. `await` só funciona **dentro de uma função** `async`.
2. `await` "espera a resposta antes de seguir para a próxima linha" (sem travar a página inteira).
3. Uma função `async` **sempre devolve uma Promise**, mesmo que você escreva `return 5`.

Tratando erros: `try / catch / finally`

```
try {
  const created = await addMagazine(dados, userId)   // se der erro, pula para o catch
  setMagazines([created, ...magazines])
} catch (e) {
  setError((e as Error).message)                      // aqui você decide o que fazer
}
```

📁 `Catalog.tsx`: `handleAdd`, `handleUpdate`, `handleTogglePublic`, `confirmDelete`.

Lançar um erro: `throw`

```
if (error) throw error
```

📁 `magazineService.ts`: cada função **lança** o erro, e quem chamou (o `Catalog`) decide o que mostrar ao usuário. É a separação certa de responsabilidades: o serviço fala com o banco, a tela fala com o humano.

Duas operações em sequência (e o meio do caminho)

```
coverUrl = await uploadCover(cover, userId)   // 1º
const created = await addMagazine(...)        // 2º; se falhar, a imagem do 1º fica órfã
```

O `catch` de `handleAdd` desfaz o primeiro passo. Esse padrão ("desfazer o que já foi feito") é essencial em sistemas reais. Detalhes no capítulo 09.

---

8. Outras ferramentas usadas no projeto

`Set` (conjunto sem repetição)

```
const vistos = new Set(['293', '294'])
vistos.has('293')   // true
vistos.add('295')
```

📁 `BulkImport.tsx`: `knownIssues` guarda os números de edição já existentes para detectar duplicatas.

Expressões regulares (regex)

Padrões para buscar e extrair texto. Entre barras: `/padrão/flags`.

```
/^(\d{2})__(\d{2})/.exec('99__12__Playboy')   // captura '99' e '12'
```

Peça	Significa
`^` e `$`	início e fim do texto
`\d`	um dígito; `\d{2}` = exatamente dois

`.`	qualquer caractere; `.+` = um ou mais
`_{1,2}`	um ou dois sublinhados
`( )`	**grupo de captura**: guarda o trecho
`(?: )`	grupo que **não** captura (só agrupa)
`?` depois de um grupo	o grupo é **opcional**
flag `i`	ignora maiúsculas/minúsculas

📁 `parseCoverFilename.ts` usa uma regex para ler `99__12__Playboy-No-293-Joana-Prado.jpg`. É o melhor exemplo do projeto para estudar regex. Teste padrões em `regex101.com` colando o seu texto.

Métodos de texto que aparecem

Método	Exemplo	Uso no projeto
`trim()`	`' oi '.trim()` → `'oi'`	limpar o que o usuário digitou
`toLowerCase()`		busca sem diferenciar maiúsculas
`includes()`	`'abc'.includes('b')`	a busca do catálogo
`replace()`	`'a-b'.replace('-', ' ')`	limpar nomes de arquivo
`split()`	`'a.b.c'.split('.')` → `['a','b','c']`	pegar a extensão do arquivo
`padStart()`	`'3'.padStart(2, '0')` → `'03'`	mostrar mês como `03`
`normalize('NFD')`	separa a letra do acento	a busca ignora acentos
`localeCompare()`	ordena respeitando o idioma	ordem A-Z com acentos

`localStorage`

Guarda texto no navegador, que **sobrevive** ao fechar a aba.

```
localStorage.setItem('tema', 'light')
localStorage.getItem('tema')        // 'light' (ou null)
```

⚠️ Pode **lançar erro** (navegação anônima, cookies bloqueados), por isso o projeto sempre usa `try/catch` ao redor (`useTheme.ts`, `useViewMode.ts`, `AgeGate.tsx`).

`crypto.randomUUID()`

Gera um identificador único. 📁 `uploadCover` usa para nomear cada arquivo, garantindo que nunca sobrescreve outro.

Erros comuns

Erro	Causa
`Cannot read properties of undefined (reading 'x')`	Acessou `.x` de algo que é `undefined`. Use `?.` ou confira antes

`x is not a function`	Chamou algo que não é função (nome errado, import errado)
Lista não atualiza na tela	Alterou o array original em vez de criar um novo (`push`, `sort` direto)
`await` dá erro de sintaxe	Esqueceu o `async` na função
Valor `0` ou `''` "desaparece"	Usou `||` ou `&&` onde precisava de `??`

Exercícios

Resolva no console ou num arquivo `exercicios.js` rodado com `node exercicios.js`.

1. Dada `const edicoes = [293, 215, 358, 164]`, gere um array novo só com as maiores que 200, ordenadas do menor para o maior, **sem alterar** `edicoes`.
2. Dado `const r = { title: 'Rita', year: null }`, escreva uma linha que gere `'Rita (ano desconhecido)'` quando `year` for `null`, e `'Rita (1999)'` quando houver ano.
3. Escreva `formatarData(mes, ano)` que devolva `'03/1999'`, `'1999'` (se faltar o mês) ou `''` (se faltar o ano). *Dica:* `padStart` e `?` / `&&`.
4. Com uma regex, extraia o número da edição de `'Playboy-No-358-Flavia'`.
5. Escreva `async function esperar(ms)` que usa `setTimeout` dentro de uma Promise, e use `await esperar(1000)` para imprimir "oi" depois de um segundo.
6. **Desafio:** reescreva `applyFilters` do zero, sem olhar `filters.ts`, só com o comportamento descrito.

✅ Checklist do capítulo 02

- ☐ Uso `const` por padrão e sei quando `let` é necessário.
- ☐ Sei a diferença entre `??` e `||` e dou um exemplo em que dá resultado diferente.
- ☐ Uso `map`, `filter` e `find` sem consultar.
- ☐ Sei por que `sort` exige cuidado e como evitar o problema.
- ☐ Faço desestruturação e spread de objetos de cabeça.
- ☐ Explico `async/await`, `try/catch` e o que `throw` faz.
- ☐ Leio uma regex simples como a de `parseCoverFilename`.

Próximo: 03. TypeScript.

CAPÍTULO 03 DE 18

TypeScript: dando forma aos dados

Objetivo

Entender o que são tipos, por que o projeto os usa em tudo e como ler (e escrever) as anotações que aparecem no código. No fim, você deve conseguir olhar `types.ts` e explicar cada linha.

---

1. Por que existe o TypeScript

JavaScript é **dinâmico**: uma variável pode ser número agora e texto depois. Isso é flexível, mas permite erros que só aparecem **quando o usuário clica**:

```
function dobrar(n) { return n * 2 }
dobrar('abc')        // NaN, sem nenhum aviso
```

TypeScript = JavaScript + tipos. Você descreve o formato dos dados, e o editor avisa os erros **antes de rodar** (o sublinhado vermelho no IntelliJ).

```
function dobrar(n: number): number { return n * 2 }
dobrar('abc')        // ❌ erro: 'string' não é atribuível a 'number'
```

Importante: o TypeScript **desaparece na compilação**. O navegador executa JavaScript comum. Os tipos existem só para ajudar quem escreve.

---

2. Tipos básicos e anotações

```
const titulo: string = 'Marina Lima'
const edicao: number = 292
const publica: boolean = true
const ids: string[] = ['a', 'b']            // lista de textos
const dados: { ano: number; mes: number } = { ano: 1999, mes: 12 }
```

Inferência: muitas vezes você nem precisa escrever

```
const titulo = 'Marina Lima'     // o TS já sabe que é string
```

Regra prática: **anote** parâmetros de função, retornos de funções exportadas e estados em que o tipo não é óbvio. Deixe o TS **inferir** o resto.

---

3. `interface` e `type`: descrevendo um objeto

📁 **No projeto:** `src/types.ts`

```
export type Condition = 'Excelente' | 'Bom' | 'Regular' | 'Ruim'

export interface Magazine {
  id: string
  title: string
  issue?: string | null
  year: number | null
  month: number | null
  cover_model?: string | null
  condition: Condition | null
  acquired: boolean
  is_public: boolean
  cover_image_url?: string | null
}
```

Leia linha por linha:

Trecho	Significa
`id: string`	Todo `Magazine` **tem** um `id`, e é texto
`issue?: string | null`	O `?` diz que o campo é **opcional** (pode nem existir); `| null` aceita também `null`
`year: number | null`	Obrigatório, mas pode ser `null` ("não sei o ano")
`condition: Condition | null`	Uma das quatro condições, ou `null`

`|` (união): "um ou outro"

```
type Condition = 'Excelente' | 'Bom' | 'Regular' | 'Ruim'
```

Isso é um **tipo literal união**: só esses quatro textos são aceitos. Se você escrever `'Otimo'`, o IntelliJ sublinha de vermelho. Foi o exercício que fizemos na Etapa 1.

`interface` × `type`

Para objetos, os dois fazem quase o mesmo. Convenção comum: `interface` para formatos de objeto, `type` para uniões e apelidos. Não perca tempo escolhendo; o importante é ser consistente.

---

4. `null`, `undefined` e o "modo estrito"

A pergunta mais valiosa que o TypeScript faz: **"e se isto não existir?"**

```
const m = magazines.find((x) => x.id === 'abc')   // tipo: Magazine | undefined
m.title                                           // ❌ erro: m pode ser undefined
m?.title                                          // ✅ ok
```

O `find` pode não achar nada, e o TS **obriga** você a tratar isso. 📁 `Catalog.tsx`: `pendingMagazine?.title`, `if (!pendingMagazine) return`.

Estreitamento de tipo (*narrowing*)

Depois de verificar, o TS "aprende":

```
if (!pendingMagazine) return
// aqui embaixo, o TS SABE que pendingMagazine existe
await deleteMagazine(pendingMagazine)
```

---

5. ⚠️ O caso em que os tipos "mentiram" (leia com atenção)

Na primeira versão, `Magazine` dizia `year: number` e `month: number`. Mas o banco tinha 64 revistas **sem ano** (`null`). O TypeScript **não reclamou**, porque ele só confia no que você escreveu: ele não consulta o banco.

Resultado em produção: os cards mostravam `null/null` na tela.

Lição: os tipos descrevem o que você **acredita** sobre os dados, não o que os dados **são**. Quando os dados vêm de fora (banco, API, arquivo), eles podem divergir do tipo. A correção foi tornar o tipo honesto (`year: number | null`) e tratar o `null` em todo lugar que usa (`MagazineCard`, `filters.ts`, o formulário). Depois que o tipo passou a admitir `null`, o TS **apontou todos os lugares** que precisavam de ajuste. É assim que ele ajuda de verdade.

---

6. Genéricos e tipos utilitários

Genéricos: um tipo "com um buraco"

```
Promise<Magazine[]>     // "uma promessa que vai entregar uma lista de Magazine"
useState<string | null>(null)
```

O que está entre `< >` preenche o buraco. 📁 `async function listMyMagazines(userId: string): Promise<Magazine[]>`.

Tipos utilitários: transformam tipos que você já tem

Utilitário	Faz	No projeto
`Omit<T, 'campo'>`	Tira campos	`type NewMagazine = Omit<Magazine, 'id'>`: uma revista **sem** `id` (quem cria o id é o banco)
`Partial<T>`	Torna tudo opcional	`updateMagazine(id, changes: Partial<NewMagazine>)`: pode atualizar só alguns campos
`Record<K, V>`	Objeto com chaves `K` e valores `V`	`Record<Status, string>` em `BulkImport.tsx`, uma tabela de rótulos para cada estado

Vantagem enorme: **uma fonte da verdade**. Se você acrescenta um campo em `Magazine`, `NewMagazine` e `Partial<NewMagazine>` ganham o campo automaticamente.

---

7. `import type` e `as`

```
import type { Magazine } from '../types'
```

`import type` deixa claro que isso só existe na compilação. O projeto **exige** isso (`verbatimModuleSyntax`).

`as`: afirmar um tipo ("confie em mim")

```
setCondition(e.target.value as Condition)
```

O `e.target.value` do `<select>` é `string`, mas **nós sabemos** que só pode ser uma das quatro condições. O `as` cala o TS. ⚠️ **Use com parcimônia**: se estiver errado, o TS não te protege mais ali. Já o `!` em `document.getElementById('root')!` diz "tenho certeza de que não é `null`".

---

8. Tipando as peças do React

Props

```
interface MagazineCardProps {
  magazine: Magazine
  onDelete?: (id: string) => void              // função opcional
  onEdit?: (magazine: Magazine) => void
}

export function MagazineCard({ magazine, onDelete, onEdit }: MagazineCardProps) { ... }
```

`(id: string) => void` descreve uma função: recebe um `string`, não devolve nada útil.

Estado

```
const [magazines, setMagazines] = useState<Magazine[]>([])
const [error, setError] = useState<string | null>(null)
```

Sem o `<Magazine[]>`, o TS deduziria `never[]` para `[]` (uma lista que nunca aceita nada).

Eventos

```
function handleSubmit(event: React.FormEvent) { ... }
function handleCoverChange(event: React.ChangeEvent<HTMLInputElement>) { ... }
```

Dica prática: se não souber o tipo de um evento, passe o mouse sobre o `onChange` no IntelliJ, ele mostra.

`children`

```
children: React.ReactNode   // qualquer coisa que o React sabe desenhar
```

---

9. Lendo um erro do TypeScript

Os erros são longos, mas seguem um padrão. Exemplo real do projeto, quando tornamos `condition` aceitável como `null`:

```
Type '"Excelente" | "Bom" | "Regular" | "Ruim" | null' is not assignable to type
'string | number | readonly string[] | undefined'.
  Type 'null' is not assignable to type 'string | number | readonly string[] | undefined'.
```

Como ler: **de baixo para cima.** A última linha costuma dizer a causa real: "`null` *não cabe no atributo* `value` *de um* `<select>`". A solução foi separar os tipos (`Condition`, sem `null`, para o formulário; e `Magazine.condition`, com `null`, para os dados). Isso é refatoração guiada pelo compilador.

---

10. O que o TypeScript NÃO faz

- **Não valida dados em tempo de execução.** O que vem do banco pode estar errado (como vimos).
- **Não substitui testes.** Ele garante *formatos*, não *comportamentos*.
- **Não aparece no site final.** Some na compilação.

Erros comuns

Sintoma	Causa	Solução
`Object is possibly undefined`	Não tratou o caso "não existe"	`?.`, `??` ou um `if` antes
`Property 'x' does not exist`	Nome errado ou tipo incompleto	Confira o tipo; use o autocompletar
Tudo `any` por todo lado	Anotou `any` para "calar" o TS	Evite `any`; use `unknown` e estreite
`Type 'string' is not assignable to type 'Condition'`	Texto solto onde espera a união	Use `as Condition` (se tiver certeza) ou valide

Exercícios

1. Em `types.ts`, torne `year` opcional (`year?: number`). Que erros o TS mostra e onde? Depois desfaça.
2. Crie um tipo `MagazineSummary` com `Pick<Magazine, 'id' | 'title'>`. (`Pick` é o oposto de `Omit`.)
3. Escreva `function rotuloDoMes(mes: number | null): string` que devolva `'Mar'` para `3` e `'–'` para `null`.
4. Crie `type Status = 'pendente' | 'ok' | 'erro'` e uma função que receba um `Status` e devolva um texto. Tente passar `'ocupado'` e leia o erro.
5. **Desafio:** por que `useState<Magazine[]>([])` precisa do `<Magazine[]>`? Teste sem e passe o mouse sobre a variável.

✅ Checklist do capítulo 03

- ☐ Explico por que tipos existem e que somem na compilação.
- ☐ Leio `types.ts` linha por linha, incluindo `?` e `| null`.
- ☐ Sei o que são união (`|`) e tipo literal.
- ☐ Uso `?.` e `??` por causa do que o TS aponta.
- ☐ Explico `Omit`, `Partial` e `Record` com exemplos do projeto.
- ☐ Conto o caso do `year: number` que "mentiu" e o que ele ensina.

Próximo: 04. React: fundamentos.

CAPÍTULO 04 DE 18

React: fundamentos

Objetivo

Entender como o React transforma **dados** em **tela**: componentes, JSX, props, listas, renderização condicional, eventos e formulários. Ao final, você deve ler qualquer componente do projeto e prever o que ele desenha.

---

1. A ideia central do React

Em vez de mandar o navegador "mude este texto, esconda aquele botão", você descreve **como a tela deve ser dado um conjunto de dados**:

```
tela = função(dados)
```

Quando os dados mudam, o React recalcula a descrição, compara com a anterior e **altera só o que mudou**. Você escreve *o resultado desejado*; o React cuida do *como*. Isso se chama programação **declarativa**.

---

2. Componente = função que devolve tela

```
function Home() {
  return <h1>Olá</h1>
}
```

Só isso. Um componente é uma **função** cujo nome começa com **letra maiúscula** e que devolve JSX. Usa-se como se fosse uma tag HTML nova: `<Home />`.

📁 **No projeto:** `src/pages/Home.tsx`, `src/components/Modal.tsx`, `src/components/MagazineCard.tsx`: todos são funções.

Por que dividir em componentes?

- **Reutilização:** o `MagazineCard` é usado 64 vezes na tela, escrito uma vez.
- **Clareza:** cada peça tem uma responsabilidade.
- **Manutenção:** mudar o card muda todos os cards.

---

3. JSX: HTML dentro do JavaScript

```
return (
  <article className="card">
    <h2 className="card__title">{magazine.title}</h2>
    <p>Nº {magazine.issue}</p>
  </article>
)
```

JSX **parece** HTML, mas é JavaScript. As diferenças que mais pegam:

Em HTML	Em JSX	Por quê
`class="card"`	`className="card"`	`class` é palavra reservada do JS
`for="campo"`	`htmlFor="campo"`	idem
`<img src="x">`	`<img src="x" />`	**toda** tag precisa fechar
`style="color: red"`	`style={{ color: 'red' }}`	`style` recebe um objeto
`onclick="..."`	`onClick={...}`	eventos em camelCase
Comentário `<!-- -->`	`{/* comentário */}`	

Regras do JSX

1. **Um único elemento raiz.** Para devolver dois irmãos sem criar uma `<div>` à toa, use um *fragmento*: `<> ... </>`.
2. **Chaves `{}` abrem uma "janela" para JavaScript:** qualquer *expressão* cabe ali.

   ```
   <p>{magazine.title}</p>
   <p>{String(month).padStart(2, '0')}</p>
   <img src={magazine.cover_image_url} alt={`Capa de ${magazine.title}`} />
   ```

3. **Só expressões, não instruções.** Dentro de `{}` você não escreve `if` nem `for`; usa `? :`, `&&` e `.map()` (veja adiante).
4. **O React escapa o texto** que você insere com `{}`. Se alguém cadastrar uma revista chamada `<script>...`, ela aparece como texto, não executa. Isso protege contra uma classe de ataques (XSS), capítulo 13.

---

4. Props: os dados que o pai entrega ao filho

```
interface MagazineCardProps {
  magazine: Magazine
  onDelete?: (id: string) => void
}

export function MagazineCard({ magazine, onDelete }: MagazineCardProps) {
  return <h2>{magazine.title}</h2>
}
```

```
// no pai:
<MagazineCard magazine={m} onDelete={setPendingDeleteId} />
```

- Props são **somente leitura**. O filho nunca altera o que recebeu.
- Podem ser **dados** (`magazine`) ou **funções** (`onDelete`).
- Opcionais ganham `?` no tipo; o componente decide o que fazer quando faltam.

O princípio mais importante do React: *dados descem, eventos sobem*

```
Catalog (dono da lista de revistas)
   │  desce: a revista e a função onDelete
   ▼
MagazineCard (só desenha)
   │  sobe: "o usuário clicou em Deletar a revista X" → chama onDelete(X)
   ▼
Catalog (decide o que fazer: abrir a janela de confirmação)
```

Quem tem o estado é quem pode alterá-lo. O filho **avisa**, o pai **decide**. Por isso o `MagazineCard` não sabe nada de banco, de confirmação ou de listas: só recebe o que mostrar e chama a função quando clicam.

`children`: o que está *entre* as tags

```
<Modal onClose={fechar}>
  <p>Qualquer conteúdo aqui</p>
</Modal>
```

No `Modal`, esse conteúdo chega na prop especial `children`. Por isso o mesmo `Modal` serve ao `ConfirmDialog` e ao formulário de edição. É **composição**: peças genéricas que se encaixam.

📁 `src/components/Modal.tsx` e `src/components/ConfirmDialog.tsx`: o `ConfirmDialog` **usa** o `Modal`.

---

5. Listas: `.map()` e a prop `key`

```
<MagazineGallery mode={viewMode}>
  {visibleMagazines.map((magazine) => (
    <MagazineCard key={magazine.id} magazine={magazine} />
  ))}
</MagazineGallery>
```

`.map()` transforma uma lista **de dados** numa lista **de elementos**.

Por que `key`?

O React precisa **identificar** cada item para saber, ao atualizar, quem é quem (quem entrou, saiu, mudou de lugar). A `key` deve ser:

- **Única** entre os irmãos;
- **Estável**: o mesmo item, a mesma key (use o `id`).

⚠️ **Não use o índice** (`key={index}`) quando a lista pode mudar de ordem ou perder itens: o React confunde os itens, e o estado interno de um card pode "pular" para outro. Use o `id`.

6. Renderização condicional

Três jeitos, todos usados no projeto:

a) Operador `&&` (mostrar ou não mostrar)

```
{magazine.cover_model && <p className="card__meta">Capa: {magazine.cover_model}</p>}
```

Se o texto existir, mostra; senão, nada. ⚠️ Cuidado com `0`: `{contagem && <p>...</p>}` mostra um `0` na tela quando `contagem` é `0`. Use `{contagem > 0 && ...}`.

b) Operador ternário `? :` (um ou outro)

```
{magazine.cover_image_url ? (
  <img src={magazine.cover_image_url} alt={`Capa de ${magazine.title}`} />
) : (
  <div className="card__placeholder">Sem capa</div>
)}
```

c) Retorno antecipado (`early return`)

```
if (loading) return <div>Carregando...</div>
if (!session) return <AuthForm />
return <div>...a página inteira...</div>
```

📁 `Catalog.tsx`: três "versões" da página (carregando, sem login, logado) numa função só.

⚠️ **Regra dos hooks:** os `return` antecipados ficam **depois** de todos os `useState` e `useEffect` (capítulo 05).

📁 O `AgeGate` devolve `null` quando a idade já foi confirmada: "não desenhe nada".

---

7. Eventos

```
<button onClick={() => onDelete(magazine.id)}>Deletar</button>
<input onChange={(e) => setTitle(e.target.value)} />
<form onSubmit={handleSubmit}>
```

Pontos de atenção:

1. **Passe uma função, não o resultado dela.**

   ```
   onClick={handleSave}            // ✅ passa a função; roda no clique
   onClick={handleSave()}          // ❌ roda AGORA, ao desenhar
   onClick={() => handleSave(id)}  // ✅ quando precisa de argumento, envolva numa arrow
   ```

2. `event.preventDefault()` no `onSubmit`: sem ele, o navegador recarrega a página ao enviar o formulário.
3. `event.stopPropagation()`: eventos "borbulham" dos filhos para os pais. No `Modal`, clicar no fundo escuro fecha a janela, mas clicar **dentro** da caixa não pode fechar. Por isso a caixa chama `stopPropagation()`.

---

8. Formulários: inputs controlados

```
const [title, setTitle] = useState('')

<input value={title} onChange={(e) => setTitle(e.target.value)} />
```

O React é o **dono** do valor do campo: cada tecla dispara `onChange`, o estado muda, a tela redesenha com o novo valor. Vantagens: dá para validar, limpar, preencher por código, mostrar o que foi digitado em outro lugar.

Tipos de campo no projeto (`MagazineForm.tsx`):

Campo	Valor vem de	Exemplo
Texto	`e.target.value`	título, modelo da capa
Número	`e.target.value` (**ainda é texto**)	ano e mês; convertemos com `Number()` na hora de enviar
Caixa de marcar	`e.target.checked`	"Já tenho esta revista"
Lista (`<select>`)	`e.target.value`	condição
Arquivo	`e.target.files`	a capa

Por que ano e mês são guardados como texto no formulário?

```
const [year, setYear] = useState(initial?.year?.toString() ?? '')
...
year: year === '' ? null : Number(year)
```

Porque o campo vazio precisa existir (`''`), e `Number('')` daria `0`, um ano falso. Guardar o texto digitado e converter **só no envio** permite distinguir "não preenchi" (`null`) de um número real.

Um formulário, dois modos

O `MagazineForm` recebe `initial?: Magazine`. Se vier, está em **modo edição**; se não, em **modo criação**. A diferença se resume a valores iniciais (`initial?.title ?? ''`), um título e um texto de botão. O resto é compartilhado. **Dois formulários quase iguais seriam duas coisas para manter, e uma sempre acabaria desatualizada.**

---

9. Lendo um componente do início ao fim: `MagazineCard`

Abra `src/components/MagazineCard.tsx` e acompanhe:

1. Recebe `magazine` e três funções **opcionais** (`onDelete`, `onEdit`, `onTogglePublic`).
2. `isOwnerView = Boolean(onTogglePublic || onEdit || onDelete)`. Se **alguma** ação foi passada, é a visão do dono. No catálogo público não passamos nenhuma: o mesmo componente aparece em modo leitura.
3. Calcula `date` e `meta`, **sem** `useState`: são **valores derivados** que se refazem a cada desenho.
4. Devolve JSX: capa (ou placeholder), título, textos secundários, etiquetas e (só para o dono) botões.

Isso ilustra o poder de **props opcionais**: um componente, duas aparências.

---

10. O que acontece quando algo muda (visão rápida)

```
clique → setState(novo valor) → React chama o componente de novo → compara o JSX novo com o anterior
→ atualiza SÓ o que mudou na tela
```

O capítulo 05 explora isso a fundo.

Erros comuns

Sintoma	Causa	Solução
`Each child in a list should have a unique "key"`	Faltou `key` no `.map()`	Adicione `key={item.id}`
`Adjacent JSX elements must be wrapped`	Dois elementos na raiz	Envolva em `<>...</>`
Página recarrega ao enviar o formulário	Faltou `preventDefault()`	Adicione no `onSubmit`
Função roda sozinha ao abrir a tela	Escreveu `onClick={fazer()}`	`onClick={fazer}` ou `() => fazer()`
`0` aparece solto na tela	Usou `{numero && ...}`	`{numero > 0 && ...}`
Campo "trava" e não deixa digitar	`value` sem `onChange`	Adicione o `onChange` que atualiza o estado

Exercícios

1. Crie `src/components/StatCard.tsx` que receba `label: string` e `value: number` e mostre os dois. Use-o em `Catalog.tsx` para exibir "Total", "Públicas" e "Procurando". *(Sim, são três cards; reaproveitar é o ponto.)*
2. Modifique `MagazineCard` para mostrar um selo "NOVA" quando a revista tiver `year` igual ao ano atual.
3. Crie um componente `EmptyState` com a mensagem "Nenhuma revista" e use no lugar do parágrafo atual.

4. Em `ConfirmDialog`, aceite uma prop `confirmLabel` com padrão `'Apagar'`.
5. **Desafio:** faça um componente `Rating` com cinco estrelas clicáveis que controla uma nota de 1 a 5 (guardada em estado no pai e passada por props).

✅ Checklist do capítulo 04

- ☐ Explico "tela = função(dados)".
- ☐ Diferencio componente, JSX e props.
- ☐ Explico "dados descem, eventos sobem" com `MagazineCard` + `Catalog`.
- ☐ Sei por que `key` existe e por que não usar o índice.
- ☐ Uso `&&`, ternário e retorno antecipado na hora certa.
- ☐ Explico um input controlado e por que `preventDefault` é necessário.
- ☐ Explico por que o formulário guarda o ano como texto.

Próximo: 05. React: estado, efeitos e hooks.

CAPÍTULO 05 DE 18

React: estado, efeitos e hooks

Objetivo

Dominar as três ideias que fazem uma tela *reagir*: **estado** (`useState`), **efeitos** (`useEffect`) e **hooks próprios**. É o capítulo mais importante da parte de React. Leia com calma e faça os exercícios.

---

1. Por que precisamos de estado

Uma variável comum **não** faz a tela mudar:

```
function Contador() {
  let n = 0
  return <button onClick={() => { n = n + 1 }}>{n}</button>   // ❌ clica e nada acontece na tela
}
```

Dois problemas: (1) o React não sabe que `n` mudou, então não redesenha; (2) mesmo se redesenhasse, a função rodaria de novo e `n` voltaria a `0`.

Estado resolve os dois: é uma memória que o React guarda **entre desenhos** e que, ao mudar, **dispara um novo desenho**.

```
import { useState } from 'react'

function Contador() {
  const [n, setN] = useState(0)                       // [valor atual, função para trocar]
  return <button onClick={() => setN(n + 1)}>{n}</button>
}
```

---

2. `useState` em detalhe

```
const [magazines, setMagazines] = useState<Magazine[]>([])
//     ▲ valor        ▲ função que troca        ▲ valor inicial
```

Regra 1: nunca altere o estado diretamente; sempre crie um novo

```
magazines.push(nova)                    // ❌ altera o array; o React não percebe
setMagazines([nova, ...magazines])      // ✅ cria um array novo
```

O React decide se redesenha **comparando** o valor novo com o antigo por *referência*. Se você muda o mesmo array por dentro, a referência é a mesma, e o React conclui "nada mudou".

Receitas para listas e objetos (todas devolvem algo **novo**):

Quero...	Escrevo
Adicionar no início	`[nova, ...lista]`
Remover	`lista.filter(x => x.id !== id)`
Trocar um item	`lista.map(x => x.id === id ? { ...x, campo: novo } : x)`
Mudar um campo de um objeto	`{ ...objeto, campo: novo }`

📁 `Catalog.tsx` usa as quatro receitas: `handleAdd`, `confirmDelete`, `handleUpdate`, `handleTogglePublic`.

Regra 2: a atualização é *agendada*, não imediata

```
setN(n + 1)
console.log(n)    // ainda mostra o valor ANTIGO
```

O novo valor só existe no **próximo desenho**.

A forma funcional: quando o novo valor depende do antigo

```
setMagazines((current) => [...created, ...current])
```

Em vez de passar o valor, você passa uma função que recebe o **valor mais recente**. Compare no projeto:

```
// handleAdd: usa a variável "magazines" capturada NO MOMENTO em que a função foi criada
setMagazines([created, ...magazines])

// BulkImport (no onImported do Catalog): forma funcional, sempre parte do valor mais recente
setMagazines((current) => [...created, ...current])
```

No `handleAdd`, existe um risco sutil: se duas atualizações acontecerem antes de o React redesenhar, a segunda parte de uma lista **desatualizada** e apaga o trabalho da primeira. A forma funcional elimina o risco. É um ótimo exercício de refatoração (capítulo 14): troque os `setMagazines([...magazines])` por versões funcionais.

Inicialização preguiçosa

```
const [theme, setTheme] = useState<Theme>(readTheme)    // sem parênteses: passa a FUNÇÃO
```

`useState(readTheme())` rodaria `readTheme()` (que lê o `localStorage`) **a cada desenho**, jogando fora o resultado depois do primeiro. Passando a função, o React a chama **uma única vez**, na criação.
📁 `useTheme.ts`, `useViewMode.ts`, `AgeGate.tsx`.

---

3. Estado derivado: não guarde o que dá para calcular

📁 `Catalog.tsx`:

```
const [magazines, setMagazines] = useState<Magazine[]>([])     // estado: a lista COMPLETA
const [filters, setFilters]     = useState<Filters>(defaultFilters)   // estado: o que o usuário escolheu
const visibleMagazines = applyFilters(magazines, filters)      // DERIVADO: só uma conta
```

Só os dois primeiros são estado. O terceiro é **calculado** a cada desenho. Se guardássemos a lista filtrada em outro `useState`, teríamos **duas cópias que podem divergir**: você apaga uma revista, esquece de atualizar a cópia filtrada, e a tela mostra uma revista fantasma.

> **Regra de ouro:** guarde o mínimo de estado possível e **derive** o resto.

O mesmo vale no `MagazineCard`: `date`, `meta` e `isOwnerView` são calculados, não guardados.

---

4. Efeitos: sincronizar com o mundo de fora

Um componente React deve ser uma **conta pura**: dados entram, JSX sai. Mas apps reais precisam de coisas que vivem **fora** do React: buscar dados na internet, mexer no `localStorage`, ouvir o teclado, ligar um relógio. Isso é um **efeito colateral**, e o `useEffect` é onde ele mora.

```
useEffect(() => {
  // 1) o que fazer DEPOIS de desenhar
  return () => {
    // 2) (opcional) a LIMPEZA: desfazer o que o passo 1 ligou
  }
}, [dependencias])   // 3) quando rodar de novo
```

O array de dependências

Escrita	Quando o efeito roda
`useEffect(fn)` (sem array)	Depois de **todo** desenho (quase nunca é o que você quer)
`useEffect(fn, [])`	**Uma vez**, quando o componente aparece
`useEffect(fn, [userId])`	Quando aparece **e** sempre que `userId` mudar

A regra: liste no array tudo que o efeito *usa* e que pode mudar. (O IntelliJ avisa quando falta algo.)

A limpeza (*cleanup*) é obrigatória quando você "liga" algo

📁 `Modal.tsx`:

```
useEffect(() => {
  function handleKey(event: KeyboardEvent) {
    if (event.key === 'Escape') onClose()
  }
  window.addEventListener('keydown', handleKey)            // liga o ouvinte
  return () => window.removeEventListener('keydown', handleKey)   // desliga ao sair
}, [onClose])
```

Sem a limpeza, cada janela aberta deixaria um ouvinte a mais vivo, mesmo depois de fechada. Quanto mais abrisse e fechasse, mais ouvintes acumulados (um **vazamento**).

Outros pares "liga/desliga" no projeto:

- `useSession.ts`: `onAuthStateChange(...)` e `subscription.unsubscribe()`.
- `MagazineForm.tsx`: `URL.createObjectURL(cover)` e `URL.revokeObjectURL(url)` (libera memória da prévia).
- `MagazineGallery.tsx`: `addEventListener('wheel', ...)` e `removeEventListener(...)`.

O problema da "resposta velha" (condição de corrida)

📁 `Catalog.tsx`:

```
useEffect(() => {
  if (!userId) { setMagazines([]); return }
  let cancelled = false
  listMyMagazines(userId)
    .then((data) => { if (!cancelled) setMagazines(data) })
    .catch((e: Error) => { if (!cancelled) setError(e.message) })
  return () => { cancelled = true }
}, [userId])
```

Imagine: o usuário entra e sai rapidamente. Há duas buscas no ar. Se a **primeira** demorar mais e chegar **depois** da segunda, ela sobrescreveria o resultado correto com dados velhos. A limpeza marca `cancelled = true` para a busca antiga, e o `if (!cancelled)` descarta a resposta atrasada. Isso funciona por causa das **closures** (capítulo 02): a função de limpeza enxerga a mesma variável `cancelled` do efeito que a criou.

Quando NÃO usar `useEffect`

A documentação do React insiste: **efeito é a exceção**. Não use para:

- **Calcular algo a partir de props/estado** → faça direto no corpo (estado derivado).
- **Reagir a um clique** → ponha a lógica no manipulador do evento.

📁 Veja a diferença no projeto:

- `useTheme.ts` usa efeito, **porque** precisa sincronizar com algo externo (o atributo no `<html>` e o armazenamento).
- `useViewMode.ts` grava no `localStorage` **dentro da função** `change`, sem efeito: a mudança nasce de um clique, e o clique é o lugar natural para a gravação.

Os avisos do `npm run lint`

O lint do projeto mostra dois avisos `set-state-in-effect`:

- `MagazineForm.tsx`: o efeito chama `setPreview(...)` logo após criar o endereço temporário da imagem.
- `Catalog.tsx`: `setMagazines([])` no começo do efeito quando não há usuário.

Eles dizem: "mudar estado *de dentro* de um efeito causa um desenho extra". Funciona, mas há alternativas mais enxutas (por exemplo, `useMemo` para o endereço da prévia). **É um ótimo exercício de refatoração.**

---

5. `useRef`: guardar algo que não deve causar redesenho

```
const scroller = useRef<HTMLDivElement>(null)
...
<div ref={scroller}>...</div>
...
scroller.current.scrollBy({ left: 300 })
```

Um `ref` é uma "caixa" onde você guarda um valor que **persiste entre desenhos** mas, ao contrário do estado, **mudar não redesenha**. Dois usos comuns:

1. **Acessar um elemento do DOM** (como o scroller em `MagazineGallery.tsx`, para rolar com os botões).
2. **Guardar um valor "de bastidor"**: `BulkImport.tsx` usa `previews = useRef<string[]>([])` para lembrar os endereços temporários das imagens e liberá-los na saída.

Estado × ref: se a tela precisa *mostrar* o valor, é estado. Se é só para *lembrar*, é ref.

---

6. Hooks próprios: reaproveitar lógica com estado

Um **hook próprio** é uma função cujo nome começa com `use` e que chama outros hooks. Ele não devolve JSX: devolve **dados e funções**.

`useSession` (`src/hooks/useSession.ts`)

Responde "quem está logado?":

```
export function useSession() {
  const [session, setSession] = useState<Session | null>(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    supabase.auth.getSession().then(({ data }) => { setSession(data.session); setLoading(false) })
    const { data } = supabase.auth.onAuthStateChange((_event, newSession) => setSession(newSession))
    return () => data.subscription.unsubscribe()
  }, [])

  return { session, loading }
}
```

Fluxo em três tempos: (1) pergunta **uma vez** se já há sessão salva; (2) **escuta** entradas e saídas; (3) **limpa** a escuta ao sair. Note o `loading`: enquanto não sabemos, mostramos "Carregando...", e não a tela de login (senão o usuário veria um "piscar" de login antes de a sessão ser reconhecida).

`useTheme` e `useViewMode`

Ambos seguem o mesmo padrão: **estado + persistência no** `localStorage`.

Hook	Guarda	Devolve
`useTheme`	`'dark'` ou `'light'`	`{ theme, toggleTheme }`
`useViewMode`	`'vertical'`, `'horizontal'` ou `'mosaico'`	`[mode, setMode]`

`useViewMode` ainda tem uma defesa: só aceita um valor **conhecido** do armazenamento (`MODES.find((mode) => mode === saved) ?? 'vertical'`). Se alguém mexer no `localStorage` e colocar lixo, o app volta ao padrão em vez de quebrar. **Nunca confie em dados que vêm de fora do seu código**, nem do navegador.

Por que isso é bom

`PublicCatalog` e `Catalog` usam o **mesmo** `useViewMode()`. A lógica vive num lugar só.

---

7. As regras dos hooks (o React exige)

1. **Chame hooks apenas no nível de cima** do componente. Nunca dentro de `if`, `for` ou função aninhada.
2. **Chame hooks só em componentes ou em outros hooks.**

Por quê? O React identifica cada hook pela **ordem** em que é chamado. Se a ordem muda entre desenhos, ele se perde.

📁 É por isso que, em `Catalog.tsx`, os `return` antecipados ("Carregando...", "sem login") ficam **depois** de todos os `useState`/`useEffect`. Se um `return` viesse antes de um hook, em alguns desenhos o hook seria chamado e em outros não, o que o React proíbe.

---

8. `key` para reiniciar um componente

```
<MagazineForm key={editingMagazine.id} initial={editingMagazine} ... />
```

O `useState(initial?.title)` só usa o valor inicial **na primeira vez**. Se você abrisse a edição de outra revista com o mesmo componente já montado, o formulário continuaria com os dados da primeira. Trocar a `key` faz o React **descartar** o componente e criar um novo, com o estado zerado. É um truque pequeno e muito útil.

O mesmo vale para o campo de arquivo: `<input key={fileInputKey} type="file" />` é "zerado" mudando a `key`, porque o navegador não permite limpar um campo de arquivo por código.

9. Elevando o estado (*lifting state up*)

Onde guardar o estado `magazines`? Quem precisa dele: a lista, o contador, os filtros, a edição, a remoção. Todos moram **na página** `Catalog`, então é ali que o estado vive. Os componentes menores recebem dados por props e avisam por funções.

> **Regra:** coloque o estado no **ancestral comum mais próximo** de todos que precisam dele. Nem mais alto (vira bagunça global), nem mais baixo (os irmãos não conseguem compartilhar).

10. Desenvolvimento × produção: o `StrictMode`

Em desenvolvimento, o `<StrictMode>` (em `main.tsx`) roda efeitos **duas vezes** para revelar efeitos sem limpeza. Se um efeito seu quebra ao ser executado duas vezes, falta uma limpeza. No site publicado isso não acontece.

Erros comuns

Sintoma	Causa	Solução
Tela não atualiza ao mudar a lista	Alterou o array direto (`push`, `sort`)	`setLista([...novo])`
Efeito roda em loop infinito	O efeito muda um estado que está nas suas dependências	Revise as dependências; talvez nem seja efeito
`Rendered more hooks than during the previous render`	Hook depois de um `return` ou dentro de `if`	Mova os hooks para o topo
Valor "velho" dentro de uma função	Closure capturou o estado de um desenho antigo	Forma funcional `setX(prev => ...)`
Ouvinte duplicado / vazamento	Efeito sem função de limpeza	Adicione o `return () => ...`
Formulário mostra dados da revista anterior	Faltou `key` ao reutilizar o componente	`key={item.id}`

Exercícios

1. Crie `src/hooks/useLocalStorage.ts` com `useLocalStorage<T>(chave, valorPadrao)` que devolve `[valor, setValor]` e grava em JSON. Use-o para guardar o último texto digitado na busca.
2. Refatore `handleAdd`, `confirmDelete` e `handleUpdate` para usar a **forma funcional** do `setMagazines`. Rode os testes e confira no navegador.
3. Faça um hook `useDebounce(valor, atrasoMs)` que só atualiza o valor depois de o usuário parar de digitar. Aplique à busca. *(Pista:* `setTimeout` *dentro do efeito e a limpeza com* `clearTimeout`*.)*

4. Elimine o aviso do lint em `MagazineForm.tsx` calculando o endereço da prévia com `useMemo` e liberando-o num efeito de limpeza.
5. **Desafio:** crie um `useOnline()` que devolve `true/false` conforme a internet, ouvindo os eventos `online` e `offline` do `window`. Mostre um aviso quando estiver offline.

✅ Checklist do capítulo 05

- ☐ Explico por que uma variável comum não faz a tela mudar.
- ☐ Nunca altero estado direto, e sei as quatro receitas para listas.
- ☐ Sei quando usar a forma funcional do `setState`.
- ☐ Explico estado derivado com o `visibleMagazines`.
- ☐ Leio o array de dependências e explico `[]`, `[x]` e a ausência dele.
- ☐ Sei por que a limpeza do efeito existe e dou três exemplos do projeto.
- ☐ Explico a variável `cancelled` e o problema que ela evita.
- ☐ Diferencio estado de ref.
- ☐ Escrevo um hook próprio e sei as regras dos hooks.

Próximo: 06. Rotas e organização.

CAPÍTULO 06 DE 18

Rotas e organização do código

Objetivo

Entender como um site de **uma única página** consegue ter vários endereços (`/` , `/catalog` , `/public`), como o React Router decide qual tela mostrar e por que o projeto está organizado em pastas do jeito que está.

---

1. SPA: *Single Page Application*

Num site tradicional, cada clique em um link pede ao servidor **uma página HTML nova**, e o navegador a recarrega inteira (aquele "piscar").

Numa **SPA**, o navegador baixa **uma só página** (`index.html`) e o JavaScript **troca o conteúdo** conforme o endereço muda, sem recarregar. É mais rápido e fluido, e é o que o nosso app faz.

Para isso funcionar, algo precisa **ler o endereço** e decidir qual componente mostrar. Esse algo é o **React Router**.

---

2. React Router no projeto

📁 `src/App.tsx` :

```
<BrowserRouter>
  <AgeGate />
  <header className="site-header">
    <Link to="/" className="brand">PLAYBOY BRASIL</Link>
    <nav className="nav" aria-label="Principal">
      <NavLink to="/" end>HOME</NavLink>
      <NavLink to="/catalog">Minha coleção</NavLink>
      <NavLink to="/public">Catálogo Público</NavLink>
    </nav>
    ...
  </header>

  <main>
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/catalog" element={<Catalog />} />
      <Route path="/public" element={<PublicCatalog />} />
      <Route path="*" element={<NotFound />} />
    </Routes>
  </main>
</BrowserRouter>
```

Cada peça

Peça	Papel
`<BrowserRouter>`	Liga o roteamento ao endereço do navegador. Envolve tudo que usa rotas
`<Routes>`	Olha o endereço atual e desenha **apenas a primeira** `<Route>` que combinar
`<Route path="..." element={...}>`	"Neste endereço, mostre este componente"
`path="*"`	Curinga: combina com **qualquer** outro endereço, a nossa página 404. Vem por último
`<Link to="...">`	O `<a>` do React Router: navega **sem recarregar** a página
`<NavLink>`	Como o `Link`, mas ganha a classe `active` quando o endereço combina (o sublinhado do menu)

O detalhe do `end`

```
<NavLink to="/" end>HOME</NavLink>
```

Sem o `end`, o `/` "combinaria" com **todos** os endereços (todos começam com `/`), e o HOME ficaria sempre ativo. O `end` diz: "só vale se for **exatamente** `/`".

O que fica fora das `<Routes>` aparece em todas as páginas

O cabeçalho e o `AgeGate` estão **fora** do `<Routes>`, por isso aparecem em qualquer rota. Só o miolo (`<main>`) troca. Isso é um **layout compartilhado**.

Link × `<a>`

Use `<Link>` para páginas **do seu app** (sem recarregar). Use `<a href>` para sites externos. 📁 O botão "Sair" do `AgeGate` é um `<a href="https://www.google.com">`, porque leva para fora do app.

---

3. O problema do 404 na hospedagem (e o `vercel.json`)

Há um detalhe que pega todo iniciante. Imagine um visitante abrindo **direto**:

```
https://meusite.com/catalog
```

O servidor **não** sabe o que é `/catalog`: ele só tem `index.html`, `assets/...`. Ele procura um arquivo chamado `catalog`, não acha e devolve **404**, antes mesmo de o React ter a chance de carregar.

A solução é mandar o servidor devolver **sempre** o `index.html`, e o React Router, já no navegador, decide a tela.

📁 `vercel.json`:

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

Em português: "qualquer endereço → entregue o `index.html`". (No `npm run dev`, o Vite já faz isso sozinho, por isso o problema só aparece depois de publicar.)

⚠️ **Se um dia o site publicado der 404 só ao recarregar uma página interna, é isto.**

---

4. Organização de pastas: separar por responsabilidade

```
src/
├── pages/        ← uma tela por rota
├── components/   ← peças de tela reutilizáveis
├── hooks/        ← lógica COM estado, reutilizável
└── lib/          ← lógica SEM tela + acesso a serviços
```

Como decidir onde um arquivo vai (o "teste das quatro perguntas")

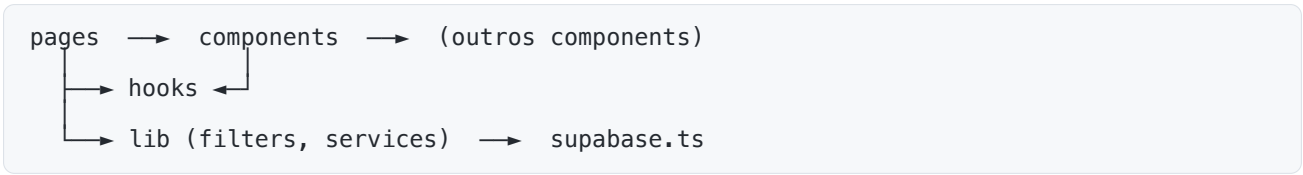
Pergunta	Se sim →
Representa uma **rota/tela inteira**?	`pages/`
Desenha **um pedaço de tela** reutilizável?	`components/`
Usa `useState` / `useEffect` mas **não desenha** nada?	`hooks/`
É uma **função comum**, sem React? Ou fala com um serviço externo?	`lib/`

Arquivo	Pasta	Por quê
`Catalog.tsx`	`pages/`	É a tela da rota `/catalog`
`MagazineCard.tsx`	`components/`	Peça reutilizada 64 vezes
`useSession.ts`	`hooks/`	Estado + efeito, sem JSX
`filters.ts`	`lib/`	Função pura, nem sabe que React existe
`magazineService.ts`	`lib/`	Fala com o Supabase
`types.ts`	`src/`	Tipos usados por todos

Por que isso importa?

- **Achar as coisas:** "onde mexo para mudar a busca?" → `lib/filters.ts`.
- **Testar:** o que está em `lib/` (sem React) é fácil de testar. Foi por isso que escrevemos testes para `filters.ts` e `parseCoverFilename.ts`, e não para as páginas.
- **Trocar uma parte sem quebrar as outras:** se um dia o banco mudar, só `magazineService.ts` muda.

Fluxo das dependências (quem importa quem)



Regra de ouro: `lib/` **nunca** importa de `components/` ou `pages/`. A lógica não deve depender da tela. O sentido das setas é sempre "de quem usa para quem é usado".

---

5. Onde mora cada responsabilidade (um exemplo completo)

"Mostrar o catálogo público" atravessa quatro arquivos, cada um com **uma** tarefa:

Arquivo	Tarefa
`PublicCatalog.tsx`	Montar a página: pede os dados, aplica os filtros, escolhe o que desenhar
`magazineService.ts`	Buscar no banco (`listPublicMagazines`)
`filters.ts`	Decidir quais revistas passam nos filtros (`applyFilters`)
`MagazineCard.tsx`	Desenhar uma revista

Se o catálogo ficar lento, você sabe onde olhar. Se mudar o desenho do card, só um arquivo muda.

---

6. Convenção de nomes do projeto

O quê	Como	Exemplo
Componentes e páginas	`PascalCase` (maiúscula em cada palavra), arquivo com o mesmo nome	`MagazineCard.tsx`
Hooks	começam com `use`	`useSession.ts`
Funções e variáveis	`camelCase`	`listMyMagazines`
Constantes fixas	`MAIÚSCULAS_COM_SUBLINHADO`	`MAX_COVER_MB` , `STORAGE_KEY`
Classes de CSS	minúsculas com hífen; `bloco__elemento` e `bloco--modificador` (convenção BEM)	`card__title` , `gallery--mosaico`
Arquivos de teste	`nome.test.ts` ao lado do arquivo testado	`filters.test.ts`

Erros comuns

Sintoma	Causa	Solução
Página recarrega ao clicar num link do menu	Usou `<a href>` em vez de `<Link>`	`<Link to="...">`
Menu "HOME" sempre ativo	Faltou `end` no `NavLink` de `/`	`<NavLink to="/" end>`

404 só ao recarregar a página (no site publicado)	Falta a regra de reescrita	`vercel.json` com `rewrites`
Rota nunca combina	Ordem errada: o `*` antes das outras	O curinga sempre por último
`useNavigate() may be used only in the context of a <Router>`	Usou o roteamento fora do `BrowserRouter`	Envolva o componente

Exercícios

1. Crie uma página `Sobre` em `src/pages/Sobre.tsx` e adicione a rota `/sobre` e o link no menu.
2. Faça a página 404 mostrar o endereço digitado. *(Pesquise `useLocation` do React Router.)*
3. Crie uma rota com parâmetro, `/revista/:id`, que mostre os detalhes de uma revista. *(Pesquise `useParams`.)*
4. Desenhe, no papel, o grafo "quem importa quem" para `Catalog.tsx`. Confira se `lib/` nunca aponta para cima.
5. **Desafio:** proteja a rota `/catalog` redirecionando para `/` quando não houver sessão (hoje o `Catalog` mostra o formulário de login dentro de si). Compare as duas abordagens: qual você prefere e por quê?

✅ Checklist do capítulo 06

- ☐ Explico a diferença entre site tradicional e SPA.
- ☐ Sei o papel de `BrowserRouter`, `Routes`, `Route`, `Link` e `NavLink`.
- ☐ Explico o `end` e o `path="*"`.
- ☐ Explico por que abrir `/catalog` direto daria 404 sem o `vercel.json`.
- ☐ Aplico o "teste das quatro perguntas" para decidir a pasta de um arquivo.
- ☐ Sei que `lib/` nunca importa de `components/` ou `pages/`.

Próximo: 07. CSS moderno.

CAPÍTULO 07 DE 18

CSS moderno

Objetivo

Entender como o visual do app é construído: o modelo de caixas, a cascata, variáveis e temas, flexbox, grid, responsividade e animações. Ao final, você deve conseguir alterar qualquer aspecto visual do app **sabendo o que está fazendo**, não por tentativa e erro.

> **Como estudar este capítulo:** deixe `npm run dev` rodando, abra o site e o painel de inspeção do navegador (clique direito → *Inspecionar*). Cada seção aqui pode ser testada **ao vivo** nesse painel: você edita o CSS e vê o resultado na hora, sem salvar nada.

---

1. Como o CSS funciona

CSS são **regras**: *seletor* (quem) + *declarações* (o que muda).

```
.card__title {                      /* seletor: elementos com a classe card__title */
  font-size: 1.2rem;                /* declaração: propriedade: valor; */
  text-transform: uppercase;
}
```

Seletores que o projeto usa

Seletor	Pega	Exemplo
`.classe`	Elementos com `class="classe"`	`.card`
`elemento`	Todas as tags daquele tipo	`button`, `body`
`A B`	`B` **dentro** de `A`	`.gallery--mosaico .card__title`
`A > B`	`B` **filho direto** de `A`	`.gallery__scroller > .card`
`A, B`	Os dois	`h1, h2, h3, p`
`[atributo='valor']`	Pelo atributo	`button[aria-pressed='true']`
`:hover`	Quando o mouse está sobre	`.card:hover`
`:focus-visible`	Foco **pelo teclado**	`:focus-visible`
`:focus-within`	Quando algo **dentro** tem foco	`.card:focus-within`
`:root`	O elemento mais externo (`<html>`)	onde moram as variáveis

A cascata e a especificidade (por que "às vezes não pega")

Quando duas regras mexem na mesma coisa, quem ganha? Três critérios, nesta ordem:

1. **Especificidade:** seletor mais específico vence (`#id` > `.classe` > `elemento`).
2. **Ordem:** a regra **mais abaixo** no arquivo vence um empate.

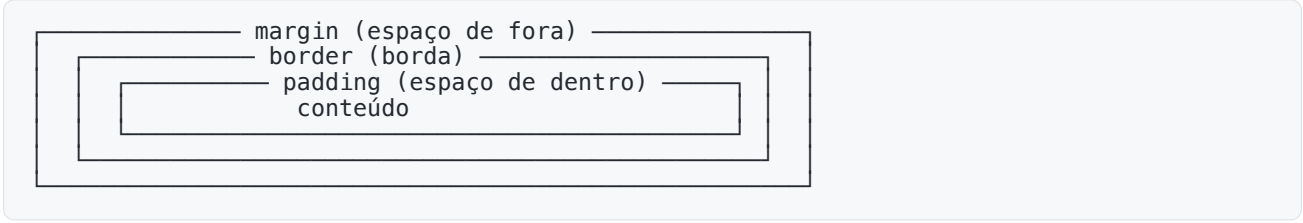
3. **Herança:** o que não for definido é herdado do pai (`color`, `font`), mas `margin` e `border` não.

⚠️ **Caso real do projeto:** os botões que são **links** (`<a class="btn">`) apareceram **roxos**. O motivo: o navegador já tem uma regra de cor para `<a>`, e a nossa `.btn` não definia `color`. A correção foi escrever `color: var(--text)` na `.btn`. Quando algo tem cor "inexplicável", é uma regra do navegador que você não sobrescreveu.

---

2. O modelo de caixas

Todo elemento é uma caixa com quatro camadas:



📁 `index.css` tem, logo no começo:

```
*, *::before, *::after { box-sizing: border-box; }
```

Isso muda a conta: com `border-box`, a `width: 200px` **já inclui** padding e borda. Sem isso, `width: 200px` + `padding: 20px` viraria 240px, e as caixas "estourariam" o layout. Quase todo projeto moderno começa assim.

Esse bloco, junto com `h1, h2, h3, p { margin: 0 }`, forma o **reset**: zera diferenças entre navegadores para você controlar o espaçamento.

---

3. Unidades

Unidade	Significa	Quando usar
`px`	Pixel fixo	Bordas finas, sombras
`rem`	Múltiplo do tamanho da fonte da página (16px)	**Tamanhos de texto e espaços** (respeita o ajuste do usuário)
`em`	Múltiplo da fonte do **elemento**	Espaços que acompanham o texto
`%`	Porcentagem do pai	Larguras
`vw` / `vh`	1% da largura/altura da janela	Tipografia fluida, telas cheias
`svh`	Altura da janela **sem** as barras do celular	`min-height: 100svh` (evita "pulos")
`fr`	Fração do espaço livre (só no grid)	`1fr 1fr`

Por que `rem` em vez de `px` para texto? Quem aumenta a fonte do navegador por necessidade visual espera que o site acompanhe. `rem` acompanha; `px` não.

---

4. Variáveis CSS e temas ⭐ (o coração do visual)

📁 `src/index.css`:

```
:root {
  --bg: #000000;
  --surface: #0d0d0d;
  --text: #f2f2f2;
  --muted: #9a9a9a;
  --accent: #ff6b8a;
  --radius: 8px;
  --gutter: clamp(1rem, 1.6vw, 1.75rem);
}

:root[data-theme='light'] {
  --bg: #ffffff;
  --text: #111111;
  --muted: #6b6b6b;
  --accent: #d6336c;
}
```

Uso: `background: var(--bg);`.

Como funciona o tema claro/escuro

1. As cores moram **só** nas variáveis.
2. O padrão (escuro) está em `:root`.
3. `:root[data-theme='light']` **redefine** as mesmas variáveis quando o `<html>` tem `data-theme="light"`.
4. O `useTheme.ts` troca esse atributo: `document.documentElement.dataset.theme = theme`.

Resultado: **o app inteiro troca de visual sem uma única linha de CSS a mais**, porque *todo* o CSS usa `var(--text)`, `var(--bg)` etc. Para criar um terceiro tema (azul, por exemplo), basta um bloco novo com as variáveis.

A regra da separação

Cores, tamanhos de espaçamento e raios ficam em **variáveis**. O resto do CSS **só as consome**. Se você achar um `#f2f2f2` solto no meio de `App.css`, é um sinal de que deveria ser `var(--text)`.

⚠️ **Nuance:** o texto sobre fotos (o mosaico, por exemplo) usa `#fff` fixo de propósito. Ele fica sobre um degradê escuro e precisa ser branco **nos dois temas**. Variáveis temáticas mudariam a cor e perderiam o contraste.

Contraste

Quando você testou `--bg: #77AD78` (verde) o texto quase sumiu. Contraste é medido por uma razão: o mínimo recomendado para texto é **4,5:1**. Verde claro com texto quase branco dá cerca de 2,3:1. Ferramenta para checar: `webaim.org/resources/contrastchecker`.

---

5. Flexbox: alinhar em uma dimensão

```
.toolbar {
  display: flex;
  flex-wrap: wrap;                    /* pode quebrar de linha */
  align-items: center;                /* alinhamento no eixo transversal */
  justify-content: space-between;     /* distribuição no eixo principal */
  gap: 1rem;                          /* espaço entre os itens */
}
```

Propriedade	Faz
`display: flex`	Liga o flex no **pai**; os filhos viram "itens"
`flex-direction: column`	Eixo principal vertical (o padrão é horizontal)
`justify-content`	Posiciona ao longo do eixo **principal**
`align-items`	Posiciona no eixo **transversal**
`gap`	Espaço entre itens (bem melhor que margens soltas)
`flex-wrap: wrap`	Itens quebram de linha em vez de espremer
`flex: 1 1 640px`	"cresce, encolhe, **base** 640px", como a caixa de filtros

Centralizar tudo (a receita mais pedida do CSS)

```
.overlay {
  position: fixed; inset: 0;                  /* cobre a tela inteira */
  display: flex;
  align-items: center;                        /* centraliza na vertical */
  justify-content: center;                    /* centraliza na horizontal */
}
```

📁 É assim que a janela do `Modal` e o `AgeGate` ficam no meio da tela.

---

6. Grid: alinhar em duas dimensões

```
.gallery--vertical {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));    /* 4 colunas iguais */
  gap: 2.5rem var(--gutter);                            /* gap: linhas colunas */
}
```

Leia `repeat(4, minmax(0, 1fr))`: "4 colunas, cada uma ocupando uma fração igual do espaço, **mínimo zero**". O `minmax(0, ...)` evita que uma imagem larga empurre a coluna para fora.

Grade que se adapta sozinha

No mosaico:

```
grid-template-columns: repeat(auto-fill, minmax(clamp(105px, 11vw, 170px), 1fr));
```

"Encaixe quantas colunas **couberem**, cada uma com pelo menos 105 a 170px." Em tela larga, 8 colunas; em celular, 3. Sem nenhuma media query.

Flex ou grid?

- **Flex:** uma linha ou coluna de itens (menus, barras de ferramentas, botões lado a lado).
- **Grid:** uma grade (galerias, layouts de página).

---

7. Responsividade

`clamp(mínimo, ideal, máximo)`

```
font-size: clamp(3rem, 13vw, 9rem);      /* o título da página inicial */
--gutter: clamp(1rem, 1.6vw, 1.75rem);   /* as margens laterais */
```

O valor cresce com a janela (`13vw`), mas **nunca** abaixo de 3rem nem acima de 9rem. Uma linha substitui várias media queries.

Media queries: regras que só valem em certas condições

```
@media (max-width: 1180px) { .gallery--vertical { grid-template-columns: repeat(3, minmax(0, 1fr)); } }
@media (max-width: 820px)  { .gallery--vertical { grid-template-columns: repeat(2, minmax(0, 1fr)); } }
```

Em tela estreita, a grade passa de 4 para 3 e para 2 colunas.

Há também media queries de **preferência e capacidade**, não de largura:

Query	Pergunta
`(hover: hover)`	O aparelho tem mouse? (as ações do card só aparecem no hover)
`(hover: none)`	É toque? (no mosaico, o texto fica sempre visível)
`(prefers-reduced-motion: reduce)`	O usuário pediu menos animação?

📁 No fim de `App.css`: respeitar `prefers-reduced-motion` é acessibilidade (capítulo 14).

🧪 **Experimente:** no navegador, abra o painel de inspeção, ative o modo "dispositivo" (ícone de celular) e vá diminuindo a largura. Veja a grade mudar de 4 para 2 colunas.

---

8. Imagens: `aspect-ratio` e `object-fit`

```
.card__media { aspect-ratio: 3 / 4; overflow: hidden; background: var(--placeholder); }
.card__cover { width: 100%; height: 100%; object-fit: cover; }
```

- `aspect-ratio: 3 / 4` reserva o espaço da capa **antes** de a imagem carregar, então a página não "pula" (o placeholder cinza que você viu).
- `object-fit: cover` faz a imagem preencher a caixa **cortando** o excesso, sem esticar. (`contain` mostraria inteira, deixando faixas vazias.)

---

9. Posicionamento

Valor	Comportamento	No projeto
`static`	Normal (padrão)	
`relative`	Normal, mas vira "âncora" para filhos `absolute`	`.card` no mosaico
`absolute`	Posicionado em relação ao ancestral `relative`	o texto sobre a capa no mosaico
`fixed`	Fixo em relação à **janela**	o `overlay` do modal
`sticky`	Rola normalmente e "gruda" no topo	o cabeçalho do site

O mosaico é um bom exemplo: o card é `position: relative`, e o bloco `.card__info` é `position: absolute` com `inset: auto 0 0 0` (colado embaixo, ocupando a largura toda), ficando **por cima** da imagem.

`z-index` decide quem fica na frente: cabeçalho `10`, janelas `20`, aviso de idade `100`.

---

10. Transições e efeitos

```
.card__cover { transition: transform 0.5s ease; }
.card:hover .card__cover { transform: scale(1.025); }
```

A transição suaviza a mudança entre o estado normal e o `:hover`. Mexer em `transform` e `opacity` é barato para o navegador; mexer em `width` ou `height` é caro.

Outros efeitos usados

Efeito	Onde	O quê
`backdrop-filter: blur(10px)`	Cabeçalho, janelas	Desfoca o que está atrás (efeito "vidro")
`color-mix(in srgb, var(--bg) 88%, transparent)`	Cabeçalho	Mistura cores: o fundo do cabeçalho é 88% da cor do tema
`linear-gradient(transparent, rgba(0,0,0,.88))`	Mosaico	Degradê que escurece a base da capa para o texto ler
`box-shadow`	Janelas	Sombra

Texto com degradê (o título da home)

```css
.hero__title {
  background: linear-gradient(180deg, var(--text) 25%, var(--muted) 100%);
  -webkit-background-clip: text; background-clip: text;
  color: transparent;
}
```

Truque: pinta o **fundo** com o degradê e o **recorta no formato das letras**, tornando o texto transparente. As cores vêm do tema, por isso funciona no claro e no escuro.

---

11. Rolagem com "ímã" (*scroll snap*): o modo horizontal

```css
.gallery__scroller {
  display: flex;
  gap: var(--gutter);
  overflow-x: auto;                       /* rola para os lados */
  overscroll-behavior-x: contain;         /* não "vaza" a rolagem para a página */
  scroll-snap-type: x proximity;          /* ímã suave */
}
.gallery__scroller > .card {
  flex: 0 0 clamp(230px, 26vw, 445px);    /* não cresce, não encolhe, largura fixa */
  scroll-snap-align: start;               /* ao parar, alinha o card à esquerda */
}
```

- `flex: 0 0 <largura>` impede o flex de espremer os cards: eles ficam em uma fila que **ultrapassa** a tela, e o `overflow-x: auto` cria a rolagem.
- `proximity` (ímã *suave*) em vez de `mandatory` (ímã *forte*): o forte brigava com a roda do mouse, que rola em pequenos passos.

---

12. Ícones em SVG

Os botões de visualização usam **SVG inline**: desenhos feitos de linhas, escritos no código.

```svg
<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.7" ...>
  <rect x="4" y="3" width="11" height="18" rx="1.5" />
  ...
</svg>
```

- `viewBox="0 0 24 24"` é a "folha" onde você desenha, em coordenadas de 0 a 24, e que escala sem perder qualidade.
- `stroke="currentColor"` faz o traço **herdar a cor do texto**, então o ícone troca de cor com o tema e com o `:hover`, sem imagem extra.
- Vantagens sobre PNG: nítido em qualquer tamanho, leve, estilizável por CSS.

---

13. HTML nativo que dispensa JavaScript

```
<details className="panel">
  <summary>+ Nova revista</summary>
  <MagazineForm ... />
</details>
```

O `<details>` abre e fecha **sozinho**, acessível pelo teclado, sem uma linha de JS.
Antes de escrever JavaScript, pergunte se o HTML já faz.

---

14. Nomeando classes: BEM

Convenção `bloco__elemento--modificador`:

```
.card                → o bloco
.card__title         → uma parte DO card
.card--missing       → uma variação do card
.gallery--mosaico    → uma variação da galeria
```

Dá previsibilidade: ao ver `card__title`, você sabe que pertence ao card.

⚠️ **Caso real do projeto:** a classe foi escrita no CSS como `.gallery--mosaic` (sem o **o**), mas o código gera `gallery--mosaico` a partir do nome do modo. Nenhuma regra do mosaico foi aplicada, e a tela mostrou uma única capa gigante. O TypeScript **não** pega erros de classe CSS (são só texto). Só olhando o resultado na tela é que apareceu. **Teste visual não substitui teste automático, nem o contrário.**

---

15. Dicas de depuração visual

1. **Inspecione** o elemento (clique direito → Inspecionar). Veja quais regras se aplicam e quais estão riscadas (perderam na cascata).
2. **Borda temporária:** `outline: 1px solid red` num elemento mostra o tamanho real da caixa.
3. **Consulte o "Computed"** (estilo calculado): mostra o valor final de cada propriedade.
4. Quando "nada acontece", suspeite de: nome de classe diferente, regra sobrescrita por outra mais específica, ou arquivo CSS não importado.

Erros comuns

Sintoma	Causa	Solução
Elemento ultrapassa a tela	Falta `box-sizing: border-box` ou largura fixa grande	Reset + `max-width: 100%`
`margin: 0 auto` não centraliza	Elemento sem largura definida	Dê `max-width` / `width`
Imagem esticada	Falta `object-fit`	`object-fit: cover`
Regra "ignorada"	Outra mais específica ou mais abaixo vence	Inspecione e compare

		especificidade
`z-index` sem efeito	O elemento é `static`	Use `position: relative` (ou outro)
Cor de link inesperada	Regra do navegador para `<a>`	Defina `color`

Exercícios

1. Em `index.css`, mude `--bg` e `--text` para um tema "sépia" novo. Quantas linhas mexeu? Quantos lugares do app mudaram?
2. Crie um **terceiro tema** (`[data-theme='azul']`) e faça o botão de tema alternar entre três.
3. Mude o número de colunas do modo vertical de 4 para 5 em telas grandes e ajuste os pontos de quebra.
4. No mosaico, faça o texto aparecer com uma animação de subida suave (`transform: translateY`).
5. Crie um estado de **carregamento** para os cards: um bloco cinza pulsando (`@keyframes` + `animation`).
6. **Desafio:** reproduza só o CSS do `Modal` (overlay + caixa centralizada) num HTML solto, sem React.

✅ **Checklist do capítulo 07**

- ☐ Explico as quatro camadas da caixa e o que `border-box` muda.
- ☐ Decido entre flex e grid para um layout novo.
- ☐ Leio `repeat(auto-fill, minmax(...))` e `clamp(...)`.
- ☐ Explico como o tema claro/escuro funciona só com variáveis.
- ☐ Resolvo um conflito de cascata inspecionando o elemento.
- ☐ Explico por que o ícone SVG usa `currentColor`.
- ☐ Conto os dois casos reais: o link roxo e a classe `--mosaic`.

Próximo: 08. Banco de dados e Supabase.

CAPÍTULO 08 DE 18

Banco de dados e Supabase

Objetivo

Entender onde e como os dados do app vivem: tabelas, SQL, segurança por linha (RLS), views, armazenamento de arquivos, autenticação e migrations. Ao final, você deve conseguir ler as regras de segurança do projeto e dizer, para qualquer pessoa (visitante, dono, outro usuário), **o que ela pode ver e fazer**.

> ⚠️ **Antes de praticar SQL:** crie um **projeto Supabase separado, só para estudo**. O banco do Vault Keeper tem dados reais. Comandos como `delete` e `update` não têm "desfazer". Nunca experimente em dados que você não quer perder.

---

1. O que é um banco relacional

Um banco **relacional** guarda dados em **tabelas**, como planilhas muito disciplinadas:

id (chave primária)	title	issue	year	month	is_public
`a1b2...`	Joana Prado Feiticeira	293	1999	12	true
`c3d4...`	Marina Lima	292	1999	11	true

- **Tabela:** um tipo de coisa (`magazines`).
- **Linha (*row*):** uma coisa (uma revista).
- **Coluna:** uma característica (`title`, `year`), com **tipo fixo** e regras.
- **Chave primária (*primary key*):** a coluna que identifica a linha de forma **única** (`id`).

Postgres é o programa de banco que o Supabase usa. **SQL** é a linguagem para conversar com ele.

---

2. A tabela `magazines`

Reconstruída a partir do que existe no banco (para estudo):

```sql
create table public.magazines (
  id              uuid primary key default gen_random_uuid(),
  user_id         uuid not null,
  title           text not null,
  issue           text,
  year            integer,
  month           integer,
  condition       text check (condition in ('Excelente', 'Bom', 'Regular', 'Ruim')),
  notes           text,
  cover_model     text,
  acquired        boolean default false,
  cover_image_url text,
  is_public       boolean not null default false,
  created_at      timestamptz not null default now(),
  updated_at      timestamptz not null default now()
);
```

Leia coluna por coluna:

Trecho	Significa
`uuid`	Identificador único e longo (`a1b2c3d4-...`), impossível de adivinhar
`default gen_random_uuid()`	Se você não informar o `id`, o banco gera um
`not null`	**Obrigatória**: o banco recusa a linha se faltar
`text`, `integer`, `boolean`	Tipos: texto, inteiro, verdadeiro/falso
`check (condition in (...))`	**Regra de validação no banco**: só esses 4 valores
`timestamptz`	Data e hora **com fuso horário**
`default now()`	Preenche com o momento da criação

Duas observações sobre o desenho (vale refletir)

1. `year` **e** `month` **aceitam** `null`. Foi uma decisão consciente: o acervo importado não tinha datas. Um campo obrigatório forçaria a inventar um valor. É por isso que o tipo TypeScript tem `year: number | null`.
2. `user_id` **não tem chave estrangeira**, um desenho que custou caro (capítulo 17). O correto seria `user_id uuid not null references auth.users(id) on delete cascade`. Assim, **apagar um usuário apagaria também suas revistas**, em vez de deixá-las órfãs, invisíveis e inalcançáveis. Foi exatamente o que aconteceu: 69 revistas ficaram sem dono depois que a conta foi apagada.

Camada de segurança: `enable row level security`

```sql
alter table public.magazines enable row level security;
```

Veja a seção 5.

---

3. SQL essencial: as quatro operações (CRUD)

Operação	SQL	Equivalente no `supabase-js`
Create	`insert into ... values (...)`	`.insert({...})`
Read	`select ... from ... where ...`	`.select().eq(...)`
Update	`update ... set ... where ...`	`.update({...}).eq(...)`
Delete	`delete from ... where ...`	`.delete().eq(...)`

Ler (`select`)

```sql
select title, year from public.magazines;                       -- só duas colunas
select * from public.magazines where is_public = true;          -- só as públicas
select * from public.magazines order by created_at desc;        -- mais novas primeiro
select count(*) from public.magazines;                          -- quantas linhas
select count(*) filter (where is_public) from public.magazines; -- contar só as públicas
```

Inserir

```sql
insert into public.magazines (user_id, title, year)
values ('11111111-...', 'Marina Lima', 1999);
```

Atualizar e apagar

```sql
update public.magazines set is_public = true where id = '...';
delete from public.magazines where id = '...';
```

⚠️⚠️ A regra de ouro do `update` e do `delete`: sempre `where`

```sql
delete from public.magazines;          -- apaga TODAS as linhas da tabela
```

Sem `where`, o comando vale para **todas** as linhas. Hábito de profissional: **escreva primeiro o `select` com o mesmo `where`**, confira quantas linhas aparecem, e só então troque por `delete` / `update`.

```sql
-- 1) confira
select count(*) from public.magazines where user_id = 'X';
-- 2) só se o número for o esperado:
delete from public.magazines where user_id = 'X';
```

Foi o que fizemos para remover as 64 duplicatas: contamos primeiro (64 pares), depois apagamos.

Um exemplo avançado do projeto: remover duplicatas

```sql
delete from public.magazines m
using public.magazines v                        -- compara a tabela com ELA MESMA (apelido v)
where m.user_id = 'DONO_ANTIGO'
  and v.user_id = 'DONO_ATUAL'
  and lower(trim(m.title)) = lower(trim(v.title))
  and m.issue is not distinct from v.issue      -- igualdade que trata NULL como valor comparável
  and m.year  is not distinct from v.year
  and m.month is not distinct from v.month;
```

Dois conceitos novos:

- `using`: junta a tabela com outra (ou com ela mesma) para decidir quais linhas apagar.
- `is not distinct from`: em SQL, `null = null` **não é verdadeiro** (é "desconhecido"). Esse operador considera dois `null` iguais, essencial quando há colunas opcionais.

`NULL` é um conceito à parte

`null` significa "**não sei**", e não "zero" nem "vazio". Por isso `where year = null` nunca acha nada: use `where year is null`.

---

4. Autenticação: quem é você?

O **Supabase Auth** cuida de cadastro, login e sessão.

O fluxo do login

```
1. Usuário digita e-mail e senha  →  supabase.auth.signInWithPassword(...)
2. O Supabase confere e devolve um TOKEN (JWT) assinado
3. O navegador guarda o token (localStorage)
4. Em cada pedido ao banco, o supabase-js envia o token junto
5. O banco lê o token e sabe: "este pedido vem do usuário X"
```

O **JWT** é um texto assinado que diz "este é o usuário X" e **não pode ser falsificado** sem a chave secreta do servidor. Dentro do Postgres, a função `auth.uid()` devolve o `id` desse usuário.

Duas chaves diferentes (a confusão mais perigosa para iniciantes)

Chave	Pode ficar no navegador?	O que permite
anon / publishable (`sb_publishable_...`)	✅ **Sim**, é pública por desenho	Só o que o RLS autorizar
service_role	❌ **Nunca**	**Ignora o RLS**: acesso total ao banco

A chave pública *não é perigosa* porque, sozinha, **só abre as portas que o RLS deixou abertas**. A `service_role` abre tudo, então só pode viver em servidores seus, jamais num app que roda no navegador.

Configurações de autenticação que você ajustou no painel

- **Allow new users to sign up** desligado → ninguém novo consegue criar conta.
- **Confirm email** ligado → quem se cadastrasse precisaria provar o e-mail.
- **Redirect URLs / Site URL** → os endereços para onde o Supabase pode mandar o usuário depois de confirmar um e-mail. É uma **lista de permitidos**: evita o ataque de *open redirect* (um link que levaria a vítima para um site falso após o login).

---

5. RLS: Row Level Security ⭐ (a ideia mais importante do capítulo)

O problema

O código do frontend roda no navegador do visitante, que pode abrir o console e escrever:

```
await supabase.from('magazines').delete()   // tenta apagar TUDO
```

Se o banco obedecesse, qualquer pessoa destruiria seus dados. Proteger só no frontend ("esconder o botão") não adianta: o atacante **não usa** o seu frontend.

A solução

Com o **RLS ligado**, o próprio banco **filtra as linhas** antes de responder, conforme **políticas** (*policies*) que você escreve. Sem política que permita, **nada** passa.

As políticas da tabela `magazines`

```
-- O dono pode LER as próprias revistas
create policy "Users can view their own magazines" on public.magazines
  for select using (auth.uid() = user_id);

-- O dono pode CRIAR revistas, mas só com o próprio user_id
create policy "Users can create their own magazines" on public.magazines
  for insert with check (auth.uid() = user_id);

-- O dono pode ATUALIZAR e APAGAR as próprias
create policy "Users can update their own magazines" on public.magazines
  for update using (auth.uid() = user_id);

create policy "Users can delete their own magazines" on public.magazines
  for delete using (auth.uid() = user_id);
```

Trecho	Significa
`for select` / `insert` / `update` / `delete`	A qual operação a regra se aplica
`using (condição)`	Quais linhas **já existentes** a operação pode enxergar/alterar
`with check (condição)`	O que uma linha **nova ou alterada** precisa satisfazer
`auth.uid() = user_id`	"o usuário do pedido é o dono da linha"

Tradução: *cada usuário só enxerga e mexe nas linhas em que* `user_id` *é o seu.*

O `with check` no INSERT: por que importa

Sem ele, um usuário poderia inserir uma revista com o `user_id` **de outra pessoa**. O `with check` exige que o `user_id` que você manda seja o **seu**.

O comportamento silencioso do RLS ⚠️

O RLS **não** responde "acesso negado" para leituras. Ele simplesmente **não devolve** as linhas proibidas. Um visitante sem login que consulta a tabela recebe `[]` (lista vazia), como se as revistas não existissem. O capítulo 17 conta o caso real.

> **Se uma consulta volta vazia e você tem certeza de que há dados, a primeira suspeita é o RLS**, antes de achar que é bug no código.

Quem é "visitante" para o banco?

O Supabase distingue **papéis** (*roles*):

Papel	Quem é
`anon`	Visitante **sem login** (só a chave pública)
`authenticated`	Usuário **logado**
`postgres` / `service_role`	Administração; **ignora o RLS**

⚠️ **Detalhe do SQL Editor:** o editor do painel pode rodar como `postgres` (enxerga tudo) ou simulando um usuário. Se um `delete` devolve `DELETE 0` mesmo havendo linhas, confira o papel selecionado ao lado do botão **Run**. Confirme com `select current_user;`.

---

6. Views: mostrar só algumas colunas

O problema que levou à view

A política antiga deixava **qualquer visitante** ler as revistas públicas, mas lia a **linha inteira**, incluindo `user_id` (identificador da sua conta) e `notes` (anotações privadas). O RLS filtra **linhas**, não **colunas**.

A solução: uma view

Uma **view** é uma consulta salva, que se comporta como uma tabela.

📁 `supabase/migrations/20261004190000_create_public_magazines_view.sql`:

```
create or replace view public.public_magazines as
select id, title, issue, year, month, condition, cover_model, cover_image_url, acquired, is_public, created_at
from public.magazines
where is_public = true;

grant select on public.public_magazines to anon, authenticated;
```

- Escolhemos **só as colunas seguras** (sem `user_id`, sem `notes`).
- O `where is_public = true` embutido garante que só as públicas aparecem.
- `grant select ... to anon` dá permissão de leitura ao visitante.

Em seguida, removemos a política que dava acesso direto à tabela:

```
drop policy if exists "Public magazines are viewable by everyone" on public.magazines;
```

Resultado: o visitante lê **só pela view**. Verificamos: a tabela direta devolve `[]`, a view devolve as 64.

A ordem das mudanças importa

1. Criar a view (**acrescenta**, não quebra nada);
2. Atualizar o código para ler da view e publicar;
3. **Só então** remover a política antiga.

Se a ordem fosse invertida, o site publicado ficaria sem dados durante o intervalo. **Mudanças de segurança e de estrutura seguem "primeiro adicione o novo, depois remova o velho".**

O aviso do painel de segurança

O Supabase marca views como **"Security Definer View"** (erro): ela roda com os direitos do **dono** da view e, por isso, ignora o RLS da tabela de baixo. Aqui isso é **intencional** (a view precisa ler linhas que o RLS esconderia do visitante, mas só entrega as colunas seguras). Mas é bom saber que o aviso existe e entender o porquê.

---

7. Storage: onde as imagens ficam

Imagens **não** vão para o banco. Bancos são ruins para arquivos grandes. O Supabase Storage guarda os arquivos e o banco guarda **só o endereço** (`cover_image_url`).

Conceitos

- **Bucket:** uma "pasta raiz" de arquivos (`magazine-covers`). Pode ser pública ou privada.
- **Objeto:** um arquivo dentro dele, com um caminho (`<user_id>/<uuid>.jpg`).

O caminho dos arquivos importa

```
const path = `${userId}/${crypto.randomUUID()}.${ext}`
```

A **pasta é o** `user_id` do dono, e o nome do arquivo é um **UUID aleatório**. Dois motivos:

1. **Segurança:** as regras do Storage verificam a pasta.
2. **Nunca sobrescrever:** nomes únicos impedem que um upload apague a capa de outra revista.

As políticas do Storage (também são RLS)

```sql
-- Qualquer um pode VER arquivos do bucket (o catálogo público precisa)
create policy "Anyone can view magazine cover files" on storage.objects
  for select to anon, authenticated using (bucket_id = 'magazine-covers');

-- Só pode ENVIAR/ALTERAR/APAGAR dentro da PRÓPRIA pasta
create policy "Users can upload their own magazine covers" on storage.objects
  for insert with check (bucket_id = 'magazine-covers' and auth.uid()::text = (storage.foldername(name))[1]);
```

`(storage.foldername(name))[1]` extrai a **primeira pasta** do caminho, e a regra exige que seja o `user_id` de quem está enviando.

A estrutura da URL pública

```
https://<projeto>.supabase.co/storage/v1/object/public/magazine-covers/<user_id>/<uuid>.jpg
```

Para **apagar**, precisamos só do trecho depois de `/magazine-covers/`. Por isso `magazineService.ts` tem `pathFromUrl`, que corta a URL.

Um ponto fraco conhecido (com a correção)

O bucket **não limita tipo nem tamanho** de arquivo. Qualquer usuário logado poderia subir um arquivo enorme ou de tipo estranho **na própria pasta**. A validação do formulário é só conforto, não segurança. A trava de verdade é no servidor:

```sql
update storage.buckets
set file_size_limit = 5242880,                                          -- 5 MB
    allowed_mime_types = array['image/jpeg', 'image/png', 'image/webp', 'image/avif']
where id = 'magazine-covers';
```

Como o cadastro está fechado, o risco hoje é baixo, mas **este é o tipo de defesa em camadas que um sistema profissional tem**. Tente aplicar no seu projeto de estudo.

Armazenamento órfão

Apagar a **linha** da tabela **não** apaga o arquivo no Storage. São duas coisas separadas, e o código precisa cuidar das duas (`deleteMagazine` apaga a linha e depois a imagem). Quando o dono de 69 revistas foi apagado, as imagens delas também sumiram: as linhas continuaram, apontando para arquivos que já não existiam (capas quebradas).

---

8. Migrations: o histórico do banco

Uma **migration** é um arquivo SQL que descreve **uma mudança** do banco. O conjunto, em ordem, conta como o banco chegou ao estado atual, como o git faz com o código.

📁 `supabase/migrations/`:

```
20261004190000_create_public_magazines_view.sql
20261004191000_remove_anon_read_on_magazines_table.sql
```

- **O prefixo é data e hora** (`2026-10-04 19:00:00`), que define a **ordem**.

- **Uma mudança por arquivo**, com nome que descreve.
- **Inclua como reverter** (em comentário), como na segunda migration.

Por que importa

Sem migrations, o banco existe só "na cabeça do painel": ninguém consegue **recriá-lo** em outro lugar ou saber **quem mudou o quê**. O repositório original que analisamos no começo tinha exatamente esse defeito: a coluna `is_public` existia no banco, mas **em nenhuma migration**.

⚠️ **Lacuna do nosso projeto:** as migrations de `supabase/migrations/` só cobrem a view e a remoção da política. A **tabela e as políticas básicas** (as dos blocos acima) foram criadas direto no painel e **não estão versionadas**. Escrever essa migration "de base" é um dos desafios do capítulo 16.

---

9. Painel do Supabase: onde olhar cada coisa

Quero...	Vou em...
Ver/editar linhas manualmente	**Table Editor**
Rodar SQL	**SQL Editor**
Ver usuários, ajustar login, e-mails	**Authentication**
Ver arquivos	**Storage**
Ver erros de segurança/performance	**Advisors**
Ver registros do que aconteceu	**Logs**
Pegar a URL e a chave pública	**Project Settings → API**

Erros comuns

Sintoma	Causa	Solução
Consulta volta `[]` mas há dados	RLS escondendo, ou papel errado	Confira políticas e `current_user`
`new row violates row-level security policy`	`with check` falhou (ex.: `user_id` diferente do seu)	Envie o `user_id` do usuário logado
`delete` / `update` mexeu em linhas demais	Esqueceu o `where`	Sempre rode o `select` antes
`where coluna = null` não acha nada	`null` não é igual a nada	`is null` / `is not distinct from`
Capa quebrada depois de apagar	Apagou a linha mas não o arquivo (ou o contrário)	Cuide das duas operações
Estimativa de linhas errada no painel	Contagem aproximada, desatualizada	Use `select count(*)`

Exercícios (em um projeto Supabase de estudo)

1. Crie uma tabela `wishlist` (`id`, `user_id`, `title`, `created_at`) com RLS e as quatro políticas do dono.
2. Insira 3 linhas e confira, em um `select`, que outro usuário **não** as enxerga.
3. Crie uma view `wishlist_public` sem a coluna `user_id`.
4. Escreva um `select` que mostre quantas revistas existem **por ano** (`group by year`).
5. Tente (num bucket de teste) enviar um arquivo para a pasta de **outro** usuário. Leia o erro.
6. **Desafio:** escreva a migration "de base" do projeto: tabela, RLS ligado, quatro políticas e as políticas do Storage. Teste recriando o banco do zero num projeto vazio.

✅ Checklist do capítulo 08

- ☐ Explico tabela, linha, coluna e chave primária.
- ☐ Escrevo `select`, `insert`, `update` e `delete` e **nunca** esqueço o `where`.
- ☐ Explico `NULL` e `is not distinct from`.
- ☐ Explico a diferença entre a chave pública e a `service_role`.
- ☐ Leio uma política RLS (`using` × `with check`) e traduzo para português.
- ☐ Explico por que a view `public_magazines` existe e por que a ordem das mudanças importou.
- ☐ Explico por que a pasta no Storage é o `user_id`.
- ☐ Explico o que é uma migration e a lacuna do projeto.

Próximo: 09. Ligando o app ao banco.

CAPÍTULO 09 DE 18

Ligando o app ao banco

Objetivo

Entender, linha por linha, como o frontend conversa com o Supabase: o cliente, a camada de serviço, o CRUD, o tratamento de erros, o upload de imagens e, principalmente, **como manter tudo consistente quando duas operações precisam andar juntas**.

---

1. O cliente do Supabase

📁 `src/lib/supabase.ts`:

```
import { createClient } from '@supabase/supabase-js'

const url = import.meta.env.VITE_SUPABASE_URL
const key = import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY

if (!url || !key) {
  throw new Error('Faltam VITE_SUPABASE_URL ou VITE_SUPABASE_PUBLISHABLE_KEY no .env.local')
}

export const supabase = createClient(url, key)
```

`createClient` devolve **um objeto** com tudo que o app precisa: `supabase.auth` (login), `supabase.from(...)` (tabelas), `supabase.storage` (arquivos). Criamos **um só**, no app inteiro, e todo mundo o importa.

Por que um só? Cada cliente guarda a sessão do usuário. Dois clientes "não se conheceriam".

---

2. A forma de toda resposta: `{ data, error }`

O `supabase-js` **não lança erro** quando o banco recusa algo. Ele devolve um objeto com **dois campos**, e você decide:

```
const { data, error } = await supabase.from('magazines').select('*')

if (error) { /* deu errado: error.message explica */ }
else       { /* deu certo: data tem o resultado */ }
```

Memorize o padrão: **depois de todo `await` do Supabase, olhe o `error` primeiro.**

---

3. A camada de serviço: `magazineService.ts`

📁 `src/lib/magazineService.ts` é **o único arquivo** que conversa com a tabela `magazines` e com o bucket.

Por que concentrar tudo ali?

- **Trocar de banco** mexe em um arquivo só.
- As telas ficam **limpas**: chamam `addMagazine(...)`, sem saber **como** isso é feito.
- Fica fácil achar "onde isso acontece?".

O padrão das funções do serviço

```
export async function listMyMagazines(userId: string): Promise<Magazine[]> {
  const { data, error } = await supabase
    .from('magazines')
    .select('*')
    .eq('user_id', userId)
    .order('created_at', { ascending: false })

  if (error) throw error      // ← converte "erro devolvido" em "erro lançado"
  return data
}
```

Convertemos o `error` devolvido em `throw`. Resultado: quem chama usa o `try/catch` normal, e o serviço **não decide** o que mostrar ao usuário. Cada camada faz uma coisa:

Camada	Responsabilidade
`magazineService.ts`	Falar com o banco; lançar erro se falhar
`Catalog.tsx`	Capturar o erro e decidir o que mostrar (`setError(...)`)

A cadeia de métodos (*query builder*)

```
supabase
  .from('magazines')          // qual tabela
  .select('*')                // quais colunas (* = todas)
  .eq('user_id', userId)      // WHERE user_id = userId
  .order('created_at', { ascending: false })   // ORDER BY created_at DESC
```

É SQL escrito como chamadas de função.

Método	SQL
`.eq(col, v)`	`where col = v`
`.select()`	`select`
`.insert({...})`	`insert`
`.update({...})`	`update ... set`
`.delete()`	`delete`
`.order(col, {ascending})`	`order by`
`.single()`	"espero **exatamente uma** linha"

`insert(...).select().single()`

```
.insert({ ...magazine, user_id: userId })
.select()          // devolva a linha que acabou de criar
.single()          // como UM objeto, não uma lista de um item
```

O `insert` sozinho não devolve a linha. Com `.select()`, ela volta **já com o** `id` que o banco gerou, o que precisamos para colocar o card novo na tela.

Por que `listMyMagazines` filtra com `.eq('user_id', userId)` se o RLS já protege?

Nesse momento da história, o RLS deixava o visitante ler **as públicas de qualquer um**. Um usuário logado leria as **suas** *mais* as públicas dos outros, e a "Minha coleção" ia misturar revistas alheias. **O RLS decide o que *pode* ser lido; o filtro decide o que *você quer* ver.** As duas camadas se complementam. Hoje, com a política pública removida, o filtro continua sendo uma boa defesa em profundidade.

---

4. O fluxo de uma tela: carregar dados

📁 `Catalog.tsx`:

```
const [magazines, setMagazines] = useState<Magazine[]>([])
const [error, setError] = useState<string | null>(null)

useEffect(() => {
  if (!userId) { setMagazines([]); return }
  let cancelled = false
  listMyMagazines(userId)
    .then((data) => { if (!cancelled) setMagazines(data) })
    .catch((e: Error) => { if (!cancelled) setError(e.message) })
  return () => { cancelled = true }
}, [userId])
```

Leia como uma história:

1. **Sem usuário?** Limpa a lista e sai.
2. **Com usuário:** pede as revistas ao serviço.
3. **Chegou?** Guarda no estado (a tela redesenha com os cards).
4. **Falhou?** Guarda a mensagem de erro (a tela mostra em vermelho).
5. **Saiu da tela antes de chegar?** O `cancelled` descarta a resposta atrasada (capítulo 05).

Os três estados de qualquer tela que carrega dados

Toda tela que busca dados precisa tratar **três** situações, não só o "deu certo":

Estado	No projeto
Carregando	`useSession` tem `loading`; `Catalog` mostra "Carregando..."
Erro	`error` vira `<p className="form__error">`

Vazio / sucesso	Lista vazia mostra uma mensagem; senão, os cards

> Um app amador só trata o sucesso. Um app profissional trata os três.

---

5. A regra "espere o banco antes de mudar a tela"

📁 `Catalog.tsx`:

```
async function confirmDelete() {
  if (!pendingMagazine) return
  try {
    setError(null)
    await deleteMagazine(pendingMagazine)                          // 1º o banco confirma
    setMagazines(magazines.filter((m) => m.id !== pendingDeleteId)) // 2º só então a tela muda
  } catch (e) {
    setError((e as Error).message)                                 // se falhou, a tela NÃO mudou
  }
  setPendingDeleteId(null)
}
```

Se o `await` falhar, o código **pula** para o `catch`, e o `setMagazines` **nunca roda**. A revista continua na tela, porque continua no banco. **A tela nunca mente sobre o que está salvo.**

O contrário: "atualização otimista"

Há outra filosofia: mudar a tela **imediatamente** e desfazer se o banco recusar. Dá sensação de velocidade, mas é mais complexa (você precisa desfazer, tratar falhas, sincronizar). Para um app pequeno, a abordagem simples (esperar) é mais segura e fácil de manter. Conhecer as duas é parte de saber escolher.

---

6. ⭐ Duas operações que precisam andar juntas: upload + gravação

Adicionar uma revista com capa são **dois passos em lugares diferentes**:

1. Enviar a imagem ao **Storage**;
2. Gravar a linha no **Postgres** com o endereço dessa imagem.

📁 `Catalog.tsx`:

```
async function handleAdd(newMagazine: NewMagazine, cover: File | null): Promise<boolean> {
  if (!userId) return false
  let coverUrl: string | undefined
  try {
    setError(null)
    if (cover) coverUrl = await uploadCover(cover, userId)                                  // 1º a imagem...
    const created = await addMagazine({ ...newMagazine, cover_image_url: coverUrl }, userId) // 2º a linha
    setMagazines([created, ...magazines])
    return true
  } catch (e) {
    // Se a imagem subiu mas a linha falhou, a imagem ficaria órfã: desfazemos.
    if (coverUrl) await removeCover(coverUrl).catch(() => {})
    setError((e as Error).message)
    return false
  }
}
```

O que pode dar errado, e o que o código faz

Falha em...	Estado do mundo	O que o código faz
Passo 1 (upload)	Nada foi gravado	O `catch` mostra o erro. Nada a desfazer
Passo 2 (gravação), **após** o 1	Há uma imagem no Storage **sem revista**	O `catch` apaga a imagem (`removeCover`)
Os dois deram certo	Tudo consistente	Atualiza a tela

Esse padrão, **"se o segundo passo falhar, desfaça o primeiro"**, chama-se **compensação**. É a versão simples de uma *transação* quando os passos moram em sistemas diferentes (Storage e Postgres não fazem parte da mesma transação).

O `.catch(() => {})` no `removeCover`

```
await removeCover(coverUrl).catch(() => {})
```

Aqui estamos **já dentro** do tratamento de um erro. Se a limpeza também falhar, não vamos mostrar um segundo erro ao usuário: ele já viu o principal. O `catch` vazio é **intencional** e **comentado**. Nunca use um `catch` vazio sem saber por quê.

E na hora de apagar? A ordem inverte

📁 `magazineService.ts`:

```
export async function deleteMagazine(magazine: Magazine): Promise<void> {
  const { error } = await supabase.from('magazines').delete().eq('id', magazine.id)
  if (error) throw error                        // 1º a linha

  if (magazine.cover_image_url) {
    await removeCover(magazine.cover_image_url).catch(() => {})   // 2º o arquivo
  }
}
```

Linha primeiro, arquivo depois. Por quê? Se fosse o contrário e a remoção da linha falhasse, você teria uma revista **sem capa** (pior: um dado visível quebrado). Do jeito certo, o pior caso é um **arquivo sobrando no Storage** (lixo invisível). **Escolha sempre a ordem cujo pior caso é o menos**

ruim.

Na edição: trocar a capa

`handleUpdate`: sobe a capa nova → atualiza a linha → só então apaga a capa **antiga**. Se algo falhar no meio, desfaz a nova.

---

7. Upload: o que acontece com um arquivo

📁 `magazineService.ts`:

```ts
export async function uploadCover(file: File, userId: string): Promise<string> {
  const ext = file.name.split('.').pop() ?? 'jpg'
  const path = `${userId}/${crypto.randomUUID()}.${ext}`

  const { error } = await supabase.storage.from(BUCKET).upload(path, file)
  if (error) throw error

  return supabase.storage.from(BUCKET).getPublicUrl(path).data.publicUrl
}
```

Passo a passo:

1. `file` é um objeto `File` do navegador (vem do `<input type="file">`).
2. `file.name.split('.').pop()` pega a **extensão** (`'capa.JPG'` → `'JPG'`).
3. `path` = `<id do usuário>/<id único>.<extensão>`.
4. `upload(path, file)` envia. O RLS do Storage confere se a pasta é a do usuário.
5. `getPublicUrl(path)` monta o endereço público (não faz pedido de rede; só monta o texto).

Antes do upload: prévia e validação (`MagazineForm.tsx`)

```ts
useEffect(() => {
  if (!cover) { setPreview(null); return }
  const url = URL.createObjectURL(cover)        // endereço TEMPORÁRIO, só no seu navegador
  setPreview(url)
  return () => URL.revokeObjectURL(url)         // libera a memória
}, [cover])
```

`URL.createObjectURL` cria um endereço (`blob:...`) para mostrar a imagem escolhida **sem enviar nada**.

```ts
if (file && !file.type.startsWith('image/')) { setCoverError('O arquivo precisa ser uma imagem.'); ... }
if (file && file.size > MAX_COVER_MB * 1024 * 1024) { setCoverError(`A imagem passa de ${MAX_COVER_MB} MB.`); ... }
```

⚠️ **Isto é conforto, não segurança.** Quem fala direto com a API pula essas checagens. A proteção real precisa estar no servidor (limites do bucket, capítulo 08).

---

8. Edição: atualizar só o que mudou

```ts
export async function updateMagazine(id: string, changes: Partial<NewMagazine>): Promise<Magazine> {
  const { data, error } = await supabase.from('magazines').update(changes).eq('id', id).select().single()
  if (error) throw error
  return data
}
```

`Partial<NewMagazine>` (capítulo 03): você pode enviar só alguns campos.

⚠️ O bug sutil: `undefined` × `null` ao editar

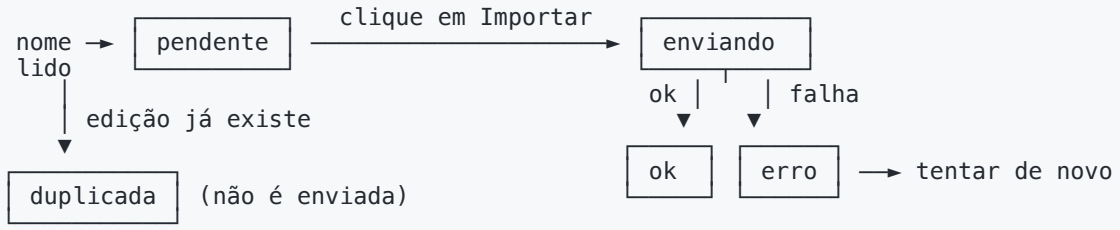
```ts
cover_model: coverModel.trim() || null      // ✅
cover_model: coverModel.trim() || undefined // ❌ ao EDITAR
```

O JavaScript **descarta** campos `undefined` ao montar o pedido. Para o banco, "você não mencionou esse campo" significa **"mantenha o valor antigo"**. Se o usuário **apagar** o modelo da capa e você mandar `undefined`, o valor antigo **fica**. Só `null` significa "esvazie isto". Foi um dos casos que o teste manual pegou.

---

9. Importação em massa: uma máquina de estados

📁 `src/components/BulkImport.tsx`. Cada arquivo escolhido vira um **item** com um estado:



Tipo dos estados (`type Status = 'pendente' | 'enviando' | 'ok' | 'erro' | 'duplicada'`): uma **união literal** (capítulo 03), então o TS impede estados inexistentes.

Cada item tem seu próprio estado

```ts
function update(id: string, changes: Partial<Item>) {
  setItems((current) => current.map((item) => (item.id === id ? { ...item, ...changes } : item)))
}
```

Mudar **um** item sem tocar nos outros: `map` + spread, forma funcional (as três ideias do capítulo 05 juntas).

Enviando em sequência

```
for (const item of items) {
  if (item.status !== 'pendente' && item.status !== 'erro') continue
  update(item.id, { status: 'enviando', error: undefined })
  try {
    coverUrl = await uploadCover(item.file, userId)
    const magazine = await addMagazine({ ... }, userId)
    imported.push(magazine)
    update(item.id, { status: 'ok' })
  } catch (e) {
    if (coverUrl) await removeCover(coverUrl).catch(() => {})
    update(item.id, { status: 'erro', error: (e as Error).message })
  }
}
```

Observações:

- `for ... of` **com** `await` **dentro** envia **um de cada vez**. Alternativa: `Promise.all` envia **todos de uma vez**. Sequencial é mais lento, mas **mais simples de acompanhar** (a barra de progresso faz sentido) e não sobrecarrega o servidor. Com 100 arquivos, `Promise.all` abriria 100 conexões simultâneas.
- **Um erro não para o lote:** o `try/catch` está **dentro** do loop, então um item que falha vira "erro" e o loop segue para o próximo.
- **Os que deram erro podem ser reenviados**: o `if` aceita `'erro'` também.
- **A compensação aparece de novo** (`removeCover` no `catch`): é o mesmo padrão do `handleAdd` .

Detectar duplicatas

```
const knownIssues = new Set([
  ...existing.map((m) => m.issue).filter(Boolean),    // edições já no catálogo
  ...items.map((i) => i.issue).filter(Boolean),       // edições já neste lote
])
const duplicate = parsed.issue !== null && knownIssues.has(parsed.issue)
if (parsed.issue) knownIssues.add(parsed.issue)
```

O `Set` torna a verificação `has(...)` instantânea e evita repetição. Note o detalhe: o número **entra no** `Set` **depois** de testado, assim duas capas iguais no mesmo lote também são detectadas.

Limite conhecido: só detecta duplicatas **com número de edição**. Capas sem número no nome nunca são marcadas.

---

10. Autenticação no app

📁 `AuthForm.tsx` :

```
const { error } = await supabase.auth.signInWithPassword({ email, password })
if (error) setMessage(error.message)
// Se deu certo, não fazemos nada: o useSession percebe sozinho.
```

Note que o formulário **não avisa o** `Catalog` de que o login deu certo. Ele só chama o login. O `useSession`, que **escuta** mudanças (`onAuthStateChange`), percebe a nova sessão, atualiza seu estado, e a tela troca. Os dois lados conversam por um terceiro (o Supabase).

O formulário é **só de login**: o cadastro está fechado no Supabase, então removemos o modo "criar conta" para não deixar um botão que sempre daria erro.

Erros comuns

Sintoma	Causa	Solução
`error` ignorado, dados `null`	Esqueceu de checar `error`	`if (error) throw error` logo após o `await`
Tela mostra algo que não foi salvo	Mudou a tela **antes** do `await`	Espere o banco, depois `setState`
Imagens sobrando no Storage	Falhou o 2º passo e nada desfez o 1º	Compensação no `catch`
Campo "apagado" volta ao editar	Mandou `undefined`	Mande `null`
Lista "pula" ou perde itens	Resposta antiga sobrescreve a nova	Variável `cancelled` na limpeza
Lote inteiro para num erro	`try/catch` fora do loop	Mova o `try/catch` para dentro

Exercícios

1. Adicione ao serviço `countMyMagazines(userId)` usando `select('*', { count: 'exact', head: true })` e mostre o total no topo da página.
2. Faça o `handleTogglePublic` ser **otimista**: mude a tela primeiro e desfaça se o banco recusar. Compare a complexidade.
3. Em `BulkImport`, troque o loop sequencial por **três envios por vez** (um "pool" simples). Quanto ficou mais rápido? O que ficou mais difícil?
4. Adicione um botão **"Excluir selecionadas"** na coleção (seleção múltipla). Pense em: o que apagar primeiro? E se uma falhar no meio?
5. **Desafio:** escreva o fluxo completo de **editar capa** num diagrama (como o da seção 6) listando o que acontece em cada ponto de falha.

✅ Checklist do capítulo 09

- ☐ Verifico `error` depois de todo `await` do Supabase.
- ☐ Explico por que o serviço faz `throw` e a tela faz `try/catch`.
- ☐ Explico "espere o banco antes de mudar a tela" e a alternativa otimista.
- ☐ Descrevo o padrão de compensação com `handleAdd`.
- ☐ Explico por que deletar usa a ordem "linha, depois arquivo".
- ☐ Sei a diferença entre `undefined` e `null` num `update`.

- ☐ Explico a máquina de estados do `BulkImport` e por que o envio é sequencial.

Próximo: 10. Lógica pura e testes.

CAPÍTULO 10 DE 18

Lógica pura e testes

Objetivo

Entender o que são testes automatizados, por que escrevemos testes para `filters.ts` e `parseCoverFilename.ts` (e não para as telas), como o Vitest funciona e como o **TDD** (escrever o teste primeiro) muda a forma de programar.

---

1. Por que testar

Sem testes, a única forma de saber se algo ainda funciona é **abrir o app e clicar em tudo**. Isso é lento, cansativo e ninguém faz com a frequência necessária. Com 30 testes, você descobre em **um segundo**:

```
 Test Files  2 passed (2)
      Tests  30 passed (30)
```

Testes são uma **rede de segurança**. Você pode refatorar, otimizar e acrescentar funcionalidades **sem medo** de quebrar o que já funcionava. E funcionam como **documentação executável**: o nome de cada teste descreve um comportamento.

---

2. O que dá para testar facilmente: funções puras

Uma **função pura**:

1. Dado o mesmo argumento, **sempre** devolve o mesmo resultado.
2. **Não** altera nada fora dela (sem rede, sem tela, sem banco, sem `Date.now()` solto).

```
// pura: só depende do que recebe
export function parseCoverFilename(filename: string): ParsedCover { ... }

// NÃO pura: depende do banco e de internet
export async function listMyMagazines(userId: string) { ... await supabase... }
```

Funções puras são **fáceis de testar**: você monta a entrada, chama, confere a saída. Sem servidor, sem navegador, sem preparar nada. Os 30 testes rodam em ~100 milissegundos.

> **Consequência para o design:** é por isso que a lógica de filtro mora em `lib/filters.ts` (puro) e não dentro do `Catalog.tsx` (cheio de estado e efeitos). **Código que mistura tudo é difícil de testar. Separar a lógica da tela é o que torna o teste possível.**

---

3. Vitest em 5 minutos

```
import { describe, expect, it } from 'vitest'
import { parseCoverFilename } from './parseCoverFilename'

describe('parseCoverFilename', () => {                    // um grupo, por assunto
  it('extrai edição, ano, mês e título', () => {          // UM comportamento, em português
    expect(parseCoverFilename('99__12__Playboy-No-293-Joana-Prado-Feiticeira.jpg')).toEqual({
      title: 'Joana Prado Feiticeira',
      issue: '293',
      year: 1999,
      month: 12,
    })
  })
})
```

Peça	Papel
`describe('assunto', () => {...})`	Agrupa testes relacionados
`it('faz tal coisa', () => {...})`	Um teste. O texto é uma frase que descreve o comportamento
`expect(valor)`	"Quero conferir este valor"
`.toEqual(esperado)`	Igualdade **profunda** (compara o conteúdo de objetos e arrays)
`.toBe(esperado)`	Igualdade estrita (`===`), para valores simples
`.toMatchObject({...})`	O objeto **contém pelo menos** estes campos
`.toHaveLength(n)`	O array tem `n` itens

Rodando

```
npm test               # roda tudo uma vez
npm run test:watch     # fica rodando a cada salvamento (ótimo enquanto escreve)
```

Um arquivo de teste vive **ao lado** do que testa e termina em `.test.ts` (`filters.test.ts`).

---

4. A estrutura de um bom teste: Arrange, Act, Assert

```
it('ignora acentos e maiúsculas', () => {
  // ARRANGE (preparar): dados conhecidos
  const list = [
    makeMagazine({ title: 'Edição de Verão', cover_model: 'Ana Souza' }),
    makeMagazine({ title: 'Especial de Inverno', cover_model: null }),
  ]
  // ACT (agir): chamar a função
  const result = applyFilters(list, filters({ search: 'EDICAO' }))
  // ASSERT (conferir): o resultado é o esperado?
  expect(titles(result)).toEqual(['Edição de Verão'])
})
```

Três fases: **prepare** os dados, **execute** a função, **confira** o resultado. Se um teste não segue isso, costuma estar fazendo coisa demais.

Regras de um bom teste

1. **Um assunto por teste.** Quando falha, você sabe o motivo sem investigar.
2. **O nome diz o comportamento** ("ignora acentos e maiúsculas"), não a implementação ("testa o normalize").
3. **Independente:** um teste não pode depender de outro rodar antes.
4. **Rápido e determinístico:** sem depender de rede, hora ou sorte.

---

5. A "fábrica" de dados de teste

📁 `src/lib/filters.test.ts`:

```
function makeMagazine(overrides: Partial<Magazine> = {}): Magazine {
  return {
    id: crypto.randomUUID(),
    title: 'Revista',
    year: 2000,
    month: 1,
    condition: 'Bom',
    acquired: true,
    is_public: false,
    ...overrides,                // o que o teste informar vence os valores padrão
  }
}
```

Uso: `makeMagazine({ title: 'Banana', year: 1990 })`.

Sem a fábrica, cada teste repetiria todos os campos de uma `Magazine`. Com ela, **cada teste só diz o que importa para ele**, e o resto vem de valores padrão. Os testes ficam curtos e legíveis, e se `Magazine` ganhar um campo, você altera **um** lugar.

Pequenos auxiliares ajudam a leitura:

```
const titles = (list: Magazine[]) => list.map((m) => m.title)
```

---

6. O que testar: casos de borda

Os testes mais valiosos não são os "caminho feliz". São os **cantos**, onde os bugs moram.

📁 O que `filters.test.ts` cobre além do óbvio:

Caso de borda	Por que importa
Lista vazia	Funções costumam quebrar com `[]`
`cover_model` é `null`	A busca junta texto; `null` não pode virar `"null"`
Espaços nas pontas (`' verão '`)	O usuário digita com espaço sem perceber
Acentos e maiúsculas	`EDICAO` deve achar `Edição`
Desempate de ano pelo mês	Duas revistas do mesmo ano

Revistas **sem ano**	Devem ir para o **fim** nas duas direções
Não alterar a lista original	A regra de imutabilidade do React

📁 `parseCoverFilename.test.ts`:

Caso de borda	Por que importa
Sublinhado simples (`89_03`) em vez de duplo	O nome real de um arquivo tinha esse formato
Mês 13	Deve virar `null`, não um mês inválido
Sem prefixo de data	Ano e mês `null`, edição mantida
Nome fora do padrão	Cai no título limpo
Arquivo `.jpg` sem nome	`'Sem título'`
`Dra.-Silva.jpg`	Só a **última** extensão sai

Pergunta para fazer a cada função: "o que acontece se a entrada for vazia? `null`? gigante? esquisita?"

---

7. TDD: o teste vem primeiro

TDD = *Test-Driven Development*. O ciclo:

```
1. 🔴 VERMELHO   Escreva um teste do comportamento desejado. Rode: ele FALHA.
2. 🟢 VERDE      Escreva o código MÍNIMO para o teste passar.
3. 🔵 AZUL       Melhore o código (refatore) mantendo os testes verdes.
   └─────────── repita para o próximo comportamento
```

Como foi com o `parseCoverFilename`

1. Escrevemos **11 testes** descrevendo como o nome do arquivo deveria ser lido.
2. Rodamos `npm test`. Resultado:

   ```
   FAIL  src/lib/parseCoverFilename.test.ts
   Error: Cannot find module './parseCoverFilename'
   ```

   Falhou, como esperado: o módulo nem existia. (Um teste que nunca falhou não prova nada.)
3. Escrevemos a função. Rodamos de novo: **30 passando** (19 antigos + 11 novos).

Por que é bom

- **Você pensa no comportamento antes do código.** O teste vira a *especificação*.
- **Você só escreve o que é necessário.**
- **Você ganha os testes de graça**, junto com o código.
- **Menos medo de mexer depois.**

> TDD não é obrigatório em tudo. Brilha em **lógica com regras** (formatar, filtrar, validar, calcular). Em telas, costuma ser exagero.

8. Um teste que nunca falha não vale nada: o teste de mutação manual

Como saber se os testes realmente pegam erros? **Quebre o código de propósito.**

Fizemos isto com `filters.ts`: trocamos

```
(b.year ?? 0) - (a.year ?? 0)        // maior para o menor
```

por

```
(a.year ?? 0) - (b.year ?? 0)        // inverte a ordem
```

Resultado: **exatamente um teste falhou**, o de "mais novas primeiro", com a mensagem `expected ['Banana', 'Cereja', 'Abacaxi'] to deeply equal ['Abacaxi', 'Cereja', 'Banana']`. Restauramos, e os 17 passaram.

Isso prova que os testes **estão vigiando** aquela linha. Existem ferramentas que fazem isso automaticamente ("testes de mutação"), mas fazer à mão uma vez é uma excelente forma de aprender.

🥒 **Experimente:** em `filters.ts`, troque `!==` por `===` na linha do filtro de condição. Rode `npm test`. Quais testes falham? Desfaça com `git restore src/lib/filters.ts`.

---

9. O que não estamos testando (e como seria)

Camada	Tem teste?	O que seria preciso
Funções puras (`lib/`)	✅ Sim	Vitest, o que já temos
Componentes React	❌ Não	**Testing Library**: renderiza o componente num "navegador falso" e simula cliques
Fluxo completo no navegador	❌ Não	**Playwright**: abre um navegador de verdade e usa o site
Serviços que falam com o Supabase	❌ Não	Um **mock** (substituto falso) do cliente, ou um banco de teste

Uma estratégia comum é a **pirâmide de testes**: **muitos** testes de lógica (rápidos e baratos), **alguns** de componentes, **poucos** ponta a ponta (lentos e frágeis). O projeto tem a base da pirâmide; os andares de cima são o próximo estudo.

---

10. Testes + git + deploy

O fluxo ideal antes de cada commit:

```
npx tsc -b && npm test && npm run build
```

Num projeto profissional, isso roda **automaticamente** a cada `git push` (integração contínua), e o deploy só acontece se tudo passar. Aqui, quem faz isso é você, à mão.

Erros comuns

Sintoma	Causa	Solução
Teste passa, mas o app está quebrado	O teste não cobre aquele caminho	Escreva um teste que **reproduza** o bug primeiro
Teste frágil: falha por motivos aleatórios	Depende de hora, rede ou ordem	Torne-o determinístico e independente
Muitos testes quebram a cada mudança pequena	Testam a implementação, não o comportamento	Teste entradas e saídas, não detalhes internos
`expected [..] to be [..]` com arrays iguais	Usou `toBe` para arrays/objetos	Use `toEqual`
Nunca tem tempo para testar	Testes demais no final, em vez de aos poucos	Escreva o teste **junto** com a função

Exercícios

1. Rode `npm run test:watch`, estrague uma linha de `applyFilters` e veja quais testes falham. Desfaça.
2. **TDD de verdade:** escreva `formatMagazineDate(month, year)` que devolva `'03/1999'`, `'1999'` ou `''`. Primeiro **os testes** (vermelho), depois a função (verde). Depois use-a no `MagazineCard`.
3. Escreva um teste de ordenação por título com **dois títulos iguais**. O que você espera? O código atual atende?
4. Pesquise `@testing-library/react` e escreva um teste que renderize o `MagazineCard` e confira que o título aparece e que, **sem** `onDelete`, o botão "Deletar" **não** existe.
5. **Desafio:** adicione `vitest --coverage` (pacote `@vitest/coverage-v8`) e descubra quais linhas de `lib/` ainda não são cobertas por testes.

✅ Checklist do capítulo 10

- ☐ Explico o que é uma função pura e por que é fácil de testar.
- ☐ Leio e escrevo `describe`, `it` e `expect(...).toEqual(...)`.
- ☐ Organizo um teste em Arrange, Act, Assert.
- ☐ Uso uma fábrica de dados para testes curtos.
- ☐ Lembro de testar casos de borda (vazio, `null`, espaços, acentos).
- ☐ Conto o ciclo vermelho, verde, azul com o `parseCoverFilename`.
- ☐ Sei por que "quebrar de propósito" valida os testes.
- ☐ Sei o que o projeto ainda não testa e como seria.

Próximo: 11. Git e GitHub.

CAPÍTULO 11 DE 18

Git e GitHub

Objetivo

Dominar o fluxo básico que você usou ao longo do projeto (`status`, `add`, `commit`, `push`), entender o que cada comando faz por baixo, saber **desfazer** coisas com segurança e conhecer o próximo nível (branches e Pull Requests).

---

1. A ideia: fotografias do projeto

Git é um sistema que guarda o **histórico** do seu código como uma sequência de **fotografias** (*commits*). Cada commit registra:

- o estado de **todos os arquivos** naquele momento;
- **quem** fez, **quando**;
- uma **mensagem** dizendo o que mudou e por quê.

Você pode voltar a qualquer fotografia antiga. Por isso o git é a sua **rede de segurança**: experimente à vontade, porque o que está commitado sempre se recupera.

GitHub é um site que guarda uma **cópia do repositório na nuvem**. Git é a ferramenta; GitHub é um serviço que a usa. (Existem outros, como GitLab.) O Vercel lê o GitHub para publicar o site.

---

2. As três "áreas" do git (o modelo mental que resolve 90% das confusões)



Etapa	Pergunta que responde
Editar	Você muda arquivos normalmente
`add`	"**Quais** mudanças vão entrar na próxima foto?"
`commit`	"Tire a foto, com esta mensagem"
`push`	"Envie as fotos novas para o GitHub"

⚠️ **Por isso "fazer o push" sozinho não envia suas edições.** O `push` só envia **commits**. Edições não commitadas ficam só no seu computador. Foi exatamente o que aconteceu quando você pediu "outro push" e não havia nada para enviar: a sua mudança estava só na pasta de trabalho.

---

3. Os comandos, um por um

`git status`: "o que está acontecendo?"

O comando que você mais usa. **Rode sempre antes de qualquer outro.**

```
git status          # completo
git status -sb      # curto: uma linha por arquivo + a situação da branch
```

Saída curta:

```
## main...origin/main [ahead 1]
 M src/pages/PublicCatalog.tsx
?? estudos/
```

Símbolo	Significa
`M` (modified)	Arquivo existente que mudou
`??`	Arquivo **novo**, que o git ainda não conhece
`A`	Novo e já preparado (`add`)
`[ahead 1]`	Você tem **1 commit** que ainda não foi enviado ao GitHub
`main...origin/main` sem nada depois	Tudo sincronizado

`git diff`: "o que exatamente mudou?"

```
git diff                          # mudanças ainda não preparadas
git diff src/pages/Catalog.tsx    # só um arquivo
```

Mostra linha por linha: `-` (em vermelho) removida, `+` (em verde) adicionada. **Leia o diff antes de commitar**: foi assim que achamos o seu `<h1></h1>` vazio.

`git add`: escolher o que entra

```
git add src/pages/Catalog.tsx       # um arquivo
git add src/                        # uma pasta
git add -A                          # TUDO (use com cuidado: confira antes com git status)
```

⚠️ `git add -A` é conveniente, mas pode levar junto algo que você não queria (um arquivo de teste, uma edição esquecida). **Prefira nomear os arquivos**, ou revise o `git status` logo antes.

`git commit`: tirar a foto

```
git commit -m "Remove o título da página do catálogo público"
```

A mensagem fica no histórico **para sempre**. Veja a seção 5 sobre como escrevê-las.

`git log`: ver o histórico

```
git log --oneline            # uma linha por commit
git log --oneline -5         # só os 5 últimos
```

O histórico real do projeto (o mais recente primeiro) tem este formato:

```
61843e9 Ícone da aba com o coelho em alta resolução
ecf29fa Troca o ícone da aba pelo coelhinho
b66359e Remove o título da página do catálogo público
2e7fc52 Modo mosaico e botões de visualização só com símbolos
647caad Importação em massa de capas, com leitura do nome do arquivo
...
cea1611 Catálogo de revistas: login, CRUD, capas, busca e edição
```

Cada código (`61843e9`) é o **identificador** da fotografia.

`git push` e `git pull`

```
git push        # envia seus commits ao GitHub
git pull        # traz commits do GitHub que você ainda não tem
```

`git remote`: onde fica a cópia da nuvem

```
git remote -v                 # mostra os endereços
git remote add origin <url>   # registra o endereço (feito uma vez)
```

`origin` é só o **apelido** convencional do repositório remoto principal. O `-u` no primeiro push (`git push -u origin main`) memoriza o destino, e depois basta `git push`.

---

4. Desfazendo coisas (a parte que dá tranquilidade)

Situação	Comando	Cuidado
Quero **descartar** minhas edições num arquivo, voltando ao último commit	`git restore src/App.tsx`	⚠️ **Perde as edições, sem volta**
Tirei um arquivo da área de preparação (`add` por engano)	`git restore --staged arquivo`	Seguro: mantém a edição
Errei a **mensagem** do último commit (ainda não enviado)	`git commit --amend -m "nova mensagem"`	Só se ainda **não** deu `push`
Quero **desfazer um commit já enviado** sem reescrever a história	`git revert <código>`	Cria um commit novo que desfaz o anterior
Quero **ver** como o projeto era antes	`git log` + `git show <código>`	Só leitura

A regra para mexer sem medo

> **Commite antes de arriscar.** Tudo que está commitado se recupera. Tudo que não está, não.

O `git restore` é seguro justamente porque, se você commita com frequência, o pior que perde são alguns minutos de edição.

Nunca faça (sem entender) em um repositório compartilhado

`git push --force` e `git reset --hard`: reescrevem ou descartam história. Podem apagar o trabalho de outras pessoas e **não têm desfazer fácil**.

---

5. Mensagens de commit que ajudam o seu eu do futuro

Bom commit: pequeno, sobre **um assunto**, com mensagem que explica o **porquê**.

```
✅ Card esconde a condição quando não informada
✅ Revistas novas entram públicas; ano e mês passam a ser opcionais
✅ Adiciona vercel.json para rotas da SPA

❌ ajustes
❌ mudanças
❌ asdf
```

Estrutura recomendada:

```
Linha de resumo curta (até ~70 caracteres), no presente

Corpo opcional: explica o PORQUÊ, o que foi considerado, o que mudou.
Por exemplo: "Sem a regra, abrir /catalog direto retorna 404, porque
o servidor procura um arquivo com esse nome."
```

O que significa a linha `Co-Authored-By: ...` que aparece nos commits

Nos commits que fizemos juntos, a última linha indica **coautoria**: um jeito padrão do git de registrar que mais de uma "mão" participou. É só texto na mensagem, que o GitHub reconhece e exibe.

Commits pequenos e frequentes

Um commit gigante ("fiz tudo ontem") é impossível de revisar e de desfazer parcialmente. **Um commit por ideia** permite voltar atrás só no que deu errado. Neste projeto, repare que o visual, os testes e o ícone viraram commits **separados**.

---

6. O `.gitignore`: o que o git não deve guardar

📁 `.gitignore`:

```
node_modules       ← milhares de arquivos baixados; o npm recria
dist               ← o resultado do build; se recria
*.local            ← .env.local e similares: SEGREDOS
.env*              ← qualquer arquivo de ambiente
.vercel            ← ligação local com a hospedagem
.DS_Store          ← lixo do macOS
```

Duas regras de ouro:

1. **Nunca** commite segredos (chaves, senhas, tokens). O histórico do git **é permanente**: se um segredo entrar num commit e você apagar depois, **ele continua no histórico**. A única solução segura é **trocar a chave**.
2. Só o que se **reproduz** (`node_modules`, `dist`) fica de fora. O `package-lock.json`, que garante as mesmas versões para todos, **entra** no git.

Antes de commitar algo novo, uma verificação de 5 segundos:

```
git status        # esse arquivo deveria estar aqui?
```

Foi assim que conferimos que o `.env.local` **não** estava na lista antes do primeiro commit.

---

7. Fazendo isso pelo IntelliJ

Ação	Atalho/Caminho
Janela de commit (com o diff ao lado)	`⌘K`
Commit and Push	Setinha ao lado de **Commit** → *Commit and Push...*
Só push	`⌘⇧K`
Ver histórico	Aba **Git** (canto inferior) → *Log*
Ver o que mudou em um arquivo	Clique duplo na lista da janela de commit
Descartar edições de um arquivo	Clique direito → *Rollback* (equivale ao `restore`)
Linhas alteradas na margem do editor	Barras **verdes** (novas) e **azuis** (alteradas)

A interface e o terminal fazem **exatamente** a mesma coisa. Aprenda os comandos no terminal (sempre funciona em qualquer lugar) e use a interface para ganhar velocidade.

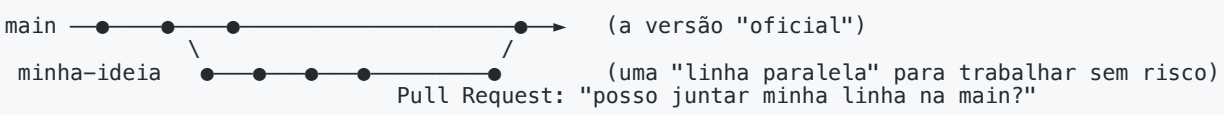
Problemas comuns no push

Mensagem	Significado	O que fazer
Pede usuário e senha	O GitHub **não aceita mais senha** em pushes	Use um **Personal Access Token**, ou o login do IntelliJ (*Settings* → *Version Control* → *GitHub*)
`rejected ... non-fast-forward`	O GitHub tem commits que você não tem	`git pull` primeiro; **não force**
`Updates were rejected`	O mesmo de cima	`git pull` e resolva conflitos, se houver

---

8. Branches e Pull Requests (o próximo nível)

Até agora trabalhamos só na `main`. Em equipes (e em projetos maiores), o fluxo é:




```
git switch -c adiciona-wishlist       # cria e entra numa branch nova
# ... edita, commita à vontade ...
git push -u origin adiciona-wishlist  # envia a branch
# No GitHub: abrir um Pull Request, revisar, aprovar, "Merge"
```

Por que usar branches?

- A `main` fica **sempre funcionando** (e é o que o Vercel publica).
- Você experimenta sem risco; se não der certo, descarta a branch.
- O **Pull Request** é o momento de **revisar** o código antes de juntar.

O Vercel também cria um **link de pré-visualização** para cada branch/PR, para você testar antes de publicar.

---

9. Um fluxo de trabalho diário completo

```
git status                      # 1. onde estou? tem algo pendente?
# ... trabalha no código ...
npx tsc -b && npm test          # 2. está tudo bem?
git status                      # 3. o que mudei?
git diff                        # 4. confiro linha por linha
git add src/components/X.tsx    # 5. escolho o que entra
git commit -m "Descreve a mudança"   # 6. tiro a foto
git push                        # 7. envio (e o Vercel publica sozinho)
```

Erros comuns

Sintoma	Causa	Solução
"Fiz push e não subiu nada"	Não fez o commit antes	`git status`, depois `add` e `commit`
Arquivo secreto entrou no histórico	Faltou o `.gitignore`	Troque a chave **imediatamente** e corrija o `.gitignore`
`git add -A` levou algo indesejado	Não conferiu o status	Prefira `add` com nomes de arquivo; confira antes
Perdi minhas edições	Usou `git restore` com edições não commitadas	Commite com frequência
Commit gigante e confuso	Acumulou muito trabalho	Commits menores, um por ideia

Exercícios

1. Rode `git log --oneline` e escolha um commit. Com `git show <código>`, descreva em uma frase o que ele mudou.
2. Faça uma mudança pequena, rode `git diff`, comite com uma boa mensagem e confira com `git log -1`.
3. Faça outra mudança e **descarte-a** com `git restore`. Confirme que sumiu.
4. Crie uma branch `experimento`, faça um commit nela e volte para a `main` com `git switch main`. Seu commit "sumiu"? Volte à branch e confira. Depois apague com `git branch -D experimento`.
5. Escreva, para os seus últimos 3 commits, mensagens **melhores** e justifique.
6. **Desafio:** no GitHub, abra um Pull Request de uma branch pequena para a `main` e junte (*Merge*). Veja o Vercel gerar o deploy.

✅ Checklist do capítulo 11

- ☐ Explico as três áreas e por que `push` sozinho não envia edições.
- ☐ Uso `status`, `diff`, `add`, `commit`, `log` e `push` sem consultar.
- ☐ Sei descartar edições (`restore`) e desfazer um commit enviado (`revert`).
- ☐ Escrevo mensagens que explicam o porquê.
- ☐ Sei o que vai e o que não vai no git, e por que segredos nunca entram.
- ☐ Faço commit e push pelo IntelliJ e pelo terminal.
- ☐ Explico o fluxo de branches e Pull Requests.

Próximo: 12. Build e deploy.

CAPÍTULO 12 DE 18

Build e deploy: do seu computador ao mundo

Objetivo

Entender o que acontece entre "o código funciona na minha máquina" e "qualquer pessoa abre o site": o **build**, a **hospedagem**, as **variáveis de ambiente em produção**, o **deploy automático** e como investigar quando algo não funciona no site publicado.

---

1. Dois mundos: desenvolvimento e produção

	Desenvolvimento (`npm run dev`)	**Produção (o site publicado)**
Endereço	`localhost:5173`	`vault-keeper-zeta.vercel.app`
Quem acessa	Só você	Qualquer pessoa
Código	Traduzido na hora, legível, com ajudas	Compactado e otimizado
Foco	Rapidez para você trabalhar	Rapidez para o visitante carregar
Variáveis de ambiente	Do seu `.env.local`	Do painel da hospedagem

Muita coisa que funciona em um pode **falhar no outro**. Conhecer as diferenças economiza horas de confusão.

---

2. O build: `npm run build`

```
"build": "tsc -b && vite build"
```

Dois passos encadeados por `&&` (o segundo só roda se o primeiro passar):

1. `tsc -b`: o TypeScript confere **todos os tipos**. Se houver um erro de tipo, o build **para**. Esse é o porquê de uma checagem de tipos "quebrar" o deploy.
2. `vite build`: junta, traduz e otimiza tudo.

O que o Vite produz em `dist/`

```
dist/
├── index.html
├── favicon.svg ... (arquivos de public/ copiados)
└── assets/
    ├── index-DPhWO8gg.js      ← TODO o JavaScript (React + seu código), compactado
    └── index-D8MUNvAy.css     ← TODO o CSS
```

Repare nos nomes com códigos estranhos (`DPhWO8gg`): são um **hash do conteúdo**. Quando você muda o código, o hash muda, o navegador percebe que é um arquivo **novo** e baixa de novo. Se o nome fosse sempre o mesmo, o navegador usaria uma cópia antiga em cache e o visitante veria o site velho. Esse truque se chama *cache busting*.

O que o Vite faz com o código

- **Minificação:** remove espaços, encurta nomes (`handleSubmit` → `a`), tira comentários. O arquivo cai de centenas para dezenas de KB.
- **Tree-shaking:** descarta o que você importou mas não usou.
- **Empacotamento:** junta dezenas de arquivos em poucos, para menos pedidos de rede.
- **Compressão:** o servidor ainda comprime (gzip) na entrega. O build mostra: `487 kB (141 kB gzip)`.

🧪 **Experimente:** rode `npm run build` e depois `npm run preview`. O `preview` serve a pasta `dist/` localmente: é **exatamente** o que vai para o ar. Abra o endereço e confira se funciona **antes** de publicar.

O site publicado é só arquivos

`dist/` contém **arquivos estáticos** (HTML, CSS, JS, imagens). Não há servidor seu rodando código. Toda a "lógica de servidor" está no Supabase. Por isso hospedar é barato e simples: qualquer lugar que entregue arquivos serve.

---

3. Variáveis de ambiente em produção ⚠️

O seu `.env.local` **não vai** para o git (de propósito). Então a hospedagem não o tem. Se você publicar sem configurar as variáveis, o site abre em **branco**, porque `supabase.ts` lança:

```
Faltam VITE_SUPABASE_URL ou VITE_SUPABASE_PUBLISHABLE_KEY no .env.local
```

Por isso cadastramos as duas variáveis **no painel do Vercel**, para o ambiente *Production*.

Um fato que muda como você pensa em segurança

As variáveis `VITE_...` são **substituídas pelos valores durante o `build`**. O texto do valor passa a fazer parte do arquivo JavaScript final. **Qualquer visitante consegue ler.** Por isso:

- ✅ A chave **publishable** pode ficar ali (é pública por desenho; o RLS protege os dados).
- ❌ **Nunca** coloque uma chave secreta (`service_role`, senha de banco) em variável `VITE_`. Ela ficaria visível para o mundo.

Mudou uma variável? É preciso um novo deploy

Como o valor é "queimado" no build, alterar a variável no painel **não** muda o site. É preciso **republicar**.

---

4. O que é o Vercel e como o deploy acontece

Vercel é uma hospedagem especializada em sites front-end. Conceitos:

Conceito	O que é
Projeto	O seu site (`vault-keeper`)
Deployment	Uma **versão publicada** (cada uma tem URL própria, ex.: `vault-keeper-3p3zdyiww-...vercel.app`)
Production	A versão "oficial", ligada ao endereço principal
Preview	Versões de teste (por branch ou Pull Request)
Alias	O endereço amigável que aponta para o deployment atual de produção (`vault-keeper-zeta.vercel.app`)

O que o Vercel faz a cada deploy

```
1. Baixa o código do GitHub (aquele commit)
2. Instala as dependências (npm install)
3. Roda o build (npm run build)
4. Publica a pasta dist/ numa rede mundial de servidores (CDN)
5. Move o alias para a versão nova
```

Dura cerca de 10 a 15 segundos neste projeto.

Deploy automático: o que acontece no seu `git push`

Com o repositório ligado ao Vercel, **cada `push` na `main` dispara um deploy de produção** sozinho. No painel (aba *Deployments*) você vê cada um com a mensagem do commit.

Na sua lista de deploys, repare nos ícones:

- `-o-` (commit): deploy disparado por um **push** no GitHub.
- `>_` (terminal): deploy feito **pela linha de comando** (`vercel deploy`).

Também dá para publicar sem git (como fizemos na primeira vez)

```
npx vercel@latest login              # entra na conta (você aprova no navegador)
npx vercel@latest link               # liga esta pasta a um projeto
npx vercel@latest env add NOME production   # adiciona uma variável
npx vercel@latest deploy --prod      # publica da pasta local
npx vercel@latest ls                 # lista os deploys
```

O fluxo recomendado é pelo git: fica **registrado**, é reproduzível e dispensa comandos extras.

---

5. A regra de reescrita para as rotas (`vercel.json`)

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

Sem isso, abrir `/catalog` direto daria 404 (capítulo 06). Se o site publicado quebrar só ao **recarregar** páginas internas, é esta regra que falta.

---

6. Configurar o Supabase para o endereço publicado

Depois de publicar, o Supabase precisa **conhecer** o endereço novo:

Onde	O que colocar	Para quê
Authentication → URL Configuration → Site URL	`https://vault-keeper-zeta.vercel.app`	Destino padrão dos links de e-mail
Redirect URLs	O mesmo endereço (com `/**` no final para aceitar qualquer página)	A lista de destinos **permitidos** (segurança contra *open redirect*)

Sem isso, o link de confirmação de e-mail mandaria o usuário para `localhost`, que só existe **no seu computador**.

---

7. Cache: por que "não atualizou"?

Navegadores guardam cópias de arquivos para carregar mais rápido. Depois de publicar, às vezes você ainda vê a versão antiga.

O que parece não atualizar	Por quê	O que fazer
A página inteira	Cache do navegador	`⌘⇧R` (recarregar ignorando o cache)
O ícone da aba (favicon)	Navegadores (principalmente o Safari) o guardam por **muito** tempo	Feche e reabra a aba; ou esvazie o cache do Safari
Dados do banco	Raramente cache; costuma ser outra coisa	Veja o capítulo 08 (RLS)

Os arquivos do `assets/` usam **hash no nome**, então não sofrem desse problema. Já `favicon.svg` e `index.html` têm nomes fixos e podem ficar "presos" no cache.

---

8. Investigando quando algo dá errado em produção

Um roteiro, do mais barato ao mais caro:

1. **Funciona no `npm run preview`?** Se não, o problema está no **build**, não na hospedagem.
2. **Console do navegador** (`⌥⌘J`, aba Console): erros de JavaScript aparecem ali.
3. **Aba Network (Rede):** o pedido ao Supabase falhou? Qual o código (401, 403, 404)?
4. **Painel do Vercel → Deployments → o deploy → Build Logs:** erros de compilação.
5. **Variáveis de ambiente:** estão no ambiente certo (*Production*)? Houve novo deploy depois de mudá-las?
6. **Supabase → Logs:** o que o banco respondeu?

Sintoma no site publicado	Causa mais provável
Página em **branco**	Variáveis de ambiente ausentes (veja o console)
404 ao recarregar uma rota	Falta o `rewrites` no `vercel.json`
Funciona local, mas **sem dados** no ar	URL/chave erradas, ou RLS bloqueando
Login manda para `localhost`	*Site URL* do Supabase não foi atualizada
Deploy "falhou"	Erro de TypeScript no `tsc -b` (leia o log)
Versão antiga aparecendo	Cache do navegador

Voltar atrás

O Vercel guarda **todos** os deploys antigos. Se publicar algo quebrado, no painel você pode **promover** um deploy anterior para produção em segundos. (E, pelo git, também dá para `git revert` e dar `push`.)

---

9. Proteção de deployments (um detalhe que confunde)

Quando você consultou a URL longa de um deploy específico (`vault-keeper-qfmkslng0-...vercel.app`), ela devolveu **302** (redirecionamento para login do Vercel). Isso é a **Deployment Protection**: URLs de versões específicas exigem login da sua conta. O **alias** de produção (`vault-keeper-zeta.vercel.app`) é público. Não é erro; é o padrão do Vercel.

---

10. Endereço próprio (domínio)

Para algo como `meucatalogo.com.br`:

1. Comprar o domínio (registro.br, Namecheap etc.), ou usar um que você já tenha.
2. No Vercel: *Settings* → *Domains* → *Add* e seguir as instruções (apontar o DNS).
3. Atualizar **Site URL** e **Redirect URLs** no Supabase com o endereço novo.

Alternativa simples e gratuita: **renomear o projeto** no Vercel muda o `.vercel.app` (por exemplo, para `vault-keeper.vercel.app`, se estiver livre).

---

11. Checklist antes de publicar

- ☐ `npx tsc -b`, `npm test` e `npm run build` passam.
- ☐ `npm run preview` abre e funciona.
- ☐ Nenhum segredo no código ou no histórico do git.
- ☐ Variáveis de ambiente cadastradas no ambiente *Production*.
- ☐ `vercel.json` com a regra de reescrita.
- ☐ URLs do Supabase atualizadas.

- ☐ Cadastro/e-mail de confirmação configurados como você quer (capítulo 13).
- ☐ Você abriu o site publicado e **usou** (não só olhou).

Exercícios

1. Rode `npm run build` e abra a pasta `dist/`. Quantos arquivos? Qual o tamanho do `.js`? Rode `npm run preview` e use o site.
2. Mude uma linha de texto, rode o build de novo e compare os **nomes** dos arquivos em `assets/`. O que mudou?
3. Pelo painel do Vercel, abra um deploy antigo e veja os *Build Logs*. Em que passo cada um demora mais?
4. Crie um projeto de estudo (um "olá mundo" em Vite), publique-o no Vercel pelo GitHub e confirme o deploy automático a cada `push`.
5. **Desafio:** quebre o build de propósito (um erro de tipo), dê `push` numa **branch** (não na `main`) e veja o Vercel recusar o deploy sem afetar o site no ar.

✅ **Checklist do capítulo 12**

- ☐ Explico as diferenças entre desenvolvimento e produção.
- ☐ Sei o que `tsc -b && vite build` faz e o que sai em `dist/`.
- ☐ Explico o hash nos nomes dos arquivos.
- ☐ Explico por que variáveis `VITE_` ficam visíveis e o que **nunca** pôr nelas.
- ☐ Descrevo o que o Vercel faz a cada `push`.
- ☐ Sei por que a regra de reescrita existe e o que o Supabase precisa saber do endereço novo.
- ☐ Sigo o roteiro de investigação quando algo falha só em produção.

Próximo: 13. Segurança.

CAPÍTULO 13 DE 18

Segurança de um app web

Objetivo

Aprender a **pensar como quem ataca** para proteger melhor. Você vai entender o que, neste projeto, protege os dados (e o que parece proteger, mas não protege), reconhecer os riscos que ainda existem e sair com um checklist para qualquer app futuro.

> Segurança não é um recurso que se "liga" no fim. É um modo de pensar que acompanha **cada** decisão.

---

1. A premissa fundamental: o navegador é território hostil

Tudo que roda no navegador do visitante está **nas mãos dele**. Ele pode:

- ler **todo** o seu JavaScript (inclusive as variáveis `VITE_`);
- abrir o console e **chamar suas funções** com argumentos que você nunca previu;
- **ignorar** o seu frontend e conversar direto com a API do banco, com ferramentas como `curl`;
- **alterar** o código em memória (esconder a validação, mostrar botões "escondidos").

Daí a regra que atravessa o projeto:

> **Validação no frontend é conforto para o usuário honesto. Segurança só existe onde o atacante não alcança: no servidor.**

Parece segurança, mas não é	A segurança real
Esconder o botão "Deletar" de quem não é dono	Política RLS de `delete` no banco
Validar tamanho e tipo da imagem no formulário	Limites do bucket no servidor
Redirecionar para o login quem não tem sessão	RLS recusando leituras sem token
O `AgeGate` de 18+	**Nada**: é um aviso, não uma verificação
Colocar a chave "escondida" num arquivo	Só chaves **públicas por desenho** no frontend

---

2. Camadas de defesa (*defense in depth*)

Nunca dependa de **uma** proteção. O projeto empilha várias:

```
Visitante ──► [ Frontend: conforto + validação ]
          ──► [ Auth: quem é você? (JWT) ]
          ──► [ RLS: o que ESTA linha permite a VOCÊ? ]
          ──► [ View: quais COLUNAS saem? ]
          ──► [ Storage policies: qual PASTA é sua? ]
          ──► [ Configurações: cadastro fechado, e-mail confirmado ]
```

Se uma camada falhar, outra ainda protege. Exemplo: o `.eq('user_id', userId)` no `listMyMagazines` é uma proteção a mais **por cima** do RLS.

---

3. Segredos e chaves

As duas chaves do Supabase (relembrando)

Chave	No navegador?	Poder
`sb_publishable_...` (anon)	✅ Sim	Só o que o RLS autoriza
`service_role`	❌ **Jamais**	**Ignora o RLS**: acesso total

Regras para segredos

1. **Nunca** commite um segredo. O histórico do git é **permanente**: apagar o arquivo depois não remove o segredo dos commits antigos. Se vazou, **troque a chave** imediatamente.
2. Segredos vivem em **variáveis de ambiente**, fora do código (`.env.local`, painel da hospedagem).
3. **Prefixo `VITE_` = público.** Tudo que começa com `VITE_` vai para o navegador de todos.
4. O arquivo `.gitignore` do projeto tem `*.local` e `.env*` para impedir acidentes.
5. Em conversas, tickets e prints: **não cole** chaves secretas nem senhas. O que é público por desenho (a chave `publishable`) tem menos risco, mas é um bom hábito tratar tudo com cuidado.

Como verificar se vazou um segredo no histórico

```
git log -p --all | grep -iE "service_role|password *=|secret"
```

Foi o que rodamos antes do primeiro `push`: o resultado vazio mostrou o histórico limpo.

---

4. RLS: a camada que de fato protege os dados

Resumo do capítulo 08, agora sob a ótica de ataque:

Atacante tenta...	O que acontece
Ler revistas **privadas** de outra pessoa	O RLS filtra: recebe `[]`
Apagar revistas **de outra pessoa**	O RLS filtra: `0` linhas afetadas
Inserir uma revista com o `user_id` de **outro** usuário	`with check` falha: erro

Enviar um arquivo para a **pasta de outro** usuário	A política do Storage recusa
Ler a tabela direto (sem login)	Sem política para `anon`: `[]`

O que o RLS não faz

- **Não filtra colunas** (só linhas). Para isso usamos a **view**.
- **Não valida o conteúdo** (um dono pode gravar texto estranho nas próprias linhas). Para isso há `check` e restrições no esquema.
- **Não impede abuso de cota** (alguém com conta pode subir muitos arquivos).

Teste que você mesmo pode fazer

No console do navegador, **sem login**:

```
await fetch('SUA_URL/rest/v1/magazines?select=*', { headers: { apikey: 'SUA_CHAVE_PUBLICA' } }).then(r => r.json())
```

Se o RLS estiver certo, o resultado é `[]` (era o exercício que fizemos). Se vierem dados privados, **há uma falha de segurança grave**. **Teste seu RLS como atacante, não só como usuário.**

---

5. O caso da view: vazamento de colunas

A política antiga deixava qualquer visitante ler as revistas públicas, mas a API devolvia a **linha inteira**, incluindo `user_id` e `notes`. O `user_id` é o identificador interno da sua conta: não é um segredo crítico, mas é **informação que o público não precisa ter**.

O princípio por trás: **mínimo privilégio.** Cada papel deve ver **o mínimo necessário**. Corrigimos com a view `public_magazines` e removemos o acesso direto à tabela.

Perguntas que valem fazer em qualquer endpoint público: *"quais campos saem aqui? precisamos de todos?"*

---

6. Autenticação e contas

O que configuramos

Ajuste	Para quê
Cadastro **fechado** (*Allow new users to sign up* desligado)	Ninguém além do dono cria conta; some uma enorme superfície de ataque
Confirmação de e-mail ligada	Quem se cadastrasse precisaria provar o endereço
Redirect URLs como lista de permitidos	Evita *open redirect*

O que ainda é uma lacuna

- **Proteção contra senhas vazadas** está desligada no Supabase (o painel de segurança avisa).

Ela rejeita senhas que já apareceram em vazamentos conhecidos.
- Não há **autenticação em dois fatores (2FA)** para a conta do dono.
- O token de sessão fica no `localStorage` do navegador (padrão do `supabase-js`). Isso significa que, se algum dia houver uma falha de **XSS** (seção 8), o token poderia ser roubado.

Boas práticas de senha (para você e para qualquer app)

- Longa, única, de um gerenciador de senhas.
- **Nunca** reutilize a senha de outro serviço.
- Ligue o 2FA nas contas importantes (GitHub, Supabase, Vercel): quem controla essas contas controla o seu projeto.

---

7. Validação: no cliente e no servidor

Onde	Para quê	Pode ser burlada?
Cliente (formulário, `accept="image/*"`, limite de 5 MB)	Resposta imediata ao usuário honesto	✅ **Sim, trivialmente**
Servidor (`check` no banco, limites do bucket, RLS)	Barrar qualquer um	❌ Não

A regra: **valide nos dois**. O do cliente é por **usabilidade**; o do servidor é por **segurança**.

A lacuna conhecida do projeto

O bucket de capas **não tem** limite de tamanho nem de tipo. A validação de 5 MB e `image/*` só existe no navegador. A trava de servidor (capítulo 08) seria:

```
update storage.buckets
set file_size_limit = 5242880,
    allowed_mime_types = array['image/jpeg', 'image/png', 'image/webp', 'image/avif']
where id = 'magazine-covers';
```

Hoje o risco é baixo porque o **cadastro está fechado**: só o dono envia arquivos. Mas é exatamente o tipo de decisão ("aceito o risco porque a camada de cima o reduz") que deve ser **consciente e anotada**.

Validação dos dados que vêm de fora (até do navegador!)

`useViewMode` só aceita um valor conhecido do `localStorage`:

```
return MODES.find((mode) => mode === saved) ?? 'vertical'
```

Nunca confie em dados que você não controla, nem os que estão no próprio navegador, que o usuário pode editar.

---

8. XSS: scripts injetados

XSS (*cross-site scripting*) é quando um atacante consegue fazer **o navegador de outra pessoa executar um script dele** dentro do seu site. Ex.: alguém cadastra uma revista chamada `<img src=x onerror=roubaToken()>` e, se o site inserisse esse texto como HTML, o código rodaria no navegador de quem visse.

Como o React protege

```
<h2>{magazine.title}</h2>
```

O React **escapa** o texto: `<script>` vira o texto literal `<script>`, mostrado na tela, **sem executar**. Você está protegido **por padrão**.

Como você poderia desfazer essa proteção sem querer

```
<div dangerouslySetInnerHTML={{ __html: magazine.title }} />    // ⚠️⚠️ NUNCA com dados de usuário
```

O nome é um aviso. Se precisar, **sanitize** (com uma biblioteca como DOMPurify). Outras armadilhas: construir `href` / `src` a partir de dados do usuário sem validar (`javascript:...`), e usar `eval`.

---

9. *Open redirect* e a lista de permitidos

Se o Supabase aceitasse **qualquer** endereço de retorno após o login, um atacante poderia criar um link tipo `seu-login?redirect=https://site-falso.com`, e a vítima, depois de logar de verdade, cairia num site falso idêntico ao seu, que roubaria dados. A **lista de Redirect URLs** (só os seus endereços) impede isso.

---

10. Operações destrutivas: o cuidado que evita desastres

Durante o projeto, vários **dados reais** foram alterados. As práticas que seguimos (e que você deve copiar):

1. **Olhe antes de mexer.** Rode um `select` com o mesmo `where` e **conte** as linhas antes do `delete` / `update`.
2. **Confira os números.** O comando prometia `DELETE 64`; o esperado era 64. Se viesse outro número, **parar**.
3. **Verifique depois.** A mensagem "Success. No rows returned" do editor **não** diz quantas linhas foram apagadas. Quem diz é uma nova consulta.
4. **Guarde como reverter.** Antes de mudar o dono de 69 revistas, anotamos os donos antigos.
5. **Prefira "adicionar o novo, depois remover o velho"** (a ordem da view).
6. **Backup.** O plano gratuito do Supabase não tem backup que você restaure. Para dados

importantes, **exporte** (Table Editor → Export → CSV) antes de operações irreversíveis.

7. **Quem apaga é o dono.** Operações irreversíveis em dados reais merecem que **uma pessoa**, e não uma automação, aperte o botão, depois de ler o que vai acontecer.

---

11. Dependências (a cadeia de fornecimento)

O projeto depende de centenas de pacotes do npm, escritos por estranhos. Riscos e hábitos:

```
npm audit            # lista vulnerabilidades conhecidas nas dependências
npm outdated         # o que está desatualizado
```

- Mantenha o `package-lock.json` no git (versões travadas).
- Desconfie de pacotes pouco conhecidos e de nomes parecidos com os famosos (*typosquatting*).
- Atualize com regularidade, e rode os testes depois.

---

12. Privacidade e responsabilidade

- **Dados pessoais:** o app guarda o e-mail do dono (no Supabase Auth). Em apps com outros usuários, leis como a **LGPD** (Brasil) exigem cuidado com o que se coleta, por quê e por quanto tempo.
- **Conteúdo e marca:** o catálogo mostra capas de uma revista adulta e usa um logotipo registrado de terceiros. Isso levanta duas questões que **não são técnicas**, mas que quem publica precisa considerar:
  1. **Público:** o aviso de 18+ é só um aviso. Verificação real de idade exige serviços próprios.
  2. **Marca registrada:** usar o nome e o logotipo de uma empresa pode ser lido como site **oficial**. Um rodapé ("acervo pessoal, não oficial") reduz essa confusão, mas **não substitui** orientação jurídica, caso o site cresça ou seja comercial.
- **Imagens de terceiros:** capas de revistas têm direitos autorais. Um acervo pessoal é uma coisa; republicar em larga escala é outra.

---

13. Checklist de segurança do projeto (estado atual)

Item	Estado
RLS ligado nas tabelas	✅
Políticas por dono (select/insert/update/delete)	✅
Leitura pública só pela view, sem `user_id`	✅
Storage: escrita só na própria pasta	✅
Chave `service_role` fora do frontend	✅
Segredos fora do git (`.gitignore`)	✅

Cadastro fechado	✅
Confirmação de e-mail ligada	✅
Redirect URLs apenas dos seus endereços	✅
Limite de tamanho/tipo no bucket (servidor)	⚠️ **Lacuna**
Proteção contra senhas vazadas	⚠️ **Lacuna**
2FA nas contas (GitHub, Supabase, Vercel)	⚠️ A conferir
Chave estrangeira em `user_id`	⚠️ **Lacuna** (causou órfãs)
Migration "de base" versionada	⚠️ **Lacuna**
Backup regular dos dados	⚠️ **Lacuna**
Atualização do Postgres	⚠️ Disponível

> **Nota de transparência:** os itens de **Auth** (cadastro fechado, confirmação de e-mail, Redirect URLs) são configurações do painel do Supabase, feitas por você. Elas **não aparecem no código** nem nas migrations, e não dá para verificá-las daqui. Revise-as de tempos em tempos no painel (*Authentication*). É a mesma lacuna que aparece em "migration de base": **configuração que vive só num painel é configuração que ninguém audita**.

> Um sistema "seguro" não é o que tem zero lacunas, mas o que **conhece as suas lacunas** e decidiu conviver com cada uma de forma consciente.

Erros comuns

Erro	Consequência	Prevenção
Colocar `service_role` numa variável `VITE_`	Acesso total ao banco para qualquer visitante	Só a chave pública no frontend
Confiar só na validação do formulário	Qualquer um burla	Valide também no servidor
Esquecer de ligar o RLS	Todos leem e apagam tudo	RLS ligado **antes** de pôr dados
`dangerouslySetInnerHTML` com dados de usuário	XSS	Deixe o React escapar
Commitar `.env`	Segredo permanente no histórico	`.gitignore` + trocar a chave
`delete` sem `where`	Perda total de dados	`select` antes; backup

Exercícios

1. **Ataque o seu projeto de estudo.** Em um projeto Supabase de teste, crie uma tabela **sem** RLS, consulte-a como `anon` e veja os dados vazarem. Depois ligue o RLS **sem** políticas e veja `[]`.
2. Escreva a política `delete` para uma tabela de `wishlist`. Tente apagar a linha de outro usuário e leia o resultado.

3. Rode `npm audit` no projeto. Quantas vulnerabilidades? Pesquise o que uma delas significa.
4. Aplique no projeto de estudo o `update storage.buckets ...` da seção 7 e tente subir um arquivo `.pdf` e um de 10 MB.
5. Procure no projeto qualquer uso de `dangerouslySetInnerHTML` ou `eval`. (Não deve haver nenhum.)
6. **Desafio:** escreva um **modelo de ameaças** de uma página: liste quem pode atacar, o que querem e qual camada os barra. Compare com a seção 2.

✅ Checklist do capítulo 13

- ☐ Explico por que segurança não pode ficar só no frontend.
- ☐ Diferencio a chave publishable da `service_role` e o que o `VITE_` implica.
- ☐ Traduzo o que cada política RLS impede a um atacante.
- ☐ Explico por que a view existe (mínimo privilégio).
- ☐ Diferencio validação do cliente e do servidor, e dou o exemplo do bucket.
- ☐ Explico XSS e por que o React protege por padrão.
- ☐ Sigo o ritual de operações destrutivas (`select` antes, conferir, verificar depois).
- ☐ Reconheço as lacunas do projeto e proponho como fechá-las.

Próximo: 14. Acessibilidade e qualidade.

CAPÍTULO 14 DE 18

Acessibilidade e qualidade de código

Objetivo

Aprender a construir interfaces que **qualquer pessoa consegue usar** (com teclado, leitor de tela, baixa visão, celular) e a avaliar a qualidade do próprio código. No final do capítulo há uma **lista honesta de melhorias** que o projeto ainda merece.

Parte A: Acessibilidade (a11y)

Por que importa

Acessibilidade não é "um favor para poucos". Quem navega só pelo teclado, usa um leitor de tela, tem baixa visão ou daltonismo, ou está no celular sob o sol, é **muita gente**, e às vezes **você mesmo** (mão ocupada, tela riscada, conexão ruim). Código acessível também é, quase sempre, **melhor para todos**.

> O abreviado **a11y** quer dizer "a" + 11 letras + "y" (*accessibility*).

---

1. HTML semântico: use a tag que descreve o que o elemento é

Em vez de...	Use	Ganho
`<div class="botao" onClick>`	`<button>`	Teclado (Tab/Enter/Espaço) e leitor de tela de graça
`<div class="topo">`	`<header>`	O leitor de tela anuncia "banner"
`<div class="menu">`	`<nav aria-label="Principal">`	Pula direto para a navegação
`<div class="conteudo">`	`<main>`	Ponto de salto "conteúdo principal"
`<div class="card">`	`<article>`	Bloco independente
`<div class="titulo">`	`<h1>` … `<h3>`	Estrutura navegável por títulos
Abrir/fechar com JS	`<details>` / `<summary>`	Funciona sem JS e com teclado

📁 **No projeto:** `App.tsx` usa `<header>`, `<nav aria-label="Principal">`, `<main>`; o `MagazineCard` é um `<article>`; os painéis usam `<details>`.

Regra de ouro: se existe uma tag nativa para o que você quer, **use-a**. Reinventar um botão com `<div>` significa reconstruir à mão o foco, o teclado e a semântica que o `<button>` já traz.

2. Hierarquia de títulos

Os leitores de tela permitem **pular de título em título**, como num índice. Por isso:

- **Uma** `<h1>` por página, descrevendo-a.
- Depois `<h2>`, `<h3>`... **sem pular níveis**.

⚠️ **Achado real:** `PublicCatalog.tsx` hoje tem `<h1></h1>` **vazio** (uma decisão de design). Para um leitor de tela, a página fica **sem título**. A solução que mantém o visual limpo é um título "só para leitores de tela":

```
.sr-only {
  position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px;
  overflow: hidden; clip: rect(0, 0, 0, 0); white-space: nowrap; border: 0;
}
```

```
<h1 className="sr-only">Catálogo público</h1>
```

Invisível na tela, lido em voz alta.

3. Texto alternativo nas imagens (`alt`)

```
<img src={url} alt={`Capa de ${magazine.title}`} loading="lazy" />      // descreve a imagem
<img src={preview} alt="" />                                            // decorativa: alt VAZIO
```

- Imagem **informativa** → `alt` descritivo ("Capa de Joana Prado Feiticeira").
- Imagem **decorativa** → `alt=""` (o leitor a ignora).
- **Nunca** omita o atributo: o leitor leria o nome do arquivo.

📁 O `BulkImport` usa `alt=""` nas miniaturas, porque o título já está ao lado.

4. Formulários: todo campo tem um rótulo

```
<label>
  Título
  <input value={title} onChange={...} />
</label>
```

Envolver o `<input>` no `<label>` liga os dois: clicar no texto foca o campo, e o leitor de tela diz "Título, campo de texto". Um campo **sem** rótulo é um campo mudo para quem não enxerga.

Onde o texto não é visível, use `aria-label`:

```
<input aria-label={`Título de ${item.file.name}`} ... />
```

5. Botões só com ícone precisam de nome

Os botões do seletor de modo (`ViewModeToggle`) são **só desenho**. Sem texto, o leitor diria "botão" e mais nada. Por isso:

```
<button type="button" aria-label="Rolagem vertical" title="Rolagem vertical" aria-pressed={mode === 'vertical'}>
  <svg ... aria-hidden="true">...</svg>
</button>
```

Atributo	Função
`aria-label`	O **nome** lido pelo leitor de tela
`title`	A dica que aparece ao passar o mouse
`aria-pressed`	Estado de botão "liga/desliga" (qual modo está ativo)
`aria-hidden="true"` no `<svg>`	O leitor ignora o desenho (o nome já está no botão)

6. Janelas (modais)

```
<div className="dialog" role="dialog" aria-modal="true">
```

- `role="dialog"` avisa que é uma janela; `aria-modal="true"` diz que o resto da página está "bloqueado".
- No `AgeGate`, `aria-labelledby="agegate-title"` liga a janela ao seu título.
- O botão principal tem `autoFocus`: o foco do teclado **já começa nele**.
- **Esc fecha** o `Modal` (um comportamento esperado de qualquer janela).

⚠️ **O que ainda falta (nossa lacuna):** o `Modal` **não prende o foco** dentro dele. Quem usa `Tab` consegue "escapar" para a página por trás. E, ao fechar, o foco **não volta** ao botão que abriu a janela. Isso se chama *focus trap* e *focus restore*.

7. Teclado: tudo que o mouse faz, o teclado também

Tecla	Esperado
`Tab` / `Shift+Tab`	Percorre os elementos interativos
`Enter` / `Espaço`	Ativa botões e links
`Esc`	Fecha janelas
Setas	Rola a fileira horizontal (o `gallery__scroller` tem `tabIndex={0}`)

🧪 **Experimente agora:** abra o site, **largue o mouse** e use só o teclado. Consegue ir ao menu, trocar o modo de visualização, abrir e fechar uma janela? Onde você se perdeu?

8. Foco visível

```
:focus-visible { outline: 2px solid var(--text); outline-offset: 2px; }
```

Quem usa o teclado precisa **ver onde está**. `:focus-visible` mostra o contorno **só** na navegação por teclado (não ao clicar com o mouse). **Nunca** escreva `outline: none` sem substituir por outro indicador.

⚠️ **Achado real:** as ações do card (Editar, Tornar pública, Deletar) ficam **invisíveis** até o `:hover`. Para o teclado, isso foi resolvido com `.card:focus-within`, que também as revela. Em telas de toque, `@media (hover: none)` as mostra sempre. **Todo efeito de hover precisa de um equivalente para teclado e toque.**

9. Contraste de cores

O texto precisa de contraste suficiente com o fundo. WCAG (as diretrizes internacionais):

- **4,5:1** para texto normal;
- **3:1** para texto grande e componentes de interface.

📁 Quando você testou `--bg: #77AD78`, o texto `#f2f2f2` ficou em ~2,3:1 e o `--muted` quase sumiu (~1,1:1). **Por isso as variáveis de cor mudam juntas, em pacote.**

Ferramenta: `webaim.org/resources/contrastchecker`. No navegador, o painel de inspeção mostra o contraste ao escolher uma cor.

10. Movimento: `prefers-reduced-motion`

```
@media (prefers-reduced-motion: reduce) {
  * { transition: none !important; }
  .card:hover .card__cover { transform: none; }
}
```

Quem tem enjoo com movimento ou distúrbio vestibular pede, no sistema, "menos animação". O app **respeita**. No JavaScript, o `scrollByPage` também verifica: `behavior: reduceMotion ? 'auto' : 'smooth'`.

11. Idioma e título da página

`<html lang="pt-BR">`: o leitor de tela escolhe a pronúncia certa. `<title>`: o que aparece na aba e é lido primeiro.

12. Tamanho dos alvos de toque

Botões muito pequenos são difíceis de tocar no celular (o recomendado é ~44px). Os botões das ações do card são pequenos: um candidato a melhoria.

Como testar acessibilidade

Ferramenta	O que faz
Só o teclado	O teste mais barato e revelador
VoiceOver (`⌘F5` no Mac)	O leitor de tela do Mac. Navegue pelo site de olhos fechados
Lighthouse (aba no DevTools do Chrome)	Relatório automático de acessibilidade
Extensão axe DevTools	Aponta problemas específicos
Zoom a 200%	O layout continua usável?

As ferramentas automáticas pegam só **uma parte** (~30–40%) dos problemas. O teste manual é insubstituível.

Parte B: Qualidade do código

1. As três "redes" automáticas do projeto

Ferramenta	Pergunta que responde	Comando
TypeScript	Os **tipos** batem?	`npx tsc -b`
oxlint	Há **práticas duvidosas**?	`npm run lint`
Vitest	A **lógica** ainda funciona?	`npm test`

Rode as três antes de cada commit.

2. Os avisos do lint (e o que ensinam)

```
Catalog.tsx:34:7: warning react(set-state-in-effect)
MagazineForm.tsx:35:19: warning react(set-state-in-effect)
```

Significam: "você chama `setState` **diretamente dentro** de um efeito, o que dispara um desenho extra". Funciona, mas há um jeito mais enxuto. **Um aviso não é erro, mas é uma pergunta que merece resposta.**

3. Comentários: explique o porquê, não o quê

```
// ❌ ruim (repete o código)
i = i + 1  // soma 1 a i

// ✅ bom (explica a decisão)
// "?? 9999" manda as revistas sem ano para o fim da lista, nas duas direções.
```

O projeto tem muitos comentários porque é material de estudo. Em código profissional, prefira **nomes bons** e comentários só para o que não é óbvio (o *porquê*).

4. Os princípios que o projeto seguiu

Princípio	Exemplo
DRY (*Don't Repeat Yourself*)	Um `MagazineForm` para criar e editar; um `Modal` para tudo
Responsabilidade única	`filters.ts` filtra; `magazineService.ts` fala com o banco
Funções pequenas e puras	`applyFilters`, `parseCoverFilename`
Nomes que explicam	`confirmDelete`, `listPublicMagazines`, `visibleMagazines`
Falhar alto e claro	O `throw` em `supabase.ts` quando faltam variáveis

Parte C: Lista honesta de melhorias (o "backlog" do projeto)

Uma revisão do código atual encontrou estas oportunidades. Elas formam um excelente roteiro de prática:

🔴 Comportamentos que merecem correção

1. Editar uma revista sem condição grava `'Bom'` sem você pedir. `MagazineForm.tsx`: `useState<Condition>(initial?.condition ?? 'Bom')`. As 64 revistas importadas têm `condition: null`. Ao abrir uma delas para editar o **título** e salvar, o formulário envia `condition: 'Bom'`, e o banco passa a dizer "Bom" sem que ninguém tenha escolhido. *Correção:* permitir um valor "não informada" no `<select>` e respeitar o `null`.

2. O formulário não edita o número da edição (`issue`) nem as notas (`notes`). Só a importação em massa preenche `issue`. Uma revista cadastrada **pelo formulário** fica sem número, e a detecção de duplicatas do `BulkImport` não a enxerga. *Correção:* acrescentar o campo "Edição" ao formulário.

3. Estado desatualizado possível nos manipuladores. `handleAdd`, `handleUpdate`, `confirmDelete` e `handleTogglePublic` usam `setMagazines([...magazines])`, a variável **capturada** no desenho (capítulo 05). *Correção:* a forma funcional `setMagazines((current) => ...)`.

4. `<h1>` **vazio** em `PublicCatalog.tsx`. *Correção:* título `sr-only` (Parte A, seção 2).

🟡 Acessibilidade

5. O `Modal` não prende nem devolve o foco (Parte A, seção 6).
6. Botões de ação pequenos para toque.
7. Falta um rótulo "Pular para o conteúdo" no topo, útil para teclado.

🟡 Estrutura

8. `Catalog.tsx` faz coisa demais (carrega, adiciona, edita, apaga, alterna, filtra, desenha). *Correção:* extrair um hook `useMagazines(userId)` com todas as operações, deixando a página só com a parte visual.
9. Os avisos do lint (`set-state-in-effect`).
10. Duplicação sutil de formato de data: a mesma lógica de "mês/ano" aparece em `MagazineCard` e `BulkImport`. *Correção:* uma função pura `formatMagazineDate` em `lib/`, **com testes** (TDD).

🟡 Dados e banco

11. `user_id` sem chave estrangeira (capítulo 08), a causa das revistas órfãs.
12. Falta a migration "de base" (tabela e políticas).
13. Bucket sem limites no servidor (capítulo 13).

🟢 Experiência

14. Sem paginação ou "carregar mais": com poucas centenas de revistas funciona (as imagens são `loading="lazy"`), mas com milhares ficaria pesado.
15. Sem estados de carregamento nos cards (um bloco cinza pulsando, *skeleton*).
16. Sem *error boundary*: um erro de código derruba a tela inteira; um componente de contenção mostraria uma mensagem amigável.
17. Sem testes de componentes (capítulo 10).
18. O título da aba é fixo (`PLAYBOY`) em todas as páginas.

Erros comuns de acessibilidade

Erro	Consequência	Correção
`<div onClick>` no lugar de `<button>`	Sem teclado, sem leitor	Use `<button>`
Imagem sem `alt`	Leitor lê o nome do arquivo	`alt` descritivo ou `alt=""`
Ícone-botão sem `aria-label`	"Botão" e mais nada	Dê um nome
`outline: none` sem substituto	Teclado "cego"	`:focus-visible`
Cinza claro sobre branco	Ilegível para muitos	Confira o contraste
Efeito só no hover	Teclado e toque ficam sem	`:focus-within` e `(hover: none)`

Exercícios

1. Faça o **teste só com teclado** no site e anote cada ponto em que travou.
2. Ative o **VoiceOver** (`⌘F5`) e navegue pelo catálogo. O que foi lido de forma confusa?
3. Corrija o **achado 1** (condição `null`), com um **teste** que prove o comportamento.
4. Corrija o **achado 4**, com a classe `.sr-only`.
5. Implemente o **achado 8**: crie `useMagazines(userId)` e deixe `Catalog.tsx` bem menor.
6. Implemente o *focus trap* no `Modal` (pesquise "focus trap react" e escreva à mão antes de usar uma biblioteca).
7. Rode o **Lighthouse** em `/public` e leia o relatório de acessibilidade. Resolva o item de maior impacto.
8. **Desafio:** adicione o campo **"Edição"** ao formulário (achado 2), atualizando tipo, formulário, serviço e testes. Anote cada arquivo que precisou mudar.

✅ Checklist do capítulo 14

- ☐ Escolho a tag semântica certa em vez de `<div>`.
- ☐ Dou `alt`, rótulo e nome acessível a imagens, campos e botões-ícone.
- ☐ Navego o site só com o teclado e sei corrigir o que trava.
- ☐ Verifico contraste e respeito `prefers-reduced-motion`.
- ☐ Explico por que todo efeito de hover precisa de equivalente para teclado e toque.
- ☐ Rodo `tsc`, lint e testes antes de commitar.
- ☐ Reconheço pelo menos cinco itens do backlog e sei como corrigir cada um.

Próximo: 15. Reconstruindo do zero.

CAPÍTULO 15 DE 18

Reconstruindo do zero: o roteiro passo a passo

Objetivo

Refazer o app **você mesmo**, numa pasta nova, em 15 etapas pequenas. Cada etapa tem uma **entrega**, um **teste de aceite** (como saber que acabou) e **dicas para quando travar**. Este é o capítulo que transforma "entendi quando vi" em "sei fazer sozinho".

Regras do jogo

1. **Crie um projeto novo.** Nome sugerido: `vault-keeper-estudo`, em `~/dev/`. **Não mexa** no original.
2. **Um projeto Supabase de estudo**, separado do real (você criará na etapa 6). Nada de testar em dados reais.
3. **Digite tudo à mão.** Os blocos aqui são **esqueletos e dicas**, não o código final. A graça é você completar.
4. **Só olhe o projeto original depois de tentar** por pelo menos 15 minutos. Quando olhar, entenda *por que* é assim, e **escreva de novo com suas palavras**, sem copiar.
5. **Um commit por etapa** (ou mais). Mensagem que explica o porquê.
6. **Antes de cada commit:** `npx tsc -b`, `npm test` (quando houver) e o app abrindo no navegador.
7. Se uma etapa travar por mais de uma hora, **volte ao capítulo** correspondente, releia e tente de novo. Travar é parte do processo, não sinal de que você não serve.

> **Ordem importa.** Cada etapa se apoia na anterior. Não pule.

Mapa das etapas

#	Etapa	Capítulos de apoio	Tempo
0	Projeto novo e ambiente	01	30 min
1	Tipos e lista estática	03, 04	1 h
2	Estado: adicionar e apagar em memória	05	2 h
3	Visual com variáveis e tema	07	3 h
4	Lógica pura com TDD: filtros	10	3 h
5	Rotas	06	1 h
6	Supabase de estudo: tabela e RLS	08	2 h

7	Login e sessão	05, 08	2 h
8	CRUD ligado ao banco	09	3 h
9	Upload de capas	08, 09	3 h
10	Edição reaproveitando o formulário	04, 05	2 h
11	Catálogo público e modos de visualização	07, 08	4 h
12	Importação em massa com TDD	10	3 h
13	Deploy	12	2 h
14	Revisão: segurança, acessibilidade e backlog	13, 14	4 h

Etapa 0: projeto novo e ambiente

```
cd ~/dev
npm create vite@latest vault-keeper-estudo -- --template react-ts
cd vault-keeper-estudo
npm install
npm run dev
git init -b main
git add -A
git commit -m "Projeto inicial (Vite + React + TypeScript)"
```

Entrega: a página de exemplo do Vite abre em `localhost:5173`.
Aceite: `git log --oneline` mostra 1 commit; `git status` está limpo.
Se travar: `ENOENT package.json` → você não está dentro da pasta (capítulo 01, seção 1).

Limpeza inicial: apague o conteúdo de `src/App.tsx` e `src/App.css` e deixe só um `<h1>` para começar do zero.

Etapa 1: tipos e lista estática

Faça:

1. `src/types.ts`: defina `Condition` (união literal) e a interface `Magazine`. Comece **enxuto**: `id`, `title`, `year`, `acquired`. Você acrescenta campos depois.
2. `src/components/MagazineCard.tsx`: recebe `magazine` por props e mostra título e ano.
3. Em `App.tsx`, crie um array de 4 revistas **fixas** e desenhe com `.map()` e `key`.

Esqueleto:

```
interface MagazineCardProps { magazine: Magazine }
export function MagazineCard({ magazine }: MagazineCardProps) {
  return ( /* article, h2, p */ )
}
```

Entrega: 4 cards na tela.
Aceite: se você trocar o ano de uma revista no array, a tela muda. Se escrever `condition: 'Otimo'`, o IntelliJ sublinha.
Se travar: `Each child should have a key` → capítulo 04, seção 5.

Etapa 2: estado: adicionar e apagar em memória

Faça:

1. Mova o array para um `useState<Magazine[]>`.
2. Crie `MagazineForm` com **um estado por campo** (título, ano, "já tenho"). Ao enviar, chame uma função `onSubmit` recebida por props. Lembre do `event.preventDefault()`.
3. No `App`, a função `handleAdd` coloca a revista nova **no início** (nova lista, nunca `push`).
4. Botão "Deletar" no card → chama `onDelete(id)` → o `App` filtra com `.filter`.
5. Antes de apagar, abra um `ConfirmDialog`. Use um estado `pendingDeleteId: string | null`. Escreva o `Modal` com `children`, fechando ao clicar no fundo (`stopPropagation` na caixa!) e com **Esc** (efeito com limpeza).

Entrega: adicionar e apagar funcionam; apagar pede confirmação.
Aceite: recarregar a página volta ao estado inicial (esperado: ainda não há banco).
Se travar: a lista não atualiza → você alterou o array direto (capítulo 05, seção 2).

Etapa 3: visual com variáveis e tema

Faça:

1. `src/index.css`: declare as variáveis em `:root` (`--bg`, `--surface`, `--text`, `--muted`, `--border`, `--accent`, `--radius`, `--gutter`), o reset (`box-sizing`, margens) e `:focus-visible`.
2. Crie `[data-theme='light']` com os mesmos nomes e valores claros.
3. Hook `useTheme` (estado + `localStorage` com `try/catch` + `document.documentElement.dataset.theme`).
4. Botão de tema no cabeçalho.
5. Em `App.css`: cabeçalho fixo (`position: sticky`), grade com `repeat(4, minmax(0, 1fr))`, card com `aspect-ratio: 3 / 4` e `object-fit: cover`, `@media (max-width: 820px)` para 2 colunas.
6. Antes de terminar, **inspecione** no navegador e teste a largura de celular.

Entrega: visual escuro e claro, responsivo.
Aceite: nenhuma cor literal (`#...`) em `App.css`, só `var(--...)`. O tema persiste ao recarregar.
Se travar: a troca de tema não pega → o atributo no `<html>` está certo? Inspecione.

Etapa 4: lógica pura com TDD: filtros

```
npm install -D vitest
npm pkg set scripts.test="vitest run" scripts.test:watch="vitest"
```

Faça (nesta **ordem**, é o ponto):

1. `src/lib/filters.test.ts`: escreva os testes **antes**, com uma fábrica `makeMagazine(overrides)`. Cubra: sem filtros, busca por título, busca ignorando acento e maiúscula, filtro de status, ordenação por ano e por título, **lista vazia**, e **não alterar a lista original**.
2. Rode `npm test`: **tudo falha** (o módulo não existe). 🔴
3. Escreva `src/lib/filters.ts` (`Filters`, `defaultFilters`, `applyFilters`), o mínimo para passar. 🟢
4. Refatore sem quebrar. 🔵
5. Componente `CatalogFilters` (campo de busca + selects) controlado por props.
6. No `App`: `const visible = applyFilters(magazines, filters)`. **Não** crie estado para a lista filtrada.

Dica para a busca sem acentos: `text.normalize('NFD').replace(/\p{Diacritic}/gu, '').toLowerCase()`.

Entrega: busca, filtro e ordenação funcionando; testes passando.
Aceite: estrague uma linha de `applyFilters` de propósito: algum teste **tem** que falhar. Se nenhum falhar, faltam testes.

Etapa 5: rotas

```
npm install react-router-dom
```

Faça: `BrowserRouter` no `App`; rotas `/` (Home), `/catalog` (Catálogo), `*` (404); menu com `NavLink` (com `end` no `/`); mova o catálogo para `src/pages/Catalog.tsx`. Crie as pastas `pages/`, `components/`, `hooks/`, `lib/` aplicando o "teste das quatro perguntas" (capítulo 06).

Entrega: navegar entre páginas sem recarregar.
Aceite: digitar `/qualquer-coisa` mostra a página 404; o item de menu ativo fica destacado.

Etapa 6: Supabase de estudo: tabela e RLS

1. Crie uma conta/projeto **novo** no Supabase (nome: `vault-keeper-estudo`).
2. No **SQL Editor**, crie a tabela. **Melhore o original** desde já: coloque a chave estrangeira.

```sql
create table public.magazines (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  title text not null,
  year integer,
  -- complete: month, issue, condition (com check), cover_model, acquired, cover_image_url, is_public, timestamps
);
alter table public.magazines enable row level security;
-- escreva as 4 políticas do dono (select, insert, update, delete)
```

1. Crie `.env.local` (confira que está no `.gitignore`!) e `src/lib/supabase.ts` com o `throw` de segurança.

Entrega: tabela com RLS e políticas; cliente criado.
Aceite (o teste de ataque): no console do navegador, **sem login**, consulte a tabela com `fetch`. Tem que voltar `[]`. Depois, comente: *"por que `[]` e não um erro?"*
Se travar: variável `undefined` → falta o `VITE_` ou reiniciar o `npm run dev` (capítulo 01, seção 5).

Etapa 7: login e sessão

Faça:

1. No painel: crie **um** usuário (Authentication → Users) com e-mail e senha **de teste**.
2. `useSession`: `getSession()` uma vez + `onAuthStateChange` + `unsubscribe` na limpeza + `loading`.
3. `AuthForm` só com login (`signInWithPassword`), mostrando o `error.message`.
4. Em `Catalog`: `loading` → "Carregando..."; sem sessão → `AuthForm`; com sessão → catálogo. **Os `return` antecipados depois dos hooks.**
5. Botão **Sair** (`supabase.auth.signOut()`).

Entrega: login e logout funcionando; recarregar mantém a sessão.
Aceite: o `AuthForm` **não** chama nada no `Catalog` ao dar certo (a tela troca sozinha pelo `useSession`).

Etapa 8: CRUD ligado ao banco

Faça:

1. `src/lib/magazineService.ts`: `listMyMagazines(userId)`, `addMagazine`, `updateMagazine`, `deleteMagazine`, `setMagazinePublic`. Todas com `if (error) throw error`.
2. `Catalog`: carregue em um `useEffect` com `cancelled`; `handleAdd`/`confirmDelete` com `try/catch` e a regra "espere o banco antes de mudar a tela". Mostre `error` em vermelho.
3. Use a **forma funcional** do `setMagazines` desde o começo (corrija o ponto fraco do original).

Entrega: os dados sobrevivem a recarregar.
Aceite: crie uma segunda conta de teste: ela **não** vê as revistas da primeira. (É o RLS.) Derrube a internet e tente apagar: o erro aparece **e a revista continua na tela**.

Etapa 9: upload de capas

Faça:

1. Bucket `magazine-covers` público + as políticas de Storage por pasta (capítulo 08). **Melhore o original:** defina `file_size_limit` e `allowed_mime_types` no bucket.
2. `uploadCover(file, userId)` (caminho `userId/uuid.ext`) e `removeCover(url)` (`pathFromUrl`).
3. No formulário: campo de arquivo, **prévia** com `URL.createObjectURL` (e `revokeObjectURL` na limpeza), validação de tipo e tamanho.
4. `handleAdd`: upload → gravação → **compensação** se a gravação falhar. `deleteMagazine`: linha → arquivo.

Entrega: revistas com capa.
Aceite: simule uma falha na gravação (quebre o nome de uma coluna de propósito): a imagem **não** deve sobrar no Storage. Tente subir um `.pdf`: o **servidor** deve recusar, não só o formulário.

Etapa 10: edição reaproveitando o formulário

Faça: `MagazineForm` com `initial?: Magazine` e `onCancel`; `useState(initial?.x ?? valorPadrão)`; `Modal` com o formulário; `key={revista.id}`; `handleUpdate` (sobe a capa nova, atualiza, apaga a antiga).

⚠️ **Evite os bugs do original:** respeite `null` na condição (não force `'Bom'`), envie `null` (não `undefined`) para campos esvaziados, e inclua o campo **Edição**.

Entrega: editar qualquer campo, inclusive trocar a capa.
Aceite: edite só o título de uma revista **sem condição** e confira no banco que a condição **continua** `null`.

Etapa 11: catálogo público e modos de visualização

Faça, nesta ordem (a ordem é parte da lição):

1. Crie a **view** `public_magazines` (só colunas seguras) e dê `grant select ... to anon`.
2. Crie a página `PublicCatalog` lendo **da view**.
3. Publique. **Só depois**, remova a política que dava leitura direta da tabela ao `anon`.
4. Hook `useViewMode` (três modos, com validação do valor salvo) e `ViewModeToggle` com **ícones SVG** (`aria-label`, `title`, `aria-pressed`).
5. `MagazineGallery`: grade vertical, fileira horizontal com `scroll-snap`, setas, roda do mouse convertida e teclado; e o mosaico com o texto sobre a capa (`position: absolute`, degradê).
6. Cuide de `prefers-reduced-motion`, `:focus-within` e `(hover: none)`.

Entrega: os três modos e o catálogo sem login.
Aceite: como visitante, a tabela direta devolve `[]`, a view devolve as públicas, **sem** `user_id`. Confira os nomes das classes CSS (o caso `--mosaic` × `--mosaico`) olhando a tela.

Etapa 12: importação em massa com TDD

Faça:

1. **Testes primeiro** para `parseCoverFilename` (padrão completo, sublinhado simples, mês inválido, sem data, nome fora do padrão, arquivo sem nome, extensão com ponto no nome). 🔴
2. Escreva a função (regex com grupos opcionais, regra do século). 🟢
3. `BulkImport`: escolher vários arquivos; item com `status` (`pendente | enviando | ok | erro | duplicada`); duplicatas por número de edição com `Set`; envio **sequencial** com compensação; progresso; reenvio dos que falharam.

Entrega: subir 10 capas de uma vez.
Aceite: um arquivo com nome fora do padrão e uma edição repetida **não** quebram o lote.

Etapa 13: deploy

1. Crie um repositório **no GitHub** (vazio, privado). `git remote add origin ...` e `git push -u origin main`.
2. Importe no **Vercel**; cadastre as duas variáveis (`VITE_...`) em *Production*.
3. `vercel.json` com a regra de reescrita.
4. No Supabase: **Site URL** e **Redirect URLs**.
5. Confira cada rota **recarregando** a página.

Entrega: site no ar.
Aceite: um `git push` publica sozinho; abrir `/catalog` direto funciona; o `console` não tem erros.

Etapa 14: revisão final

1. **Segurança:** refaça o checklist do capítulo 13. Feche o cadastro, ligue a confirmação, tente atacar o seu RLS.
2. **Acessibilidade:** teste só com teclado, com o VoiceOver e com o Lighthouse.
3. **Backlog:** escolha **três** itens da Parte C do capítulo 14 e implemente-os, com testes onde couber.
4. Escreva um `README.md` explicando o projeto, como rodar e como foi feito o deploy.

Entrega: um app que você construiu, entende, defende e sabe melhorar.

Como saber que aprendeu de verdade

Sem consultar nada, consiga:

- ☐ Explicar, desenhando, a viagem de um clique em "Adicionar revista" até o banco e de volta.
- ☐ Criar uma tabela nova com RLS e testá-la **como atacante**.
- ☐ Escrever, com TDD, uma função pura com casos de borda.
- ☐ Criar um hook próprio e explicar as regras dos hooks.
- ☐ Diagnosticar por que um site publicado está em branco.
- ☐ Explicar o que o git faz em cada uma das três áreas.
- ☐ Listar três lacunas de segurança de um projeto e como fechá-las.

Se conseguiu os sete, você deixou de ser iniciante.

Próximo: 16. Exercícios e desafios.

CAPÍTULO 16 DE 18

Exercícios e desafios

Como usar este capítulo

Aqui estão **quatro tipos de prática**, em ordem de dificuldade:

1. **Perguntas de revisão** (com gabarito): teste rápido da teoria.
2. **Depuração**: código quebrado para você consertar.
3. **Leitura de código**: explicar o que um trecho faz sem rodá-lo.
4. **Mini-projetos**: funcionalidades novas, com critérios de aceite.

> **Regra:** responda **por escrito**, com suas palavras, **antes** de olhar o gabarito. Respostas na cabeça parecem certas; no papel, mostram o que você realmente sabe.

Parte 1: Perguntas de revisão (com gabarito)

Cubra a coluna da direita e responda antes.

Ferramentas e JavaScript

#	Pergunta	Gabarito
1	Qual a diferença entre `dependencies` e `devDependencies`?	As primeiras o app precisa **no navegador** (React, Supabase); as outras só ajudam **ao desenvolver** (TypeScript, Vite, Vitest)
2	Por que o prefixo `VITE_` existe?	O Vite só envia ao navegador variáveis com esse prefixo; evita vazar as outras por acidente
3	Diferença entre `??` e `||`?	`??` só troca `null`/`undefined`; `||` troca **qualquer** falsy (inclusive `0` e `''`)
4	Por que `sort()` pede cuidado?	**Altera** o array original; copie antes ou ordene o resultado de um `filter`
5	O que `.filter(Boolean)` faz?	Remove itens falsy (`null`, `undefined`, `''`, `0`)
6	`await` funciona onde?	Só dentro de funções `async`
7	Por que o `localStorage` fica dentro de `try/catch`?	Pode **lançar erro** (navegação anônima, cookies bloqueados)

TypeScript e React

#	Pergunta	Gabarito
8	O que `year: number | null` significa?	O campo existe, mas pode ser `null` ("não sei o ano")
9	O que `Omit<Magazine, 'id'>`	Uma `Magazine` **sem** o campo `id`

	produz?	
10	Por que os tipos "mentiram" sobre `year`?	O tipo dizia `number`, mas o banco tinha `null`; o TS só confia no que você escreveu
11	Por que `key` em uma lista?	Para o React identificar cada item entre desenhos; deve ser única e estável (use o `id`)
12	O que é "dados descem, eventos sobem"?	O pai passa dados e funções por props; o filho avisa o pai chamando a função
13	Por que não `magazines.push(x)`?	Altera o array por dentro; a referência é a mesma e o React não percebe a mudança
14	O que é estado derivado? Exemplo?	Valor **calculado** a partir de outros, sem `useState`: `visibleMagazines = applyFilters(...)`
15	Quando usar a forma funcional `setX(prev => ...)`?	Quando o novo valor depende do anterior; evita usar um valor "velho" capturado
16	Para que serve o `return` dentro do `useEffect`?	Limpeza: desliga o que o efeito ligou (ouvintes, assinaturas, endereços temporários)
17	O que a variável `cancelled` evita?	Que uma resposta **atrasada** sobrescreva dados mais novos
18	Estado × ref?	Estado: a tela mostra e mudar redesenha. Ref: só lembra; mudar **não** redesenha
19	Regra dos hooks?	Só no topo do componente (nunca em `if`/loop) e só em componentes/hooks
20	Para que serve trocar a `key` de um componente?	O React descarta o antigo e cria um novo, com o estado zerado

CSS, rotas e organização

#	Pergunta	Gabarito
21	O que `box-sizing: border-box` muda?	A `width` já inclui padding e borda
22	Como o tema claro/escuro funciona só com variáveis?	O padrão está em `:root`; `[data-theme='light']` redefine as mesmas variáveis; o hook troca o atributo no `<html>`
23	Flex ou grid?	Flex: uma linha/coluna. Grid: uma grade (duas dimensões)
24	O que `clamp(a, b, c)` faz?	Valor que cresce com a janela (`b`), nunca abaixo de `a` nem acima de `c`
25	Por que `end` no `NavLink to="/"`?	Sem ele o `/` combinaria com todos os endereços e ficaria sempre ativo
26	Por que o `vercel.json` tem `rewrites`?	Sem ele, abrir `/catalog` direto dá 404: o servidor procura um arquivo com esse nome
27	Onde fica um arquivo de lógica sem React?	`src/lib/`

Banco, segurança e deploy

#	Pergunta	Gabarito
28	O que é RLS?	O banco **filtra linhas** conforme políticas; sem política que permita, nada passa
29	`using` × `with check`?	`using`: quais linhas existentes a operação enxerga. `with check`:

		o que uma linha **nova/alterada** deve satisfazer
30	Por que uma consulta pode voltar `[]` sem erro?	O RLS **esconde** as linhas proibidas em silêncio
31	Diferença entre a chave publishable e a `service_role`?	A primeira é pública e só abre o que o RLS deixar; a segunda **ignora o RLS** e jamais vai ao navegador
32	Por que existe a view `public_magazines`?	O RLS filtra **linhas**, não colunas; a view entrega só as colunas seguras (sem `user_id` / `notes`)
33	Por que a pasta no Storage é o `user_id`?	As políticas do Storage conferem a pasta; e UUIDs impedem sobrescrever
34	O que é uma migration?	Um arquivo SQL ordenado que descreve **uma mudança** do banco, versionada no git
35	Por que validar no cliente **e** no servidor?	O do cliente é conforto (burlável); só o do servidor é segurança
36	Por que a ordem do `deleteMagazine` é "linha, depois arquivo"?	O pior caso passa a ser um arquivo órfão (lixo invisível), e não uma revista sem capa
37	O que é o padrão de compensação?	Se o 2º passo falha, **desfaz o 1º** (apagar a imagem que já subiu)
38	Por que `git push` sozinho não envia edições?	O push só envia **commits**; edições não commitadas ficam só na pasta de trabalho
39	O que significa `[ahead 1]` no `git status`?	Você tem 1 commit local que não foi enviado ao GitHub
40	Por que variáveis `VITE_` ficam visíveis a qualquer visitante?	O valor é **gravado dentro do JavaScript** durante o build

Pontuação: 36 ou mais → pronto para a etapa seguinte. 28 a 35 → releia os capítulos das perguntas que errou. Menos de 28 → releia o material com calma, fazendo os exercícios de cada capítulo.

Parte 2: Depuração (o código está quebrado, conserte)

Para cada trecho: **o que está errado? Como corrigir?** Gabarito no final da seção.

Desafio D1

```
const [items, setItems] = useState<string[]>([])

function adicionar(texto: string) {
  items.push(texto)
  setItems(items)
}
```

Sintoma: clica em "Adicionar" e a lista na tela não muda.

Desafio D2

```tsx
function Lista({ revistas }: { revistas: Magazine[] }) {
  return (
    <ul>
      {revistas.map((r, i) => <li key={i}>{r.title}</li>)}
    </ul>
  )
}
```

Sintoma: ao apagar uma revista do meio, os itens de baixo "pulam" estranhamente, e um campo digitado dentro de um `<li>` vai parar no item errado.

Desafio D3

```tsx
<button onClick={apagar(m.id)}>Deletar</button>
```

Sintoma: ao abrir a página, todas as revistas são apagadas sozinhas.

Desafio D4

```tsx
useEffect(() => {
  listMyMagazines(userId).then(setMagazines)
}, [])
```

Sintoma: o usuário sai e entra com outra conta, mas continua vendo as revistas da primeira.

Desafio D5

```tsx
async function salvar() {
  const created = await addMagazine(dados, userId)
  setMagazines([created, ...magazines])
}
```

Sintoma: quando o banco recusa (sem permissão), nada aparece para o usuário e o console mostra `Uncaught (in promise)`.

Desafio D6

```sql
delete from public.magazines
where title like '%Verão%'
   or user_id = 'abc' ;
```

Sintoma: o comando apagou **muito mais** revistas do que o esperado.

Desafio D7

```tsx
if (loading) return <p>Carregando...</p>
const [filters, setFilters] = useState(defaultFilters)
```

Sintoma: erro `Rendered more hooks than during the previous render`.

Desafio D8

```
const { data } = await supabase.from('magazines').select('*')
setMagazines(data)
```

Sintoma: às vezes o app quebra com `Cannot read properties of null`.

Desafio D9

```
.gallery--mosaic { display: grid; ... }
```

```
<div className={`gallery gallery--${mode}`}>   {/* mode vale 'mosaico' */}
```

Sintoma: ao escolher o mosaico, aparece uma única capa gigante.

Desafio D10

```
cover_model: coverModel.trim() || undefined
```

Sintoma: o usuário apaga o campo "Modelo da capa" e salva, mas ao reabrir o valor antigo voltou.

Gabarito da Parte 2

#	Problema	Correção
D1	`push` altera o array original; `setItems(items)` passa a **mesma** referência, e o React acha que nada mudou	`setItems((atual) => [...atual, texto])`
D2	`key={i}` (índice): o React confunde os itens quando a lista muda de ordem ou perde itens	`key={r.id}`
D3	`apagar(m.id)` **executa** a função ao desenhar, em vez de esperar o clique	`onClick={() => apagar(m.id)}`
D4	O array de dependências é `[]`, então o efeito roda só uma vez; faltou `userId` nas dependências (e a limpeza)	`}, [userId])` com a variável `cancelled`
D5	Sem `try/catch`: o `throw` do serviço vira uma Promise rejeitada sem tratamento	`try { ... } catch (e) { setError((e as Error).message) }`
D6	O `or` apaga as revistas que casam com **qualquer** das duas condições, e `like '%Verão%'` pega nomes de todos os donos	Use `and`; e **rode antes um** `select count(*)` com o mesmo `where`
D7	Um hook (`useState`) depois de um `return` antecipado: nos desenhos com `loading`, o hook não é chamado	Mova todos os hooks para **antes** dos `return`
D8	Ignorou `error`: se a consulta falha, `data` é `null`	`if (error) throw error` antes de usar `data`
D9	A classe do CSS (`--mosaic`) não casa com a gerada (`--mosaico`); o TS não vê erro em texto de classe	Alinhe os nomes; **confira olhando a tela**

D10	`undefined` some do envio e o banco mantém o valor antigo	`|| null`

Parte 3: Leitura de código

Sem rodar, explique **em português**, linha a linha, o que cada trecho faz e **por quê**.

L1. `src/lib/filters.ts`: a função `normalize` e a linha do `.includes(term)`.
L2. `src/hooks/useSession.ts`: os três passos do efeito e o papel do `loading`.
L3. `src/components/MagazineGallery.tsx`: o ouvinte do `wheel`. Quando ele **não** bloqueia a rolagem da página? Por quê?
L4. `src/components/BulkImport.tsx`: a função `handleFiles`. Em que situação duas capas do **mesmo lote** são marcadas como duplicadas?
L5. `src/lib/parseCoverFilename.ts`: a regex `PATTERN`. Quais partes são opcionais? O que `_{1,2}` aceita?
L6. `src/pages/Catalog.tsx`: `handleUpdate`. Liste, em ordem, tudo o que acontece quando o usuário troca a capa de uma revista, e o que acontece em cada ponto de falha.
L7. `supabase/migrations/...remove_anon_read...sql`: por que esta migration veio **depois** da view e do `push`?

Como conferir: explique em voz alta, como se ensinasse a um colega. Onde você hesitou ou disse "não sei" é onde precisa reler o capítulo.

Parte 4: Mini-projetos

Cada um é uma funcionalidade nova **no seu projeto de estudo**, com critérios de aceite. Estão em ordem de dificuldade.

⭐ Nível 1 (1 a 3 horas)

M1. Campo "Edição" e "Notas" no formulário. *(Corrige dois achados do capítulo 14.)*

- Aceite: criar e editar uma revista preenche `issue` e `notes`; o `BulkImport` agora detecta como duplicada uma edição criada pelo formulário.

M2. Condição "não informada".

- Aceite: editar o título de uma revista com `condition: null` **não** muda a condição no banco.

M3. Contadores no topo.

- Aceite: "Total", "Na coleção", "Procurando" e "Públicas", calculados (estado derivado), com um componente `StatCard` reutilizável.

M4. Realce do termo buscado.

- Aceite: ao buscar "joana", o trecho aparece em destaque no título do card. *(Dica: função pura + teste primeiro.)*

⭐⭐ Nível 2 (3 a 8 horas)

M5. Desfazer ao apagar.

- Aceite: ao apagar, aparece um aviso "Revista apagada. **Desfazer**" por 5 segundos. *(Pense: apagar de verdade só depois do prazo? Ou apagar já e recriar? Qual é mais seguro?)*

M6. Seleção múltipla.

- Aceite: marcar várias revistas e "Tornar públicas", "Tornar privadas" ou "Apagar" em lote, com confirmação. O que acontece se uma falhar no meio?

M7. Exportar para CSV.

- Aceite: um botão baixa a coleção em `.csv` que abre corretamente no Excel, com acentos. *(Pesquise `Blob` e `URL.createObjectURL`. Escreva a função de conversão como **função pura**, com testes.)*

M8. Página de estatísticas.

- Aceite: rota `/estatisticas` com revistas por década, por condição e total por mês. Barras feitas só com CSS.

M9. Paginação ou "carregar mais".

- Aceite: o catálogo carrega 24 revistas por vez (usando `.range(...)` do Supabase). Compare com a abordagem atual.

M10. Tema que segue o sistema.

- Aceite: na primeira visita, o tema respeita `prefers-color-scheme`; depois, o que o usuário escolher.

⭐⭐⭐ Nível 3 (1 a 3 dias)

M11. Testes de componentes. Instale `@testing-library/react` e teste `MagazineCard` (com e sem ações), `ViewModeToggle` e `ConfirmDialog`.

M12. Teste ponta a ponta. Instale o **Playwright** e escreva um roteiro: abrir o site, passar pelo aviso de idade, trocar o modo de visualização, conferir que as capas aparecem.

M13. Miniaturas. Gerar versões reduzidas das capas (no envio, com `canvas`) para o mosaico carregar muito mais rápido. Compare o peso da página antes e depois.

M14. Arrastar e soltar. Soltar vários arquivos sobre o painel de importação (eventos `dragover` / `drop`).

M15. Busca no servidor. Mova a busca para o banco (`.ilike(...)`), com *debounce* (hook `useDebounce`) e tratamento do estado de carregamento. Quando vale mais a pena que filtrar no navegador?

M16. PWA. Faça o app instalável no celular (manifesto + *service worker*). O que muda para o uso offline?

M17. A migration de base. Escreva a migration que recria **todo** o banco (tabela com chave estrangeira, RLS, políticas, view, bucket com limites) e prove: crie um projeto Supabase vazio, aplique, e o app funciona idêntico.

⭐⭐⭐⭐ Para quem quer ir mais longe

M18. Multiusuário real. Abra o cadastro, com perfis (`profiles`), coleções públicas por pessoa (`/u/nome`), e RLS pensada para isso. *Aviso:* este é um problema de segurança sério. Faça o modelo de ameaças antes (capítulo 13).

M19. Escrever a sua própria "camada de dados". Troque o Supabase por uma API simples sua (Node + Express + SQLite) mantendo `magazineService.ts` como única mudança no frontend. Isso prova que a separação em camadas valeu.

M20. Ensine. Escreva um tutorial de **uma** das etapas do capítulo 15 para um amigo iniciante, e refine até ele conseguir seguir sozinho. Ensinar é a melhor forma de dominar.

Como medir seu progresso

Sinal	Você está...
Precisa copiar o código para qualquer coisa	Começando
Entende o código ao ler, mas trava ao escrever do zero	No meio do caminho
Escreve, quebra e conserta sozinho, lendo as mensagens de erro	Ganhando autonomia
Consegue ensinar a outra pessoa e escolher entre abordagens, justificando	Dominando o básico

Hábitos que aceleram o aprendizado

1. **Estude em blocos curtos e frequentes** (1 hora por dia vale mais que 7 horas num domingo).
2. **Leia as mensagens de erro inteiras.** Costumam dizer o problema e a linha.
3. **Reduza o problema:** se algo não funciona, isole o menor trecho que reproduz o erro.
4. **Explique em voz alta** (ao "patinho de borracha") antes de pedir ajuda.
5. **Pesquise com a pergunta certa:** "[tecnologia] [mensagem de erro entre aspas]".
6. **Leia código de outras pessoas.** A documentação oficial (react.dev, supabase.com/docs, MDN) é a melhor fonte.
7. **Comite com frequência.** Isso vira coragem de experimentar.
8. **Anote o que aprendeu** com suas palavras (um diário técnico).

Próximo: 17. Erros e lições do projeto.

CAPÍTULO 17 DE 18

Erros e lições do projeto

Por que este capítulo existe

Programadores experientes **não** erram menos. Eles **descobrem os erros mais rápido** e **sabem como investigar**. Este capítulo reúne os tropeços **reais** que aconteceram ao construir o Vault Keeper, **inclusive os meus** (o assistente também errou várias vezes). Cada caso segue o mesmo roteiro:

> **Sintoma → Causa → Como descobrimos → Lição**

Leia como histórias de detetive. Antes de ler a causa, tente adivinhar.

O método de depuração em 7 passos

Todos os casos abaixo foram resolvidos com variações disto:

1. **Reproduza.** Consiga fazer o erro acontecer de novo, de forma previsível.
2. **Leia a mensagem inteira.** Ela quase sempre diz o problema, e muitas vezes a linha.
3. **Reduza.** Isole o menor trecho que ainda falha.
4. **Formule uma hipótese** ("acho que é X porque...").
5. **Teste a hipótese** com um experimento barato (um `console.log`, um `select`, uma inspeção).
6. **Conserte uma coisa por vez** e confira.
7. **Registre a lição** (e, se couber, escreva um teste que impeça o erro de voltar).

> **Nunca mude cinco coisas ao mesmo tempo.** Se funcionar, você não sabe qual resolveu. Se piorar, não sabe qual estragou.

Grupo A: ambiente e ferramentas

A1. `npm error code ENOENT ... package.json` (duas vezes)

- **Sintoma:** `npm run dev` e `cd vault-keeper && npm install` falharam com "no such file or directory".
- **Causa:** o terminal estava em `~` e depois em `~/dev`, mas o projeto está em `~/dev/vault-keeper`.
- **Como descobrimos:** a **mensagem dizia o caminho**: `open '/Users/unknown1/dev/package.json'`. O npm procurou o arquivo na pasta errada. O prompt também mostrava `dev %`, a pasta atual.

- **Lição:** o terminal sempre **está em algum lugar**. Antes de rodar um comando do projeto, olhe o prompt. *Leia o caminho no erro.*

A2. `command not found: gh`

- **Sintoma:** o comando `gh repo clone ...` falhou.
- **Causa:** a CLI do GitHub não estava instalada.
- **Solução:** `git clone https://github.com/...` faz o mesmo e **já vem com o git**.
- **Lição:** muitas ferramentas têm um caminho alternativo. Quando o primeiro falha por falta de uma ferramenta, procure o que você **já tem**.

A3. `sed -i` no Mac não fez o que o Linux faz

- **Sintoma:** tentei renomear `coverModel` para `cover_model` com `sed -i 's/.../.../g' arquivo`. O terminal reclamou (`undefined label`), mas o **TypeScript passou** (`TIPOS OK`) e o projeto parecia pronto.
- **Causa:** no macOS, o `sed -i` **exige um argumento a mais** (`sed -i '' ...`). O comando falhou e **a renomeação não aconteceu**. O TypeScript não reclamou porque o cliente do Supabase ainda não conhecia as colunas.
- **Como descobrimos:** uma `grep` depois mostrou que `coverModel` **continuava lá**.
- **Consequência se passasse batido:** o banco recusaria `coverModel` na hora de gravar.
- **Lição: "compilou" não é "funciona".** Depois de uma alteração em massa, **confirme** com uma busca (`grep -rn "nome" src`). E scripts de Linux nem sempre rodam igual no Mac.

A4. Colar a chave no terminal errado

- **Sintoma:** o Vercel esperava um valor em um terminal, mas a chave apareceu em **outro**, rodando o servidor de desenvolvimento.
- **Causa:** eram **dois terminais diferentes** (o do app do Claude e o do IntelliJ). O comando foi aberto em um, a colagem aconteceu no outro.
- **Como descobrimos:** a captura de tela do usuário mostrou o texto no lugar errado, e o comando original continuava aguardando.
- **Lição:** confirme **qual janela está em foco** antes de colar. Em caso de dúvida, olhe o prompt da aba. (A chave era pública, então não houve dano. Com um segredo, teria sido um incidente.)

A5. A animação "travou" no navegador de testes

- **Sintoma:** textos pareciam apagados numa captura de tela do tema claro; uma operação pesada estourou o tempo.
- **Causa:** o painel do navegador estava **oculto**, e navegadores **pausam ou desaceleram** animações e tarefas de abas escondidas. As cores estavam corretas; a captura pegou uma transição pela metade.
- **Como descobrimos:** medimos as **cores calculadas** (`getComputedStyle`) em vez de confiar no olho, e repetimos com a aba em primeiro plano.
- **Lição:** quando a imagem engana, **meça**. E teste o que é visual **com a janela visível**.

Grupo B: código

B1. Os tipos "mentiram": `null/null` na tela

- **Sintoma:** os cards mostravam `null/null` no lugar da data.
- **Causa:** `Magazine` dizia `year: number`, mas as 64 revistas importadas tinham `year = null`. O TypeScript **não consulta o banco**; ele confia no que você escreve.
- **Como descobrimos:** ao olhar o **site real** com dados reais. Na etapa de dados de exemplo, nunca apareceu.
- **Correção:** tipo honesto (`year: number | null`) e tratar o `null` em todo lugar. O TS apontou cada ponto.
- **Lição:** tipos descrevem o que você **acredita**, não o que os dados **são**. Dados que vêm de fora podem divergir. **Teste com dados reais.**

B2. A classe CSS que não casava: `gallery--mosaic` × `gallery--mosaico`

- **Sintoma:** ao escolher o mosaico, apareceu **uma única capa gigante**.
- **Causa:** o código gera a classe a partir do nome do modo (`gallery--mosaico`), mas no CSS eu escrevi `.gallery--mosaic`. Nenhuma regra se aplicou.
- **Como descobrimos: olhando a tela**. O TypeScript e os testes passaram (classes de CSS são só texto).
- **Lição:** alguns erros **só** aparecem olhando. Teste visual e teste automático se complementam. E evite duplicar um nome em dois lugares: aqui, a classe poderia vir de **uma constante**.

B3. Os botões roxos

- **Sintoma:** os botões da página inicial (que são `<a>`) saíram com texto roxo.
- **Causa:** o navegador tem uma regra de cor para links; a nossa `.btn` não definia `color`.
- **Como descobrimos:** o olho, numa captura. A inspeção mostraria a regra do navegador sobrepondo.
- **Lição:** a **cascata**. Cor "inexplicável" costuma ser uma regra do navegador que você não sobrescreveu.

B4. `undefined` × `null` ao editar

- **Sintoma (previsto):** o usuário apaga o "Modelo da capa" e salva, mas o valor antigo continua.
- **Causa:** `|| undefined` faz o campo **sumir do envio**, e o banco entende "não mexa". Só `null` esvazia.
- **Como descobrimos:** ao **raciocinar sobre o envio** antes de o bug aparecer em produção.
- **Lição:** em atualizações parciais, `undefined` = "não mencionei", `null` = "limpe isto".

B5. O filtro que faltava: coleção misturada

- **Sintoma (previsto):** a "Minha coleção" mostraria também revistas **públicas de outras pessoas**.
- **Causa:** a política pública deixava qualquer um (inclusive logado) ler as públicas; `select('*')`

sem filtro traria todas.

- **Como descobrimos:** ao **pensar nas consequências da nova política** antes de ela ir ao ar.
- **Lição:** o RLS decide o que *pode* ser lido; o filtro decide o que *você quer* ver. **Pense no efeito colateral de cada mudança de segurança.**

B6. O `import { use, useState }` que sobrou

- **Sintoma:** o TypeScript acusou uma variável importada e nunca usada.
- **Causa:** muito provavelmente o `use` entrou pelo autocompletar do editor, durante o teste do IntelliJ.
- **Lição:** o rigor do compilador (`noUnusedLocals`) mantém o código limpo. **Aceite os avisos**, não os silencie.

B7. A edição que falhou por causa de uma edição simultânea

- **Sintoma:** uma alteração automática não aplicou, porque o texto procurado não existia mais.
- **Causa:** você tinha mudado o botão de "Apagar" para "Deletar" **ao mesmo tempo**.
- **Lição:** editar o mesmo arquivo em dois lugares gera conflito. **Avise** (ou releia) quando mexer em um arquivo que outra pessoa/ferramenta também edita. Isto é o que o git ajuda a gerenciar.

B8. Teste vermelho de propósito

- **Sintoma:** `Cannot find module './parseCoverFilename'` ao rodar os testes.
- **Causa:** era **intencional**: no TDD, o primeiro passo é ver o teste **falhar**.
- **Lição:** um teste que **nunca** falhou não prova nada. Quebre de propósito para validar.

Grupo C: dados e banco

C1. "O banco está vazio": 0 linhas... que na verdade eram 133 ⚠️ (erro do assistente)

- **Sintoma:** a ferramenta de listagem de tabelas mostrou `rows: 0`. **Afirmei** que não havia dados reais em risco e que podíamos reaproveitar o banco com segurança.
- **Causa:** aquele número vinha de uma **estimativa do painel**, desatualizada. Uma consulta de verdade, `select count(*)`, revelou **133 revistas**.
- **Como descobrimos:** ao preparar o fechamento do cadastro, rodei uma contagem real antes de mexer em qualquer coisa.
- **Consequência:** nada foi alterado até então (só leituras), mas a informação que dei estava errada.
- **Lição (a mais importante do capítulo): estimativas não são fatos.** Antes de afirmar algo sobre dados, rode uma consulta de verdade. E: **diga quando errar**, corrija e siga.

C2. 69 revistas "órfãs" invisíveis

- **Sintoma:** 64 + 0 não somavam 133. Havia 69 revistas **sem dono** que nenhuma conta enxergava.
- **Causa:** `user_id` **não tinha chave estrangeira**. Quando o dono (uma conta antiga) foi apagado, as revistas ficaram, apontando para um usuário que não existe. O RLS só mostra ao dono, então ninguém as via. As imagens delas também tinham sumido do Storage (capas quebradas).
- **Como descobrimos:** uma consulta que juntou `magazines` com `auth.users` (`left join`) mostrou `dono_existe: false`.
- **Lição: chave estrangeira com** `on delete cascade` teria apagado as revistas junto com o usuário (sem órfãs). Dados sem dono são lixo invisível. **Modele as relações no banco, não só na cabeça.**

C3. A consulta voltou `[]` e não um erro

- **Sintoma:** o teste de ataque (sem login) devolveu uma lista vazia, em vez de "acesso negado".
- **Causa:** é assim que o RLS funciona: **filtra em silêncio**. (E as revistas eram privadas.)
- **Lição:** vazio **não** quer dizer "não existe". Se uma consulta volta vazia e você tem certeza de que há dados, **suspeite do RLS primeiro.**

C4. A coluna que existia só no painel

- **Sintoma:** o código usava `is_public`, mas nenhuma migration do repositório original a criava.
- **Causa:** a coluna e a política pública foram criadas **direto no painel**, sem versionar.
- **Lição:** configuração que vive só num painel é configuração que ninguém audita nem consegue **recriar**. Versione o banco com migrations.

C5. A conta apagada antes do delete

- **Sintoma:** depois de pedir ao usuário que rodasse o `delete` das duplicatas, **o banco não tinha mudado** (ainda 69 + 64), mas a conta de agosto tinha **sumido**.
- **Causa:** o usuário tinha removido a conta no painel (um passo opcional do plano) e respondido "tudo ok", mas o `delete` **não foi executado**.
- **Como descobrimos: reli o banco antes de agir**, em vez de confiar na resposta.
- **Lição: verifique o estado real antes de cada passo seguinte.** "Tudo ok" é uma informação; o banco é a verdade.

C6. `DELETE 0` e "Success. No rows returned"

- **Sintoma:** primeiro, um `0`; depois, a mensagem "Success. No rows returned" ao rodar o `delete` certo.
- **Causa:** a mensagem do editor para um `delete` **não diz quantas linhas** foram afetadas. (E um `0` pode indicar que o editor rodou com um **papel restrito** que o RLS filtra.)
- **Como descobrimos:** conferimos com um `select current_user`, depois recontamos as linhas.
- **Lição: verifique depois de apagar.** A mensagem de sucesso não prova que o resultado foi o esperado.

C7. Duas respostas que se contradiziam

- **Sintoma:** o usuário escolheu "mandar as 5 órfãs para a conta de agosto" **e** "a conta final é a de hoje".
- **Causa:** as duas respostas respondiam a perguntas separadas, sem perceber o conflito.
- **O que fizemos: não agimos**; apontamos a contradição, explicamos a consequência e perguntamos de novo.
- **Lição:** diante de instruções conflitantes, **pare e esclareça.** Mexer em dados reais com base num palpite é assumir um risco desnecessário.

C8. Operações em dados reais: o ritual que seguimos

Em todas as mudanças de dados:

1. **Leitura primeiro** (contar, listar, conferir).
2. **Número esperado** declarado antes (`DELETE 64`).
3. **Comando reversível quando possível** (guardamos os donos antigos antes do `update`; salvamos o nome original em `notes` antes de converter os títulos).
4. **Verificação depois.**

Grupo D: segurança

D1. O vazamento de `user_id` na API pública

- **Sintoma:** ao consultar como visitante, a resposta trazia `user_id`.
- **Causa:** o RLS filtra **linhas**, não colunas.
- **Correção:** a view `public_magazines`, seguida da remoção da política antiga, **nesta ordem**.
- **Lição: mínimo privilégio**; e a **ordem** em mudanças de segurança: primeiro adicione o novo, depois remova o velho.

D2. Confirmação de e-mail desligada num site público

- **Sintoma (previsto):** com o cadastro aberto e sem confirmação, qualquer pessoa criaria contas falsas, e o bucket (sem limites) aceitaria arquivos delas.
- **Correção:** cadastro **fechado** e confirmação ligada. O catálogo público não precisa de conta.
- **Lição:** o risco está na **combinação** (cadastro aberto + sem limite no bucket). **Camadas se compensam.**

D3. Segredos na conversa

- **Situação:** durante a configuração, uma chave precisava ir para o Vercel.
- **O que fizemos:** o assistente **não digitou** a chave: copiou para a área de transferência (sem mostrá-la) e o usuário colou.
- **Lição:** o princípio de que **você** entra com credenciais, por mais que seja uma chave pública, é um bom hábito. E sempre se pergunta: *"esta chave pode ser pública?"* (a `publishable` sim, a `service_role` jamais).

Grupo E: processo e comunicação

E1. `git push` e "não há nada para enviar"

- **Sintoma:** o usuário pediu "outro push"; não havia commits novos.
- **Causa:** a edição estava **só na pasta de trabalho**, sem commit. O push só envia commits.
- **Lição:** as três áreas do git (capítulo 11).

E2. O `<h1></h1>` vazio

- **Sintoma:** um commit publicaria uma página sem título.
- **Como descobrimos:** lendo o **diff** antes do commit (`git diff`).
- **Lição: leia o diff antes de commitar.** Uma decisão de design pode ter um efeito colateral (aqui, de acessibilidade). Avisar, e deixar a pessoa decidir, é o certo.

E3. "Vercel não está ligado ao GitHub" (erro do assistente) ⚠️

- **Sintoma:** afirmei que o deploy não era automático e que faltava conectar o repositório.
- **Causa:** a CLI do Vercel **já tinha ligado** o repositório, no momento do `vercel link`. Eu **presumi** sem verificar.
- **Como descobrimos:** o usuário mandou uma captura do painel mostrando o repositório conectado há 50 minutos.
- **Lição: verifique antes de afirmar.** E quando um fato for provado errado, **corrija abertamente**.

E4. O ícone de baixa resolução

- **Sintoma:** o ícone da aba saiu serrilhado.
- **Causa:** a imagem de origem tinha só 276×356 px.
- **Correção:** uma imagem de 840×1279 px, com **recorte do fundo** (um xadrez de dois tons de branco, `237` e `254`, que deixava um "fantasma" quadriculado quando o limiar era frouxo).
- **Lição:** qualidade de saída depende da qualidade de **entrada**. E em processamento de imagem, **olhe o resultado ampliado.**

E5. O que significa "estar pronto"

Em vários momentos, "compilou" e "os testes passaram" **não** bastavam. O que mais pegou erros foi **usar o produto de verdade**: abrir o site, clicar, olhar com dados reais, olhar como visitante, testar o teclado.

As 12 lições que mais valem

1. **Leia a mensagem de erro inteira.** Ela quase sempre diz onde olhar.
2. **O terminal está em algum lugar.** Confira a pasta antes de rodar.
3. **"Compilou" não é "funciona".** Verifique o resultado.
4. **Tipos descrevem o que você acredita, não o que os dados são.** Teste com dados reais.

5. **Estimativas não são fatos.** Conte de verdade.
6. **Vazio não é "não existe".** Suspeite do RLS.
7. **Valide no servidor.** O navegador é território hostil.
8. **Leia o `select` antes do `delete`.** E confira o número esperado.
9. **Verifique o estado real antes de cada passo.** "Tudo ok" não substitui olhar.
10. **Ordem importa:** adicione o novo antes de remover o velho.
11. **Leia o diff antes de commitar.**
12. **Quando errar, diga.** Corrija, aprenda e siga em frente.

Exercício final do capítulo

Escolha **três** casos e, para cada um:

1. Escreva **como você teria detectado** o problema **mais cedo**.
2. Escreva **um teste ou uma verificação automática** que o impediria de voltar. (Ex.: para B2, um teste que confirma que toda classe usada no TSX existe no CSS; para C2, uma restrição de chave estrangeira.)
3. Anote, no **seu diário técnico**, a lição em uma frase.

Próximo: 18. Cola rápida e glossário.

CAPÍTULO 18 DE 18

Cola rápida e glossário

Material de **consulta**: não é para ler do começo ao fim, mas para voltar quando esquecer algo. Cada seção aponta o capítulo onde o assunto é explicado.

Parte 1: Cola rápida

Terminal (cap. 01)

```
pwd                          # onde estou?
ls -la                       # listar (inclui ocultos)
cd ~/dev/vault-keeper        # entrar numa pasta
cd ..                        # subir uma pasta
mkdir -p pasta               # criar pasta
cat arquivo                  # mostrar conteúdo
cp a b  /  mv a b            # copiar / mover (ou renomear)
rm arquivo                   # APAGAR (sem lixeira!)
cmd1 && cmd2                 # roda cmd2 só se cmd1 der certo
```

npm e Vite (cap. 01, 12)

```
npm install                  # instala tudo do package.json
npm install pacote           # instala e registra em dependencies
npm install -D pacote        # registra em devDependencies
npm uninstall pacote
npm run dev                  # servidor local com recarga automática
npm run build                # tsc -b + vite build  →  pasta dist/
npm run preview              # serve o dist/ (o que vai para o ar)
npm test                     # testes uma vez
npm run test:watch           # testes a cada salvamento
npm run lint                 # práticas duvidosas
npm audit                    # vulnerabilidades nas dependências
npx tsc -b                   # só conferir os tipos
```

Antes de cada commit: `npx tsc -b && npm test && npm run build`.

Git (cap. 11)

```
git status -sb               # situação (curta)
git diff                     # o que mudei, linha a linha
git add arquivo              # preparar um arquivo
git add -A                   # preparar TUDO (confira antes!)
git commit -m "mensagem"     # tirar a foto
git log --oneline            # histórico
git push                     # enviar ao GitHub
git pull                     # trazer do GitHub
git restore arquivo          # DESCARTAR minhas edições (sem volta)
git restore --staged arquivo # desfazer o add
git revert <código>          # desfazer um commit já enviado (cria outro)
git switch -c nome           # criar e entrar numa branch
```

Símbolos do `status`: `M` modificado · `??` novo · `A` preparado · `[ahead 1]` 1 commit sem enviar.

JavaScript (cap. 02)

```
const { a, b } = obj                // desestruturar
const copia = { ...obj, b: 2 }      // copiar e trocar um campo
const nova = [x, ...lista]          // copiar e acrescentar no início
obj?.campo                          // sem erro se obj for null/undefined
valor ?? 'padrão'                   // padrão só para null/undefined
lista.map(x => ...)                 // transforma cada item
lista.filter(x => ...)              // mantém os que passam
lista.find(x => ...)                // primeiro que passa (ou undefined)
lista.some(...) / .every(...)       // algum / todos passam?
lista.includes(x)                   // contém?
[...lista].sort((a, b) => a - b)    // ordenar SEM alterar a original
lista.filter(Boolean)               // tira null/undefined/''/0
texto.trim().toLowerCase()
texto.normalize('NFD').replace(/\p{Diacritic}/gu, '')   // tira acentos
String(3).padStart(2, '0')          // '03'
try { await f() } catch (e) { ... } // tratar erro de async
```

TypeScript (cap. 03)

```
type Condition = 'A' | 'B'                  // união literal
interface X { a: string; b?: number | null } // ? = opcional
Omit<X, 'a'>        // sem o campo a
Partial<X>          // tudo opcional
Pick<X, 'a'>        // só o campo a
Record<K, V>        // objeto com chaves K e valores V
useState<X[]>([])   // genérico
function f(a: string): Promise<X> { }
import type { X } from './x'
valor as Condition  // afirmar um tipo (use com cuidado)
```

React (cap. 04, 05, 06)

```
function Comp({ a, onX }: Props) { return <div>{a}</div> }        // componente + props
{cond && <A />}   {cond ? <A /> : <B />}   if (!x) return <C />   // condicional
{lista.map(i => <Item key={i.id} item={i} />)}                     // lista (key = id!)
<button onClick={() => fazer(id)}>   <form onSubmit={fn}>         // eventos
<input value={v} onChange={e => setV(e.target.value)} />          // controlado
event.preventDefault()   event.stopPropagation()

const [x, setX] = useState(valorInicial)       // estado
setX(prev => prev + 1)                          // forma funcional
setLista([novo, ...lista])  // adicionar
setLista(lista.filter(i => i.id !== id))        // remover
setLista(lista.map(i => i.id === id ? { ...i, c: v } : i))   // trocar um

useEffect(() => {
  // faz algo
  return () => { /* limpeza */ }
}, [dependencias])          // [] = uma vez

const ref = useRef<HTMLDivElement>(null)       // guardar sem redesenhar
```

Hooks: regras
Só no topo do componente (nunca em `if`/loop) e só em componentes ou outros hooks
`return` antecipado **depois** de todos os hooks

```
// Rotas
<BrowserRouter><Routes>
  <Route path="/" element={<Home />} />
  <Route path="*" element={<NotFound />} />
</Routes></BrowserRouter>
<Link to="/x">   <NavLink to="/" end>
```

CSS (cap. 07)

```
:root { --cor: #000; }   var(--cor)                        /* variáveis */
*, *::before, *::after { box-sizing: border-box; }         /* reset essencial */
display: flex; justify-content: center; align-items: center; gap: 1rem;
display: grid; grid-template-columns: repeat(4, minmax(0, 1fr));
grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));   /* grade que se adapta */
font-size: clamp(1rem, 3vw, 2rem);                         /* fluido */
aspect-ratio: 3 / 4;   object-fit: cover;
position: sticky; top: 0;   position: fixed; inset: 0;
@media (max-width: 820px) { ... }
@media (hover: hover) { ... }   @media (prefers-reduced-motion: reduce) { ... }
transition: transform .3s ease;   transform: scale(1.05);
:hover  :focus-visible  :focus-within  [aria-pressed='true']
```

Centralizar: `display:flex; align-items:center; justify-content:center` no pai.

SQL e Supabase (cap. 08, 09)

```sql
select * from tabela where col = 'x' order by criado desc;
select count(*) filter (where is_public) from tabela;
insert into tabela (a, b) values ('x', 1);
update tabela set a = 'y' where id = '...';
delete from tabela where id = '...';        -- SEMPRE com where; rode o select antes!
where coluna is null           -- (NÃO "= null")
a is not distinct from b       -- igualdade que trata NULL como valor
```

```
const { data, error } = await supabase.from('t').select('*').eq('col', v).order('c', { ascending: false })
if (error) throw error
.insert({...}).select().single()      .update({...}).eq('id', id)      .delete().eq('id', id)
supabase.auth.signInWithPassword({ email, password })    supabase.auth.signOut()
supabase.storage.from('bucket').upload(path, file)       .getPublicUrl(path)     .remove([path])
```

Modelos de política RLS

```sql
alter table public.t enable row level security;

create policy "dono lê"     on public.t for select using (auth.uid() = user_id);
create policy "dono cria"   on public.t for insert with check (auth.uid() = user_id);
create policy "dono altera" on public.t for update using (auth.uid() = user_id);
create policy "dono apaga"  on public.t for delete using (auth.uid() = user_id);

-- Storage: só a própria pasta
... using (bucket_id = 'b' and auth.uid()::text = (storage.foldername(name))[1])

-- Limites do bucket (servidor)
update storage.buckets set file_size_limit = 5242880,
  allowed_mime_types = array['image/jpeg','image/png','image/webp'] where id = 'b';
```

Vitest (cap. 10)

```
import { describe, expect, it } from 'vitest'
describe('assunto', () => {
  it('faz tal coisa', () => {
    expect(f(entrada)).toEqual(saida)       // objetos/arrays
    expect(x).toBe(3)                       // valores simples
    expect(obj).toMatchObject({ a: 1 })     // contém pelo menos
    expect(lista).toHaveLength(2)
  })
})
```

TDD: 🔴 teste falha → 🟢 código mínimo → 🔵 refatora.

Vercel (cap. 12)

```
npx vercel@latest login | link | ls
npx vercel@latest env add NOME production
npx vercel@latest deploy --prod
```

Variáveis `VITE_*` são gravadas no build e ficam **visíveis**; mudou? **Novo deploy.**

Atalhos do IntelliJ (Mac)

`⌘⇧O` abrir arquivo · `⌘B` ir à definição · `⇧F6` renomear com segurança · `⌥⏎` correções · `⌘K` commit · `⌘⇧K` push · `⌥F12` terminal · `⌘/` comentar · `⌘⇧F` buscar no projeto.

Diagnóstico rápido

Sintoma	Primeira coisa a olhar
`ENOENT package.json`	Pasta do terminal
Tela em branco no ar	Console do navegador; variáveis de ambiente
Lista vazia mas há dados	RLS / papel do SQL Editor
Tela não atualiza	Alterou estado direto em vez de criar novo
404 ao recarregar	`rewrites` no `vercel.json`
Classe CSS "não pega"	Nome diferente entre CSS e TSX; inspecione
Cor de link estranha	Defina `color` (regra do navegador)
Ícone da aba antigo	Cache do navegador
Push "não enviou nada"	Faltou o commit

Parte 2: Glossário (A a Z)

a11y: abreviação de *accessibility* (acessibilidade).

Alias (Vercel): endereço amigável que aponta para a versão de produção atual.

anon / publishable key: chave **pública** do Supabase; só abre o que o RLS autoriza.

API: conjunto de "pedidos" que um sistema aceita. A API do Supabase é o que o `supabase-js` chama.

Arrow function: função escrita com `=>`. `(a) => a * 2`.

async / await: forma de escrever código que espera operações demoradas (internet) sem travar a página.

Auth: o serviço de login e cadastro (Supabase Auth).

Branch: uma "linha paralela" de commits para trabalhar sem afetar a principal.

Build: o processo que transforma seu código-fonte na versão final e otimizada (`dist/`).

Bucket: uma "pasta raiz" de arquivos no Storage.

Cache: cópia guardada para carregar mais rápido; pode mostrar versões antigas.

Cascata (CSS): regras de quem vence quando duas regras mexem na mesma coisa.

CDN: rede de servidores espalhados pelo mundo que entregam seus arquivos rapidamente.

Closure: uma função que "lembra" as variáveis do lugar onde nasceu.

Commit: uma "fotografia" do projeto, com mensagem, guardada no histórico do git.

Componente: função React que devolve um pedaço de tela.

CRUD: Create, Read, Update, Delete: as quatro operações básicas sobre dados.

CSS: linguagem de estilo (cores, tamanhos, posição).

Deploy: publicar uma versão do app no ar.

DOM: a árvore de elementos da página, que o navegador mantém e o React atualiza.

Efeito (`useEffect`): código que sincroniza o React com algo de fora (rede, teclado, armazenamento).

Estado (`useState`): memória do componente; quando muda, a tela é redesenhada.

Estado derivado: valor **calculado** a partir de outros, em vez de guardado.

Flexbox: layout em uma dimensão (linha ou coluna).

Frontend / Backend: o que roda no navegador / o que roda no servidor.

Função pura: mesma entrada → mesma saída, sem efeitos colaterais; fácil de testar.

Git / GitHub: sistema de histórico de código / site que guarda uma cópia na nuvem.

Grid: layout em duas dimensões.

HMR: *Hot Module Replacement*, a atualização instantânea da página ao salvar no `npm run dev`.

Hook: função do React (`useState`, `useEffect`...) ou criada por você (`useSession`) que dá estado/efeitos a um componente.

Imutabilidade: nunca alterar um valor existente; criar uma cópia alterada.

Interface (TypeScript): descrição do formato de um objeto.

JSON: formato de texto para dados (`{ "a": 1 }`).

JSX: sintaxe de HTML dentro do JavaScript.

JWT: texto assinado que prova quem é o usuário; o Supabase o envia em cada pedido.

key (React): identificador estável de um item numa lista.

Lint: verificação automática de práticas duvidosas (`oxlint`).

localStorage: pequeno armazenamento do navegador que sobrevive a fechar a aba.

Migration: arquivo SQL versionado que descreve uma mudança do banco.

Minificação: remover espaços e encurtar nomes para o arquivo ficar menor.

Modal: janela que aparece por cima da página.

Mock: substituto falso de uma peça (banco, rede) para testar.

`null` × `undefined`: "vazio de propósito" × "nunca recebeu valor".

Open redirect: ataque que usa um redirecionamento para levar a vítima a um site falso.

Órfão: dado que perdeu o dono ou a referência (revista sem usuário, arquivo sem linha).

Postgres: o programa de banco de dados relacional usado pelo Supabase.

Primary key: coluna que identifica uma linha de forma única.

Props: dados que o pai entrega ao componente filho.

Pull Request: pedido para juntar uma branch na principal, com revisão.

Query builder: a cadeia `.from().select().eq()` que monta consultas.

Reset (CSS): regras iniciais que zeram diferenças entre navegadores.

Responsivo: layout que se adapta ao tamanho da tela.

RLS (*Row Level Security*): regras do banco que filtram quais linhas cada usuário pode ver ou alterar.

Rota: um endereço (`/catalog`) associado a uma tela.

Scroll snap: "ímã" que alinha o conteúdo ao parar de rolar.

Semântico (HTML): usar a tag que descreve o que o elemento **é** (`<nav>`, `<button>`).

service_role: chave **secreta** do Supabase que ignora o RLS. Jamais vai ao navegador.

SPA: *Single Page Application*, um site de uma única página HTML que troca o conteúdo por JavaScript.

SQL: linguagem para conversar com bancos relacionais.

Storage: o serviço de arquivos do Supabase.

StrictMode: ajuda do React em desenvolvimento que roda efeitos duas vezes para revelar bugs.

Tabela / Linha / Coluna: a estrutura de um banco relacional.

TDD: *Test-Driven Development*, escrever o teste antes do código.

Token (CSS): variável de design (`--bg`, `--text`).

Tree-shaking: o build descarta o código importado e não usado.

TypeScript: JavaScript com tipos, que somem na compilação.

União literal: tipo que aceita só certos valores (`'a' | 'b'`).

UUID: identificador longo e único (`a1b2c3d4-...`).

Variável de ambiente: configuração guardada fora do código (`.env.local`).

View (SQL): consulta salva que se comporta como uma tabela; aqui, para expor só colunas seguras.

Vite: ferramenta que serve o app em desenvolvimento e o empacota para produção.

Vitest: executor de testes do ecossistema Vite.

XSS: ataque em que um script do atacante roda no navegador de outra pessoa dentro do seu site.

Onde continuar aprendendo

Assunto	Fonte gratuita e confiável
HTML, CSS, JavaScript	**MDN Web Docs** (`developer.mozilla.org/pt-BR`)
React	**react.dev** (a documentação oficial é excelente, com exercícios)
TypeScript	**typescriptlang.org/docs**
Supabase	**supabase.com/docs**
SQL	Tutoriais de PostgreSQL; pratique no SQL Editor de um projeto de estudo
Git	**git-scm.com/book/pt-br**
Acessibilidade	**web.dev/learn/accessibility**
Segurança web	**OWASP Top 10** (`owasp.org`)

Próximo passo sugerido: volte ao README, escolha a semana em que está e siga o roteiro de 10 semanas. Quando chegar ao capítulo 15, construa o seu.